



A Computational Study of the Vortex Particle Method: Pressure Recovery and Numerical Ramifications

Tryfonas Themass

Submitted in partial fulfillment of the requirements for the degree of
Mechanical Engineer

National Technical University of Athens

Fluids Section

Advisor: Professor Spyridon Voutsinas

Athens, 2025

This page is intentionally left blank.

Acknowledgments

First and foremost in my acknowledgments comes Professor Spyros Voutsinas, my academic advisor and mentor for this thesis and for my academic journey. The support and guidance he provided me with during my studies has been incalculable and without him this thesis would not be possible. I would thus want to extend my most earnest gratitude and appreciation for his guidance and support for the past years.

I first met Professor Voutsinas, asking for his advice when trying to perform a post-mortem analysis on a UAV we had designed as EUROAVIA Athens, which had failed to fly due to dynamic instabilities. Initially his deep knowledge overwhelmed me, but his perseverance to put up with me helped transform this first meeting into a lasting relationship. Under the auspices of solving this question began for me a multiyear endeavor on the study of aerodynamics and fluid mechanics.

I am deeply appreciative of the time he dedicated to mentoring me during my academic journey, often going far above what is normally expected. His deep knowledge and expertise has not only inspired me but also shown me the immense depth of the engineering science. Whether it be through his guidance in solving problems or through long theoretical and technical discussions he has made a deep imprint in my way of thought and in the way I see this field. His mentorship has been a learning experience for me, helping shape me into who I am today.

I would also like to thank him for his patience, eagerly and willingly addressing my inquiries. His thoughtful advice and genuine encouragement has helped motivate me throughout my work. I feel lucky to have had the chance to learn from him. He has continuously been an inspiring figure throughout my studies, and I truly consider him to be my academic father figure. I sincerely hope that he will continue to guide and teach me in the future.

Last but not least, I am also deeply grateful to my friends and family for their constant support, encouragement and belief in me.

This page is intentionally left blank.

Contents

Contents	v
1 Introduction	1
1.1 Motivation	1
1.2 Historical Overview	3
1.3 Thesis Outline	5
2 Definitions and Governing Equations	7
2.1 Eulerian Formulation of the Navier Stokes	7
2.1.1 Continuity	7
2.1.2 Momentum	7
2.2 Pressure for an Incompressible Fluid	8
2.3 Helmholtz Decomposition	10
2.4 Lagrangian Formulation of the Navier-Stokes	13
2.4.1 Lagrangian Description	13
2.4.2 Particle Approximations	14
2.4.3 Reynolds Transport Theorem	14
2.4.4 Mass Continuity Equation	15
2.4.5 Vorticity and Dilatation Equations	16
3 The Vortex Particle Method for Incompressible flows	20
3.1 Function Approximation	21
3.2 Flow Equations	25
3.3 Integral Representation and Green's Function.	25
3.4 Particle–Mesh (PM)	27
3.5 Remeshing	29
3.6 Poisson Solvers	33
3.6.1 FFT-Based Solvers	34
3.6.2 Cyclic Reduction Algorithm	34
3.6.3 Multigrid Solvers	35
3.7 Viscous Splitting	39
3.8 Pressure Recovery	40
3.9 Vorticity Correction	42
3.10 The VPM Algorithm	43

4	Numerical Analysis of the PM Method	45
4.1	Fourier and Von Neumann Analysis	46
4.2	Fourier Symbol of a Discrete Operator	46
4.3	Diffusion Equation	48
4.4	Vorticity Correction Schemes	55
5	Compact Finite-Difference Schemes: Improved Spectral Accuracy	60
5.1	Overview of Compact Finite Difference Schemes	60
5.2	Fourier Symbol and Spectral Accuracy	62
5.3	Modified Stencil Compact Schemes	63
5.3.1	Fourier Analysis	65
5.3.2	Designing the Right-Hand Sides	66
5.3.3	The Algorithm	68
6	Validation and Verification	70
6.1	Validation of Diffusion Schemes	70
6.1.1	Ricker Wavelet (Mexican Hat)	70
6.1.2	Diffusion of the Ricker Wavelet in Time	72
6.1.3	Numerical Results	76
6.2	Validation of the VPM algorithm	79
6.2.1	Hill's Vortex	80
6.2.2	Parametric Numerical Study on the Evolution of Hill's Vortex	83
6.2.3	Numerical Results	84
6.3	Validation of Pressure Field Calculation	90
6.3.1	The Lamb-Oseen Vortex	90
6.3.2	Numerical Results	93
	Developed Software	98
	Appendix A	100
	Appendix B	104

Nomenclature

$\nabla \cdot$	The divergence operator
ρ	The fluid density
σ	The deviatoric stress tensor
$\vec{u}$	The flow velocity field
I	The identity tensor
p	The flow pressure field
t	The time

Chapter 1

Introduction

1.1 Motivation

Computational Fluid Dynamics (CFD) has historically been dominated by *Eulerian* grid-based methods (finite volume, finite difference, etc.), which solve the Navier–Stokes equations on a mesh. These methods have well-known strengths, such as robust enforcement of solid boundary conditions and a long track record across diverse flow problems. With sufficient grid resolution, Eulerian solvers can achieve high-fidelity results. However, they also suffer from inherent limitations. For instance, the computational domain must be truncated at a finite outer boundary, requiring artificial far-field conditions. Moreover, capturing long wakes or vortical structures demands either very large domains or special treatments to avoid excessive numerical diffusion as the grid is coarsened far from the body. Multi-body or moving boundary problems can further complicate mesh generation and introduce interpolation errors. These issues make certain flow scenarios (notably, those involving extensive wake development or multiple moving parts) particularly challenging for a purely Eulerian approach.

In contrast, *Lagrangian* particle methods offer a complementary perspective by representing the fluid as moving computational elements. One prominent Lagrangian technique is the *Vortex Particle Method* (VPM), in which the vorticity field is discretized into particles that convect with the flow. Lagrangian methods are *mesh-free* and inherently adapt to the flow: computational elements naturally cluster in regions of intense vorticity and sparse out where the flow is quiescent.

A key advantage of vortex methods in CFD is the reduction of numerical diffusion during convective transport. In grid-based methods, convective terms must be discretized and often introduce artificial diffusion or dispersion errors; by contrast, a vortex particle scheme (in theory) simply carries vorticity along particle trajectories, essentially advecting information with zero inherent numerical dissipation. Additionally, the absence of a static mesh means that certain stability limitations are relaxed. In particular, as noted by Anderson and Greengard [1], the time-step in a purely Lagrangian vortex simulation is not bound by the usual Courant–Friedrichs–Lewy condition based on flow speed and grid size, but rather by a milder criterion related to local strain rates (sometimes called a “Lagrangian CFL” limit). This permits larger time-steps in many scenarios, enhancing efficiency.

Last but not least, particle methods can embed the true far-field (infinite domain) condition exactly, eliminating the need for artificial outer boundaries.

Despite their appealing advantages, vortex particle methods face their own computational and numerical challenges. A central difficulty lies in handling solid boundaries and viscosity. Imposing no-slip boundary conditions on a surface with a particle method often requires generating vorticity-carrying particles to represent the boundary layer or coupling to a separate boundary element method for the potential flow induced by the solid. Purely Lagrangian simulations have historically struggled with high-Reynolds-number wall-bounded flows because resolving the thin boundary layers would demand an exceedingly large number of particles near surfaces.

Moreover, maintaining a well-resolved vorticity field over long simulation times may require occasional redistribution of particles, commonly referred to as *remeshing*. Without remeshing, particle clustering or depletion can occur in certain regions, leading to loss of accuracy or the creation of spurious small-scale vorticity due to particle overlap issues. Remeshing techniques periodically project the particle data onto an underlying mesh and then interpolate back to a new set of particles, which helps to maintain a roughly uniform particle distribution and control particle distortion. While effective, this procedure introduces a subtle blending of Lagrangian and Eulerian viewpoints, and it must be designed carefully to avoid introducing excessive diffusion or violating conservation laws.

Among all these issues, an especially important and relatively less explored aspect of vortex particle methods is the accurate recovery of the *pressure* field. In traditional Eulerian CFD, pressure is computed as part of the solution procedure at each time step, ensuring consistency with the momentum equations and boundary conditions. In a VPM however, the primary transported quantity is vorticity (or circulation strength of particles), and pressure does not naturally emerge from the Lagrangian formulation without additional effort.

Recovering the pressure in a vortex methods involves solving a pressure Poisson equation on an auxiliary grid or applying an integral formulation, using the particle velocities and vorticity as sources. This can be numerically delicate and computationally intensive, because pressure depends on spatial gradients of the velocity field; any small error in the particle representation of the velocity can be magnified in the pressure solution. Nonetheless, pressure information is crucial for several reasons: it is needed to evaluate force and moment coefficients on immersed bodies, to assess acoustic emissions, and, importantly, to serve as a coupling quantity in hybrid Eulerian–Lagrangian simulations.

The prospect of *hybrid Eulerian–Lagrangian* approaches has become a strong motivation for incorporating pressure computation in VPMs. Hybrid methods seek to combine the strengths of Eulerian and Lagrangian solvers within a single framework. For example, one may use a conventional Eulerian solver (such as a Navier–Stokes finite-volume method) in the near-field region close to solid boundaries, where com-

plex geometry and viscous boundary layers are present, while using a vortex particle method in the far-field wake region, where vorticity is convected over long distances. Whichever coupling strategy is used, it is clear that without a reliable way to compute pressure from the vortex method side, the advantages of hybrid schemes cannot be fully realized. Thus, an accurate and efficient pressure computation in vortex particle methods is a key enabling technology for next-generation CFD schemes that marry Eulerian and Lagrangian techniques.

In summary, the vortex particle method offers significant benefits for simulating certain classes of flows (chiefly those with strong vorticity dynamics in unbounded domains), but it also presents unique challenges. This thesis is motivated by those challenges, in particular the problem of pressure recovery in incompressible flows, which constitutes a post-processing or *a-posteriori* computation. Additionally, we explore the numerical ramifications of schemes employed in Particle Mesh, in terms of the Laplace operator, and propose a method for computing accurate spectral derivatives, while incurring moderate cost. By addressing how to obtain the pressure consistently from a particle-based flow solution, we aim to enhance the applicability of vortex methods both as stand-alone solvers and as components in hybrid CFD frameworks. The following sections provide a brief historical context for vortex methods and outline the approach and organization of this work.

1.2 Historical Overview

The concept of solving fluid flow in a vorticity-carrying Lagrangian framework dates back several decades. Early *vortex method* formulations in the 1970s and 1980s (starting with the pioneering work of Chorin [7, 6]) demonstrated that it was possible to simulate incompressible, inviscid flows by following discrete vortex elements. These methods showed great promise for capturing vortex dynamics without the numerical diffusion inherent in grid-based schemes. In the late 1970s, the concept of *smoothed particle hydrodynamics* (SPH) was developed for astrophysical flow problems and quickly found applications in fluid dynamics. Gingold and Monaghan's work [11] exemplified how Lagrangian particles with kernel-based smoothing could successfully model compressible flows and complex free-surface phenomena.

Over the years, vortex methods were extended and refined: core spreading techniques were introduced to model diffusion, and the concept of *particle strength exchange* [9, 10] was developed to incorporate viscous effects in a particle context. By the 1990s, the *Vortex Particle Method* had emerged as a robust mesh-free technique for simulating incompressible flows, supported by a rigorous mathematical foundation (e.g. convergence theory [13, 2], and error analysis) and improved numerical algorithms. Comprehensive expositions, such as Cottet and Koumoutsakos (2000) [8], helped formalize the methodology, covering aspects like smooth particle kernels, particle regularization of the Biot–Savart law, and remeshing strategies to maintain a quasi-uniform particle distribution.

Over time, researchers applied VPM to increasingly complex scenarios. Vortex particle simulations have been successfully used to model the wake flows of fixed-wing aircraft [5], rotorcraft [27], and wind turbines [26], where large coherent vortices play a dominant role in the far-field flow. In many such applications, VPM has been coupled with potential flow models or Boundary Element Methods (BEM) to satisfy boundary conditions on solid surfaces. For instance, a typical approach for rotor aerodynamics is to use a blade element or panel method to compute the lift on the rotor blades (thus obtaining the shed vorticity or circulation) and then convect that vorticity into the far wake using a vortex particle simulation. This separation of tasks leverages the strengths of each method: the panel method or Eulerian near-field solver enforces the blade surface boundary conditions and computes pressure loads, while the vortex method handles the convection of vorticity in the wake without numerical diffusion. However, this partition also highlights the historical fact that vortex methods on their own rarely computed the pressure field explicitly. Instead, pressure-related quantities (like forces) were obtained from the auxiliary potential-flow solution on the solid boundaries, not from the particle simulation itself.

The difficulty of incorporating viscosity and wall effects directly into vortex particle methods led to ongoing research into hybrid methods. Notably, *viscous-inviscid coupling* techniques were developed, wherein a thin boundary-layer or Navier–Stokes solver is applied in a region near the wall, and an inviscid Lagrangian vortex method is applied outside that region. Pioneering studies in the late 20th century and early 2000s demonstrated this approach on two-dimensional airfoils and simple 3D configurations. Similarly, coupling a RANS solver (solving the Reynolds-averaged Navier–Stokes equations on a grid) with a vortex particle wake model has been explored as a way to efficiently simulate high-Reynolds-number flows around lifting bodies. These studies reported that the hybrid approach can drastically reduce the computational cost compared to a full-domain Eulerian simulation, without sacrificing accuracy in predicting global quantities like lift and drag, as long as the coupling between the solvers is handled carefully.

A critical lesson from these efforts has been the importance of exchanging pressure information consistently. In strong coupling schemes, the pressure along the interface must be shared between the Eulerian and Lagrangian parts to ensure momentum balance and avoid double-counting or omission of forces. This realization has brought the issue of pressure computation in vortex methods to the forefront. Recent works, such as those by Papadakis [23], have implemented such hybrid CFD methodologies in the context of compressible flows, but an extension to incompressible flows is needed.

1.3 Thesis Outline

This thesis is organized into several chapters that collectively address the motivations and objectives outlined above.

Chapter 2 (*Definitions and Governing Equations*) establishes the theoretical foundation for our study. In this chapter, the fundamental equations of incompressible fluid flow are derived and reviewed in both Eulerian and Lagrangian frameworks. We begin with the Navier–Stokes equations in their conventional form (conservation of mass and momentum in an Eulerian description) and then reformulate them in vorticity form. The chapter introduces the passage from the velocity–pressure formulation to the vorticity–streamfunction formulation, highlighting how the pressure term can be eliminated from the vorticity transport equation. It also defines key concepts such as vorticity, circulation, and the Helmholtz decomposition, which are essential for formulating the Vortex Particle Method.

Chapter 3 (*The Vortex Particle Method for Incompressible Flows*) provides a comprehensive overview of the vortex particle algorithm that is central to this work. The chapter describes how the continuous vorticity field is discretized into particles and details the numerical operations that constitute a complete VPM simulation. These include the choice of smoothing kernel for particle interpolation, the computation of velocity from vorticity (using the streamfunction or Biot–Savart law), and the treatment of vortex stretching and diffusion in the Lagrangian context. Special attention is given to the methodology for *pressure recovery*: we outline the developed two-step procedure in which an intermediate field is obtained by solving a Poisson equation on a mesh, followed by an algebraic relation to compute the pressure. The chapter also discusses practical algorithmic considerations such as time-stepping schemes, the enforcement of incompressibility, and remeshing techniques to preserve particle distribution quality.

Chapter 4 (*Numerical Analysis of the Particle Method*) delves into the stability and error analysis of the particle mesh schemes. In this chapter, we rigorously analyze how various numerical components (advection, diffusion) influence the accuracy and stability of the method. A Fourier (von Neumann) analysis is performed on simplified model equations to determine growth or decay rates of errors across different wavelengths, which provides insight into the dissipative and dispersive characteristics introduced by our numerical schemes. We also examine convergence properties, drawing on the theoretical results of prior researchers, and we identify the conditions under which the vortex particle method can be expected to produce reliable results.

Chapter 5 (*An Efficient Compact Finite-Difference Scheme for Improved Spectral Accuracy*) introduces a novel numerical scheme to address

one of the challenges identified in the preceding analysis: the computation of derivatives that are resistant to numerical noise. The chapter proposes a high-order compact finite-difference discretization for the Laplacian operator on the mesh used for pressure and diffusion calculations. Compared to standard explicit finite-difference schemes (which can impose severe stability limits and error due to large eigenvalues in their spectral representation), the compact scheme achieves higher accuracy and more favorable spectral properties. We derive the scheme’s formulation, demonstrate its improved resolution of diffusive processes, and discuss how it is incorporated into the particle–mesh framework of the VPM. By using this scheme, the numerical diffusion of vorticity is more faithfully captured, and the allowable time step for explicit integration can be increased while maintaining stability.

Chapter 6 (*Validation and Verification*) presents a series of numerical experiments designed to verify the correctness and assess the performance of the developed vortex particle method. We validate the pressure recovery procedure and the overall solver by comparing numerical results to known analytical solutions (Lamb-Oseen vortex). Additionally we manufacture a solution (a three-dimensional Gaussian or “Mexican hat” vorticity profile) where an exact solution for velocity evolution is derived for comparison. We also test the complete method on more complex flow scenarios (Hill Vortex) to demonstrate its capability in capturing detailed flow physics. Key results – such as the accuracy of the recovered pressure field, the conservation of momentum and enstrophy, and the stability of the method over long integration times – are reported and discussed.

Additional derivations and detailed data supporting this work are provided in the Appendices, which include mathematical proofs and supplementary figures that would be otherwise too lengthy for the main text.

Chapter 2

Definitions and Governing Equations

This chapter rigorously derives the governing equations for fluid flow in two complementary frameworks: the Eulerian formulation (based on the Navier–Stokes equations) and the Lagrangian formulation (which is well suited for particle methods such as the Vortex Particle Method, VPM). In the Eulerian framework, we derive the conservation of mass, momentum, and the associated pressure Poisson equation. Then, by introducing a Helmholtz decomposition of the velocity field, we simplify the treatment of the pressure term. Next, we transition to the Lagrangian framework by defining particle approximations and applying the Reynolds Transport Theorem to derive the time-dependent Navier-Stokes equations.

2.1 Eulerian Formulation of the Navier Stokes

The governing equations for fluid flow are the Navier-Stokes equations, which for an incompressible fluid are derived from the conservation of mass and momentum.

2.1.1 Continuity

The continuity equation can be expressed as:

$$\frac{\partial \rho}{\partial t} + \nabla(\rho \mathbf{u}) = 0 \Rightarrow \frac{\partial \rho}{\partial t} + (\mathbf{u} \cdot \nabla)\rho = -\rho(\nabla \cdot \mathbf{u})$$

For an incompressible fluid this becomes:

$$\nabla \cdot \mathbf{u} = 0 \tag{2.1}$$

2.1.2 Momentum

The momentum equation can be expressed as:

$$\frac{\partial(\rho \mathbf{u})}{\partial t} + \nabla(\mathbf{u} \otimes (\rho \mathbf{u})) = \nabla \cdot (-pI + \boldsymbol{\sigma}) \tag{2.2}$$

Using the product rule and the identity:

$$\nabla \cdot (\mathbf{a} \otimes \mathbf{b}) = (\nabla \cdot \mathbf{a})\mathbf{b} + \mathbf{a} \cdot \nabla \mathbf{b}$$

we rewrite the convective term in (2.2) as:

$$\begin{aligned} \rho \frac{\partial(\mathbf{u})}{\partial t} + \mathbf{u} \frac{\partial \rho}{\partial t} + (\nabla \mathbf{u}) \cdot \rho \mathbf{u} + \mathbf{u} \cdot \nabla \rho \mathbf{u} &= \nabla \cdot (-p\mathbf{I} + \boldsymbol{\sigma}) \\ \Rightarrow \mathbf{u} \left(\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) \right) + \rho \left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} \right) &= \nabla \cdot (-p\mathbf{I} + \boldsymbol{\sigma}) \end{aligned} \quad (2.3)$$

The leftmost expression that is in parentheses is the mass conservation equation (2.1.1) and therefore zero by definition. So:

$$\rho \left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} \right) = \nabla \cdot (-p\mathbf{I} + \boldsymbol{\sigma}) \quad (2.4)$$

By employing the identity for the vector triple product:

$$\mathbf{a} \times (\mathbf{b} \times \mathbf{c}) = (\mathbf{a} \cdot \mathbf{c})\mathbf{b} - (\mathbf{a} \cdot \mathbf{b})\mathbf{c}$$

We can rewrite the inertial term as:

$$\mathbf{u} \cdot \nabla \mathbf{u} = \nabla \left(\frac{\mathbf{u}^2}{2} \right) - \mathbf{u} \times (\nabla \times \mathbf{u})$$

Substituting into (2.4) the momentum equation becomes:

$$\Rightarrow \frac{\partial \mathbf{u}}{\partial t} + \nabla \left(\frac{\mathbf{u}^2}{2} \right) - \mathbf{u} \times (\nabla \times \mathbf{u}) = \frac{1}{\rho} \nabla \cdot (-p\mathbf{I} + \boldsymbol{\sigma})$$

Defining $\nabla \times \mathbf{u} = \boldsymbol{\omega}$, the momentum equation becomes:

$$\frac{\partial \mathbf{u}}{\partial t} + \nabla \left(\frac{\mathbf{u}^2}{2} \right) - \mathbf{u} \times \boldsymbol{\omega} = \frac{1}{\rho} \nabla \cdot (-p\mathbf{I} + \boldsymbol{\sigma}) \quad (2.5)$$

2.2 Pressure for an Incompressible Fluid

If we apply the divergence operator to the momentum equation (2.5) we get:

$$\nabla \cdot \left(\frac{\partial \mathbf{u}}{\partial t} + \nabla \left(\frac{\mathbf{u}^2}{2} \right) - \mathbf{u} \times \boldsymbol{\omega} \right) = \nabla \cdot \left(\frac{1}{\rho} \nabla \cdot (-p\mathbf{I} + \boldsymbol{\sigma}) \right)$$

Since $\nabla \cdot pI = \nabla p$

$$\Rightarrow \frac{\partial \nabla \cdot \mathbf{u}}{\partial t} + \nabla^2 \left(\frac{\mathbf{u}^2}{2} \right) - \nabla \cdot (\mathbf{u} \times \boldsymbol{\omega}) = \frac{1}{\rho} (-\nabla^2(p) + \nabla \cdot \nabla \cdot \boldsymbol{\sigma})$$

Finally we have the following Poisson equation for the pressure:

$$\Rightarrow \nabla^2 p = -\rho \left(\frac{\partial \nabla \cdot \mathbf{u}}{\partial t} + \nabla^2 \left(\frac{\mathbf{u}^2}{2} \right) - \nabla \cdot (\mathbf{u} \times \boldsymbol{\omega}) \right) + \nabla \cdot \nabla \cdot \boldsymbol{\sigma}$$

Since $\nabla \cdot \mathbf{u} = 0$ (2.1.1) we have:

$$\begin{aligned} \nabla^2 p &= -\rho \left[\nabla^2 \left(\frac{\mathbf{u}^2}{2} \right) - \nabla \cdot (\mathbf{u} \times \boldsymbol{\omega}) \right] + \nabla \cdot \nabla \boldsymbol{\sigma} \\ &= -\rho \nabla \cdot (\mathbf{u} \cdot \nabla \mathbf{u} + \nabla \cdot \boldsymbol{\sigma}) \\ &= -\rho \nabla \cdot (\mathbf{u} \cdot \nabla \mathbf{u}) + \nabla \cdot \nabla \cdot \boldsymbol{\sigma}. \end{aligned} \tag{2.6}$$

For an incompressible Newtonian fluid with *constant* viscosity, the (viscous) stress tensor $\boldsymbol{\sigma}$ can be written as

$$\boldsymbol{\sigma} = 2\mu \mathbf{D} = \mu(\nabla \mathbf{u} + (\nabla \mathbf{u})^T),$$

where $\mathbf{D}$ is the symmetric rate-of-strain tensor: $\nabla^2 \boldsymbol{\sigma}$ in (2.6) simplifies to

$$\nabla^2 \boldsymbol{\sigma} = \mu \nabla^2 [\nabla \mathbf{u} + (\nabla \mathbf{u})^T].$$

If μ is a constant, the divergence of the stress tensor simplifies to:

$$\nabla \cdot \boldsymbol{\sigma} = \mu \nabla^2 \mathbf{u}$$

Thus, combining everything, the final form of the Poisson equation is:

$$\nabla^2 p = -\rho \nabla \cdot (\mathbf{u} \cdot \nabla \mathbf{u}) + \mu \nabla \cdot \nabla^2 \mathbf{u}$$

The problem with solving this equation is that the boundary conditions for the pressure field are not known. If we take the dot product of the momentum equation with a vector $\mathbf{n}$ normal to the boundary, we get the following boundary condition for the pressure:

$$\frac{1}{\rho} \nabla_n p = -\mathbf{n} \cdot \frac{\partial \mathbf{u}}{\partial t} - \nabla_n \cdot \left(\frac{\mathbf{u}^2}{2} \right) - \mathbf{n} \cdot (\boldsymbol{\omega} \times \mathbf{u}) + \nabla_n \cdot \boldsymbol{\sigma}$$

Solving this equation requires computing the time derivative of the velocity field.

2.3 Helmholtz Decomposition

To overcome this problem we can use the Helmholtz decomposition of the velocity field into a solenoidal and an irrotational part. The velocity field can therefore be expressed as:

$$\mathbf{u}(\mathbf{x}) = \mathbf{u}_\infty(\mathbf{x}) + \mathbf{u}_i(\mathbf{x}) + \mathbf{u}_\omega(\mathbf{x}), \quad (2.7)$$

where $\mathbf{u}_\infty$ is a known freestream contribution, $\mathbf{u}_i$ is the irrotational component, and $\mathbf{u}_\omega$ is the solenoidal (divergence-free) part associated with vorticity. Specifically:

$$\nabla \phi = \mathbf{u}_i$$

$$\nabla \times \boldsymbol{\psi} = \mathbf{u}_\omega \quad (2.8)$$

Substituting (2.7) this into the continuity equation (2.1.1) we get:

$$\nabla \cdot \mathbf{u} = \nabla \cdot \mathbf{u}_i + \nabla \cdot \mathbf{u}_\omega = 0$$

$$\Rightarrow \nabla \cdot \nabla \phi + \nabla \cdot \nabla \times \boldsymbol{\psi} = 0$$

so that

$$\nabla^2 \phi = 0 \quad (2.9)$$

Similarly, the vorticity becomes:

$$\boldsymbol{\omega} = \nabla \times \mathbf{u} = \nabla \times (\nabla \times \boldsymbol{\psi}) = \nabla(\nabla \cdot \boldsymbol{\psi}) - \nabla^2 \boldsymbol{\psi}.$$

By choosing the gauge $\nabla \cdot \boldsymbol{\psi} = 0$, we obtain:

$$\nabla^2 \boldsymbol{\psi} = -\nabla \times \mathbf{u}_\omega = -\boldsymbol{\omega}. \quad (2.10)$$

One of the key insights of this decomposition provides emerges by taking the curl of (2.8) in its velocity form:

$$\nabla \times (\nabla \times \mathbf{u}_\omega) = \nabla^2 \mathbf{u}_\omega = -\nabla \times \boldsymbol{\omega}.$$

The corresponding solution for an infinite domain is obtained through integration by parts as:

$$\mathbf{u}_\omega(\mathbf{x}) = \int_{\Omega} \mathbf{K}(\mathbf{x} - \mathbf{y}) \boldsymbol{\omega}(\mathbf{y}) d\mathbf{y},$$

where $\mathbf{K}$ is the Biot–Savart kernel, encoding the fundamental relationship between vorticity and the velocity it induces.

In similar fashion to the continuity equation, substituting (2.7) into the momentum equation (2.5) we get:

$$\frac{\partial(\mathbf{u}_\omega + \nabla \phi)}{\partial t} + \nabla \left(\frac{\mathbf{u}}{2} \right) - \mathbf{u} \times \boldsymbol{\omega} = \frac{1}{\rho} \nabla \cdot (-p\mathbf{I} + \boldsymbol{\sigma})$$

$$\begin{aligned}
&\Rightarrow \frac{\partial(\mathbf{u}_\omega)}{\partial t} + \frac{\partial \nabla \phi}{\partial t} + \nabla \left(\frac{\mathbf{u}^2}{2} \right) - \mathbf{u} \times \boldsymbol{\omega} = \frac{1}{\rho} \nabla \cdot (-p\mathbf{I} + \boldsymbol{\sigma}) \\
&\Rightarrow \nabla \left(\frac{\partial \phi}{\partial t} + \frac{\mathbf{u}^2}{2} + \frac{p}{\rho} \right) + \frac{\partial \mathbf{u}_\omega}{\partial t} - \mathbf{u} \times \boldsymbol{\omega} = \frac{\nabla \cdot \boldsymbol{\sigma}}{\rho} \\
&\Rightarrow \nabla \left(\frac{\partial \phi}{\partial t} + \frac{\mathbf{u}^2}{2} + \frac{p}{\rho} \right) = -\frac{\partial \mathbf{u}_\omega}{\partial t} + \mathbf{u} \times \boldsymbol{\omega} + \frac{\nabla \cdot \boldsymbol{\sigma}}{\rho} \tag{2.11}
\end{aligned}$$

Noticing how the left hand side of the equation is the gradient of some quantity, the right hand side of the equation can also be expressed as the gradient of a scalar field q :

$$\Rightarrow \frac{\nabla q}{\rho} = \frac{\partial \mathbf{u}_\omega}{\partial t} - \mathbf{u} \times \boldsymbol{\omega} - \frac{\nabla \cdot \boldsymbol{\sigma}}{\rho}$$

Applying the divergence operator to the above equation we get:

$$\begin{aligned}
&\Rightarrow \frac{\nabla^2 q}{\rho} = \nabla \cdot \left(\frac{\partial \mathbf{u}_\omega}{\partial t} - \mathbf{u} \times \boldsymbol{\omega} - \frac{\nabla \cdot \boldsymbol{\sigma}}{\rho} \right) \\
&\Rightarrow \frac{\nabla^2 q}{\rho} = \frac{\partial \nabla \cdot \mathbf{u}_\omega}{\partial t} - \nabla \cdot (\mathbf{u} \times \boldsymbol{\omega}) - \frac{\nabla \cdot \nabla \cdot \boldsymbol{\sigma}}{\rho}
\end{aligned}$$

And by definition $\nabla \cdot \mathbf{u}_\omega = 0$ so:

$$\Rightarrow \frac{\nabla^2 q}{\rho} = -\nabla \cdot (\mathbf{u} \times \boldsymbol{\omega}) - \frac{\nabla \cdot \nabla \cdot \boldsymbol{\sigma}}{\rho} \tag{2.12}$$

Using the vector identity:

$$\nabla \cdot (\mathbf{A} \times \mathbf{B}) = (\nabla \times \mathbf{A}) \cdot \mathbf{B} - (\nabla \times \mathbf{B}) \cdot \mathbf{A},$$

with $\mathbf{A} = \mathbf{u}$ and $\mathbf{B} = \boldsymbol{\omega}$, we get:

$$\nabla \cdot (\mathbf{u} \times \boldsymbol{\omega}) = (\nabla \times \mathbf{u}) \cdot \boldsymbol{\omega} - (\nabla \times \boldsymbol{\omega}) \cdot \mathbf{u}.$$

Since $\nabla \times \mathbf{u} = \boldsymbol{\omega}$, this simplifies to:

$$\nabla \cdot (\mathbf{u} \times \boldsymbol{\omega}) = \boldsymbol{\omega}^2 - (\nabla \times \boldsymbol{\omega}) \cdot \mathbf{u}.$$

Thus, substituting into (2.12):

$$\frac{\nabla^2 q}{\rho} = -\boldsymbol{\omega}^2 + (\nabla \times \boldsymbol{\omega}) \cdot \mathbf{u} - \frac{\mu}{\rho} \nabla \cdot \nabla^2 \mathbf{u}. \tag{2.13}$$

Substituting the q field into (2.11) we get the following algebraic equation:

$$\Rightarrow \nabla \left(\frac{\partial \phi}{\partial t} + \frac{\mathbf{u}^2}{2} + \frac{p}{\rho} + \frac{q}{\rho} \right) = 0$$

$$\Rightarrow \frac{\partial \phi}{\partial t} + \frac{\mathbf{u}^2}{2} + \frac{p}{\rho} + \frac{q}{\rho} = cst \quad (2.14)$$

To solve the above equation, we can first solve for the q field (2.12) and then substitute the solution into the equation for the pressure field (2.14). The boundary conditions for the equation for the q field can be once again expressed by projecting the equation onto the normal vector $\mathbf{n}$ of the boundary:

$$\begin{aligned} \mathbf{n} \cdot \frac{\nabla q}{\rho} &= \mathbf{n} \cdot \frac{\partial \mathbf{u}_\omega}{\partial t} - \mathbf{n} \cdot (\mathbf{u} \times \boldsymbol{\omega}) - \mathbf{n} \cdot \frac{\mu}{\rho} \nabla^2 \mathbf{u} \\ \Rightarrow \frac{\partial q}{\partial n} \frac{1}{\rho} &= \frac{\partial}{\partial t} (\mathbf{n} \cdot \mathbf{u}_\omega) - \mathbf{n} \cdot (\mathbf{u} \times \boldsymbol{\omega}) - \frac{\mu}{\rho} \nabla_n^2 \mathbf{u} \end{aligned} \quad (2.15)$$

Summary of Equations

Summarizing the equations we have derived so far:

- The continuity equation for an incompressible fluid that gives the irrotational part of the velocity field:

$$\nabla^2 \phi = 0$$

- The equation for the solenoidal part of the velocity field:

$$\nabla^2 \boldsymbol{\psi} = -\boldsymbol{\omega}$$

- The equation for the velocity field:

$$\mathbf{u} = \nabla \phi + \nabla \times \boldsymbol{\psi} = \mathbf{u}_i + \mathbf{u}_\omega \quad (2.16)$$

- The equation for the q field:

$$\begin{aligned} \frac{\nabla^2 q}{\rho} &= -\nabla \cdot (\mathbf{u} \times \boldsymbol{\omega}) - \frac{\mu}{\rho} \nabla \cdot \nabla^2 \mathbf{u}. \\ &= -\boldsymbol{\omega}^2 + (\nabla \times \boldsymbol{\omega}) \cdot \mathbf{u} - \frac{\mu}{\rho} \nabla \cdot \nabla^2 \mathbf{u}. \end{aligned} \quad (2.17)$$

- The algebraic equation for the pressure field:

$$\frac{\partial \phi}{\partial t} + \frac{\mathbf{u}^2}{2} + \frac{p}{\rho} + \frac{q}{\rho} = cst$$

- The boundary conditions for the q field:

$$\frac{\nabla_n q}{\rho} = \frac{\partial (\mathbf{n} \cdot \mathbf{u}_\omega)}{\partial t} - \mathbf{n} \cdot (\mathbf{u} \times \boldsymbol{\omega}) - \frac{\mu}{\rho} \nabla_n^2 \mathbf{u}$$

The last equation is a Neumann boundary condition for the q field. This equation must be combined with the boundary conditions for the solid boundaries of the domain. The boundary conditions for a solid boundary depend on the formulation of our flow regime.

2.4 Lagrangian Formulation of the Navier-Stokes

The particle formulation of the Navier-Stokes equations, as employed in the Vortex Particle Method (VPM), transitions the mathematical framework from an Eulerian (grid-based) description to a Lagrangian (particle-tracking) one. In what follows, we detail the fundamental *particle approximations* central to such a Lagrangian approach.

2.4.1 Lagrangian Description

In the Lagrangian description, the fluid is treated as individual elements that move, rotate, and deform, each carrying its own flow variables (e.g., velocity, temperature). By specifying each element's initial position and following it in time, one obtains a complete picture of the flow across all points in space and at every instant. This approach naturally extends particle mechanics, with velocity as the primary quantity describing each element's motion.

The velocity of a fluid element residing in an inertial (Eulerian) frame of reference X_p is expressed as:

$$u_p = \frac{\partial X_p}{\partial t}$$

Control Volume Operator In a *Lagrangian* framework, each *particle* (or particle-like control volume) $V_p(t)$ is tracked through space and time. We define an operator that maps a field property q to a *particle-level* (i.e., control-volume-based) quantity Q_p . Specifically:

$$\langle q \rangle_p = \frac{1}{V_p(t)} \int_{V_p(t)} q \, dV_p(t).$$

and equivalently:

$$Q_p = \langle q \rangle_p V_p = \int_{V_p(t)} q \, dV_p(t). \quad (2.18)$$

This yields the *total value of q* inside $V_p(t)$. For example, if q is vorticity, then $\langle q \rangle_p$ is the spatial average of that velocity field within the particle and Q_p the circulation of the particle.

2.4.2 Particle Approximations

By applying this logic, we can construct particle-level integrals for mass, dilatation, and vorticity. Specifically:

$$\begin{aligned} M_p &= \langle \rho \rangle_p V_p = \int_{V_p(t)} \rho dV_p(t) \\ \Theta_p(t) &= \langle \theta \rangle_p V_p(t) = \langle \nabla \cdot \mathbf{u} \rangle_p V_p(t) = \int_{V_p(t)} \theta dV_p(t) \\ \Omega_p &= \langle \omega \rangle_p V_p(t) = \langle \nabla \times \mathbf{u} \rangle_p V_p(t) = \int_{V_p(t)} \omega dV_p(t) \end{aligned}$$

Here, Θ_p and Ω_p represent, respectively, the total *dilatation* and *vorticity* contained in the volume $V_p(t)$ with $\theta \equiv \nabla \cdot \mathbf{u}$ and $\omega \equiv \nabla \times \mathbf{u}$.

These particle-based integrals (M_p, θ_p, Ω_p) act as flow markers carrying fluid information, but they do *not* necessarily correspond to literal fluid parcels at the microscopic scale. Rather, they are *computational constructs* whose volumes and positions evolve according to the local fluid velocity in a Lagrangian method. Such an approach allows one to track the evolution of mass, vorticity, or other properties more naturally than in a fixed-grid (Eulerian) perspective, and it underpins the formulation of the Vortex Particle Methods.

2.4.3 Reynolds Transport Theorem

According to Reynolds' transport theorem, if we consider the rate of change of the volume integral of quantity Q in a material volume $V(t)$ with surface $S(t)$ having normal $\mathbf{n}$ and velocity $\mathbf{u}$, then:

$$\frac{dQ_{sys}}{dt} = \frac{d \int_V Q dV}{dt} = \int_V \frac{\partial Q}{\partial t} dV + \int_S Q(\mathbf{u} \cdot \mathbf{n}) dS, \quad (2.19)$$

Applying the divergence theorem to the surface integral:

$$\int_S (Q\mathbf{u} \cdot \mathbf{n}) dS = \int_V \nabla \cdot (Q\mathbf{u}) dV, \quad (2.20)$$

Yielding the form:

$$\frac{dQ_{sys}}{dt} = \int_V \left(\frac{\partial Q}{\partial t} + \nabla \cdot (Q\mathbf{u}) \right) dV.$$

Expanding:

$$\frac{dQ_{sys}}{dt} = \int_V \left(\frac{\partial(Q)}{\partial t} + \nabla(Q) \cdot \mathbf{u} + Q \nabla \cdot \mathbf{u} \right) dV. \quad (2.21)$$

Finally given the definition of the material derivative as:

$$\frac{DQ}{Dt} \equiv \frac{\partial Q}{\partial t} + \mathbf{u} \cdot \nabla Q \quad (2.22)$$

We get:

$$\frac{dQ_{sys}}{dt} = \int_V \left(\frac{DQ}{Dt} + Q \nabla \cdot \mathbf{u} \right) dV. \quad (2.23)$$

2.4.4 Mass Continuity Equation

Setting $Q = \rho$ we can define the control volume's mass:

$$Q_{sys} = \int_V Q dV = M_{sys}$$

Mass conservation requires that the above quantity is equal to zero and therefore:

$$\frac{dM_v}{dt} = 0 = \int_V \left(\frac{D(\rho)}{Dt} + \rho \nabla \cdot \mathbf{u} \right) dV. \quad (2.24)$$

Since this must hold for any arbitrary volume, we obtain the differential form of the continuity equation:

$$\frac{D\rho}{Dt} + \rho \nabla \cdot \mathbf{u} = 0. \quad (2.25)$$

$$\frac{D\rho}{Dt} = -\rho \nabla \cdot \mathbf{u} = -\rho \theta. \quad (2.26)$$

Specializing to a *fluid particle* (or control volume) in which the density is (locally) uniform, then ρ_p . $\theta_p \equiv \nabla \cdot \mathbf{u}_p$ is the (local) volumetric strain rate of the particle. If the particle's volume is V_p , then its mass is

$$M_p = \rho_p V_p.$$

Next, expanding the material derivative of $\rho_p = M_p/V_p$:

$$\frac{D}{Dt} \left(\frac{M_p}{V_p} \right) = \frac{V_p \frac{DM_p}{Dt} - M_p \frac{DV_p}{Dt}}{V_p^2}. \quad (2.27)$$

Thus,

$$\frac{V_p \frac{DM_p}{Dt} - M_p \frac{DV_p}{Dt}}{V_p^2} = -\frac{M_p}{V_p} \theta_p.$$

Multiply both sides by V_p^2 ,

$$V_p \frac{DM_p}{Dt} - M_p \frac{DV_p}{Dt} = -M_p V_p \theta_p, \quad (2.28)$$

or equivalently,

$$V_p \frac{DM_p}{Dt} = M_p \frac{DV_p}{Dt} - M_p V_p \theta_p. \quad (2.29)$$

If a *fluid particle* is defined to move and deform with the fluid velocity, we can express that:

$$\frac{DV_p}{Dt} = V_p (\nabla \cdot \mathbf{u}_p) = V_p \theta_p$$

and therefore:

$$\begin{aligned} V_p \frac{DM_p}{Dt} &= M_p (V_p \theta_p) - M_p V_p \theta_p = 0. \\ &\Leftrightarrow \frac{DM_p}{Dt} = 0. \end{aligned}$$

In essence this derivation shows that if we consider the particle's volume to be deforming in accordance with the flow velocity, the mass *contained* in the fluid parcel (which is advecting with the flow) is a *constant in time* because there is no leakage of mass across its deforming boundary. Mathematically:

$$\frac{DV_p}{Dt} = V_p \theta_p \quad \Leftrightarrow \quad \frac{DM_p}{Dt} = 0.$$

2.4.5 Vorticity and Dilatation Equations

Vorticity Evolution Equation The momentum equation for an incompressible fluid is given by

$$\frac{\partial \mathbf{u}}{\partial t} + \nabla \left(\frac{|\mathbf{u}|^2}{2} \right) - \mathbf{u} \times \boldsymbol{\omega} = \frac{1}{\rho} \nabla \cdot (-pI + \boldsymbol{\sigma}),$$

Taking the curl of both sides yields:

$$\frac{\partial \boldsymbol{\omega}}{\partial t} - \nabla \times (\mathbf{u} \times \boldsymbol{\omega}) = \nabla \times \frac{1}{\rho} \nabla \cdot (-pI + \boldsymbol{\sigma}). \quad (2.30)$$

Using the vector identity:

$$\nabla \times (\mathbf{u} \times \boldsymbol{\omega}) = \mathbf{u}(\nabla \cdot \boldsymbol{\omega}) - \boldsymbol{\omega}(\nabla \cdot \mathbf{u}) + (\boldsymbol{\omega} \cdot \nabla) \mathbf{u} - (\mathbf{u} \cdot \nabla) \boldsymbol{\omega},$$

and noting that, by definition, $\nabla \cdot \boldsymbol{\omega} = 0$, the above identity reduces to:

$$\nabla \times (\mathbf{u} \times \boldsymbol{\omega}) = -\boldsymbol{\omega}(\nabla \cdot \mathbf{u}) + (\boldsymbol{\omega} \cdot \nabla) \mathbf{u} - (\mathbf{u} \cdot \nabla) \boldsymbol{\omega}.$$

Substitution into the vorticity equation (2.30) yields

$$\frac{\partial \boldsymbol{\omega}}{\partial t} + (\mathbf{u} \cdot \nabla) \boldsymbol{\omega} = -\boldsymbol{\omega}(\nabla \cdot \mathbf{u}) + (\boldsymbol{\omega} \cdot \nabla) \mathbf{u} + \nabla \times \frac{1}{\rho} \nabla \cdot (-pI + \boldsymbol{\sigma}).$$

By introducing the material derivative,

$$\frac{D\boldsymbol{\omega}}{Dt} = \frac{\partial \boldsymbol{\omega}}{\partial t} + (\mathbf{u} \cdot \nabla) \boldsymbol{\omega},$$

the vorticity equation can be written in the compact form:

$$\frac{D\boldsymbol{\omega}}{Dt} = -\boldsymbol{\omega}(\nabla \cdot \mathbf{u}) + (\boldsymbol{\omega} \cdot \nabla) \mathbf{u} + \nabla \times \frac{1}{\rho} \nabla \cdot (-pI + \boldsymbol{\sigma}). \quad (2.31)$$

Under the assumptions of a Newtonian fluid it was shown that $\nabla \cdot \boldsymbol{\sigma} = \mu \nabla^2 \mathbf{u}$, hence:

$$\nabla \cdot (-pI + \boldsymbol{\sigma}) = -\nabla p + \mu \nabla^2 \mathbf{u}.$$

Since the curl of a gradient is identically zero, the term on the right-hand side of equation (2.31) reduces to

$$\nabla \times \frac{1}{\rho} (-\nabla p + \mu \nabla^2 \mathbf{u}) = \nabla \times \left(\frac{\mu}{\rho} \nabla^2 \mathbf{u} \right).$$

Defining the kinematic viscosity as $\nu = \mu/\rho$ and noting that the curl and Laplacian operators commute for sufficiently smooth fields, the expression simplifies further to

$$\nabla \times (\nu \nabla^2 \mathbf{u}) = \nu \nabla^2 (\nabla \times \mathbf{u}) = \nu \nabla^2 \boldsymbol{\omega}.$$

Substitution into equation (2.31) yields

$$\frac{D\boldsymbol{\omega}}{Dt} = -\boldsymbol{\omega}(\nabla \cdot \mathbf{u}) + (\boldsymbol{\omega} \cdot \nabla) \mathbf{u} + \nu \nabla^2 \boldsymbol{\omega}. \quad (2.32)$$

Rearranging the terms results in:

$$\frac{D\boldsymbol{\omega}}{Dt} + \boldsymbol{\omega}(\nabla \cdot \mathbf{u}) = (\boldsymbol{\omega} \cdot \nabla) \mathbf{u} + \nu \nabla^2 \boldsymbol{\omega}. \quad (2.33)$$

Recalling from Reynolds Transport Theorem that states:

$$\frac{dQ_{sys}}{dt} = \int_V \left(\frac{DQ}{Dt} + Q \nabla \cdot \mathbf{u} \right) dV.$$

we can express:

$$\frac{d\Omega_p}{dt} = \int_V \left(\frac{D(\omega)}{Dt} + \omega \nabla \cdot \mathbf{u} \right) dV.$$

Integrating equation (2.33) results in:

$$\frac{d\Omega_p}{dt} = \int_V \left(\frac{D(\omega)}{Dt} + \omega \nabla \cdot \mathbf{u} \right) dV_p = \int_V \left((\boldsymbol{\omega} \cdot \nabla) \mathbf{u} + \nu \nabla^2 \boldsymbol{\omega} \right) dV_p$$

Hence:

$$\frac{d\Omega_p}{dt} = \int_V \left((\boldsymbol{\omega} \cdot \nabla) \mathbf{u} \right) dV_p + \int_V \left(\nu \nabla^2 \boldsymbol{\omega} \right) dV_p \quad (2.34)$$

Employing the control-volume operator defined in (2.18) we can rewrite the above expression into:

$$\boxed{\frac{d\Omega_p}{dt} = \langle (\boldsymbol{\omega} \cdot \nabla) \mathbf{u} \rangle_p V_p + \langle \nu \nabla^2 \boldsymbol{\omega} \rangle_p V_p} \quad (2.35)$$

Dilatation Evolution Equation On a similar manner to the vorticity evolution equation if we take the divergence of the momentum equation (in the form of (2.4)) we have:

$$\begin{aligned} \nabla \cdot \left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} \right) &= \nabla \cdot \left(\frac{1}{\rho} \nabla \cdot (-pI + \boldsymbol{\sigma}) \right) \\ \frac{\partial \nabla \cdot \mathbf{u}}{\partial t} + \nabla \cdot (\mathbf{u} \cdot \nabla \mathbf{u}) &= \nabla \cdot \frac{1}{\rho} \nabla \cdot (-pI + \boldsymbol{\sigma}) \end{aligned}$$

We can expand the divergence of the advection term as follows:

$$\nabla \cdot (\mathbf{u} \cdot \nabla \mathbf{u}) = (\mathbf{u} \cdot \nabla)(\nabla \cdot \mathbf{u}) + (\nabla \cdot \mathbf{u})^2 - 2|\nabla \mathbf{u}| \quad (2.36)$$

$$= (\mathbf{u} \cdot \nabla) \theta + \theta^2 - 2|\nabla \mathbf{u}|. \quad (2.37)$$

Where $\nabla \mathbf{u}$ refers to the vector gradient, that equals the transpose of the Jacobian matrix and is defined as the following tensor:

$$\nabla \mathbf{u} = \delta^{jk} \frac{\partial \mathbf{u}^i}{\partial x_j} e_i \otimes e_k = \frac{\partial \mathbf{u}^i}{\partial x_j} e_i \otimes e_j = \begin{bmatrix} \frac{\partial \mathbf{u}}{\partial x} & \frac{\partial v}{\partial x} & \frac{\partial w}{\partial x} \\ \frac{\partial \mathbf{u}}{\partial y} & \frac{\partial v}{\partial y} & \frac{\partial w}{\partial y} \\ \frac{\partial \mathbf{u}}{\partial z} & \frac{\partial v}{\partial z} & \frac{\partial w}{\partial z} \end{bmatrix},$$

Additionally we can simplify the equation right hand side using the following vector calculus identity:

$$\nabla \cdot \left(\frac{\mathbf{A}}{\phi} \right) = \frac{\phi (\nabla \cdot \mathbf{A}) - (\nabla \phi) \cdot \mathbf{A}}{\phi^2},$$

The RHS thus becomes:

$$\nabla \cdot \frac{1}{\rho} \nabla \cdot (-p\mathbf{I} + \boldsymbol{\sigma}) = \frac{\rho (\nabla \cdot \nabla \cdot (-p\mathbf{I} + \boldsymbol{\sigma})) - (\nabla \rho) \cdot \nabla \cdot (-p\mathbf{I} + \boldsymbol{\sigma})}{\rho^2},$$

Which after some steps simplifies to:

$$\frac{\nabla^2 p + \nabla \cdot \mu \nabla^2 \mathbf{u}}{\rho} - \frac{(\nabla \rho) \cdot (-\nabla p + \mu \nabla^2 \mathbf{u})}{\rho^2},$$

Combining the above,

$$\frac{\partial \theta}{\partial t} + (\mathbf{u} \cdot \nabla) \theta + \theta^2 - 2|\nabla \mathbf{u}| = \nabla \cdot \left(\frac{1}{\rho} \nabla \cdot (-p\mathbf{I} + \boldsymbol{\sigma}) \right).$$

Recognizing the material derivative the equation becomes:

$$\frac{D\theta}{Dt} = -\theta^2 + 2|\nabla \mathbf{u}| + \nabla \cdot \left(\frac{1}{\rho} \nabla \cdot (-p\mathbf{I} + \boldsymbol{\sigma}) \right).$$

$$\frac{D\theta}{Dt} + \theta(\nabla \cdot \mathbf{u}) = 2|\nabla \mathbf{u}| + \nabla \cdot \left(\frac{1}{\rho} \nabla \cdot (-p\mathbf{I} + \boldsymbol{\sigma}) \right).$$

Integrating the above equation inside the particle control volume results in:

$$\int_V \left(\frac{D\theta}{Dt} + \theta(\nabla \cdot \mathbf{u}) \right) dV = \int_V \left(2|\nabla \mathbf{u}| + \nabla \cdot \left(\frac{1}{\rho} \nabla \cdot (-p\mathbf{I} + \boldsymbol{\sigma}) \right) \right) dV$$

Finally applying RTT once again we get:

$$\boxed{\frac{d\Theta}{dt} = \left\langle 2|\nabla \mathbf{u}| + \frac{\nabla^2 p + \nabla \cdot \mu \nabla^2 \mathbf{u}}{\rho} - \frac{(\nabla \rho) \cdot (-\nabla p + \mu \nabla^2 \mathbf{u})}{\rho^2} \right\rangle_p V_p} \quad (2.38)$$

For an incompressible fluid the above expression is identically equal to zero since

$$\nabla \cdot \mathbf{u} = \theta = 0$$

and

$$\frac{d\Theta p}{dt} = \int_V \left(\frac{D(\theta)}{Dt} + \theta \nabla \cdot \mathbf{u} \right) dV = 0.$$

Chapter 3

The Vortex Particle Method for Incompressible flows

Introduction

Vortex Particle Methods (VPM) for incompressible flows constitute a class of Lagrangian numerical techniques in which fluid parcels, carrying vorticity, are represented by a finite number of particles. This approach has garnered significant attention for its ability to capture complex flow phenomena, especially those dominated by vortex dynamics such as wake instabilities, turbulent eddies, and vortex sheet roll-up. In contrast to grid-based (Eulerian) methods, VPMs naturally adapt to regions of high vorticity concentration, thereby reducing numerical diffusion and permitting efficient tracking of highly localized flow structures. Furthermore, the Lagrangian nature of the method alleviates certain stability constraints tied to the explicit advection schemes in conventional finite-volume or finite-difference formulations.

The mathematical foundation of VPM relies on recasting the governing equations for fluid motion in the vorticity form of the Navier–Stokes or Euler equations. The velocity field, $\mathbf{u}(\mathbf{x}, t)$, and its associated vorticity, $\boldsymbol{\omega} = \nabla \times \mathbf{u}$, form the core fields of interest. By discretizing the vorticity over a set of particles, the fluid motion can be tracked by ordinary differential equations governing the trajectories of these particles and the evolution of their vorticity strengths. Such a discretization lends itself naturally to adaptive strategies, where particles can be remeshed or redistributed according to local flow features to maintain high accuracy. Additionally, through well-established integral representations of velocity from vorticity, the long-range interactions can be computed using fast summation algorithms, thereby addressing the $\mathcal{O}(N^2)$ bottleneck in naive N -body formulations.

Despite the attractive features of VPM, certain numerical subtleties warrant careful treatment. The accurate transfer of quantities from particles to any auxiliary grid (if a hybrid particle–mesh approach is employed), as well as the subsequent projection of velocities or vorticities back to the particles, must be performed with high-order interpolation techniques to minimize numerical diffusion and enforce incompressibility. Furthermore, evaluating the pressure field, which is typically derived from the divergence-free condition, calls for the solution of a Poisson equa-

tion, whose discretization can be delicate when coupled with particle-based vorticity transport. Finally, managing vorticity stretching, diffusion, and potential topological changes is essential to the physical fidelity of the simulation, demanding careful time-integration and remeshing algorithms.

This chapter begins by detailing the essential tools for function approximation within a particle framework, and proceeds through derivations of the governing flow equations, their particle–mesh interaction, and the associated interpolation and projection operators. The subsequent sections highlight the time-stepping operators, culminating in the discussion of remeshing strategies and pressure recovery. We address the corrections necessary to conserve vorticity accurately, and we conclude with a concise overview of the entire Vortex Particle Method algorithm. Through this systematic exposition, we aim to underscore both the theoretical underpinnings and the practical considerations crucial to the successful deployment of the Vortex Particle Method for incompressible flows.

3.1 Function Approximation

In the Lagrangian framework of PDEs, fluid properties in the domain are not discretized in a mesh, but rather on a set of particles that are convected according to the flow velocity $\mathbf{u}$. The integral representation of a suitably defined scalar function f over a domain Ω can be expressed using the following convolution:

$$f(\mathbf{r}) = \int_{\Omega} f(\mathbf{r}') \delta(\mathbf{r} - \mathbf{r}') d\mathbf{r}', \quad (3.1)$$

where $\mathbf{r}$ is a position in the domain Ω , $d\mathbf{r}'$ represents an infinitesimal volume element, and δ is the Dirac delta function. Directly employing this delta distribution makes numerical discretization prohibitively expensive and can produce very large values, evaluating close to the singularity. To address this issue, Chorin and Bernard [7] proposed the vortex blob method, which spreads each particle’s vorticity over a finite-radius blob. Specifically, this approach replaces the singular delta function with a smooth cutoff function ζ . Thus the approximation can be written as:

$$f_i(\mathbf{r}) = \int_{\Omega} f(\mathbf{r}') \zeta(\mathbf{r} - \mathbf{r}', h) d\mathbf{r}', \quad (3.2)$$

where h is a parameter is equivalent to the “smoothing length,” playing the same role as spatial discretization parameters in conventional methods. Such an interpolation function is visualized in 3.1.

The kernel function ζ of order r must satisfy the following properties:

- **Dirac-measure property:**

$$\lim_{h \rightarrow 0} \zeta(\mathbf{r} - \mathbf{r}', h) = \delta(\mathbf{r} - \mathbf{r}').$$

- **Normalization:**

$$\int \zeta(\mathbf{r} - \mathbf{r}', h) d\mathbf{r}' = 1.$$

- **Symmetry:**

$$\int (\mathbf{r} - \mathbf{r}')^i \zeta(\mathbf{r} - \mathbf{r}', h) d\mathbf{r}' = 0 \quad \text{if } |i| \leq r - 1.$$

- **Compact support:**

$$\int |\mathbf{r} - \mathbf{r}'|^i \zeta(\mathbf{r} - \mathbf{r}', h) d\mathbf{r}' < \infty$$

Interpolation Kernel Visualization

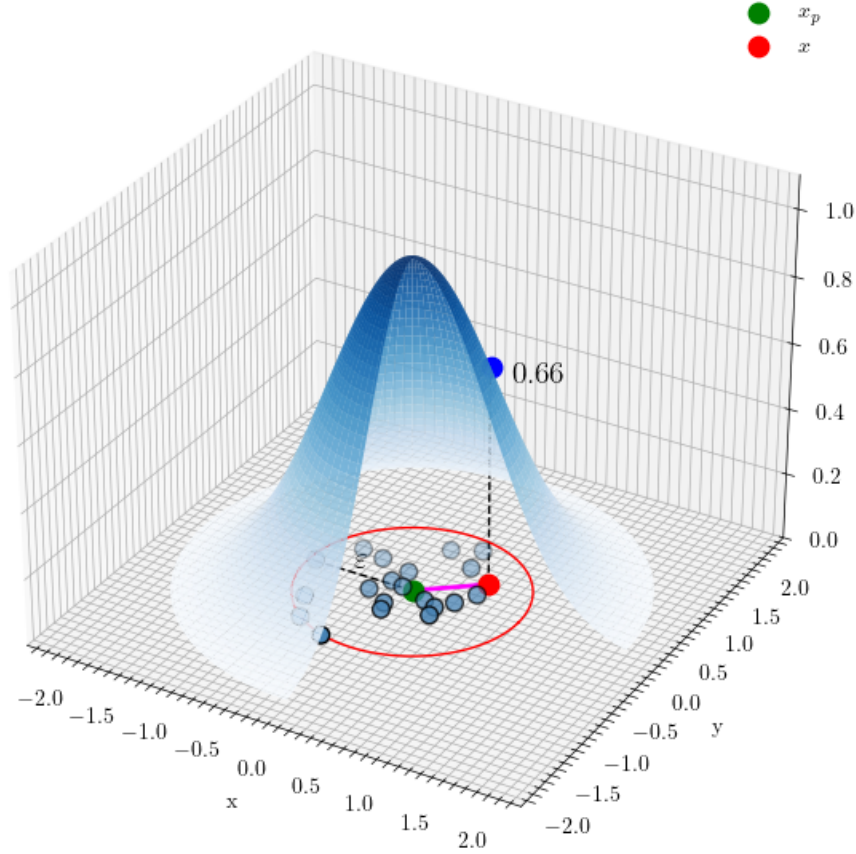


Figure 3.1: Visualization of a Gaussian Interpolation Kernel

The first two properties of the kernel W are necessary for achieving a consistent interpolation (i.e., transitioning from (3.1) to (3.2)). The third property ensures that the cutoff function ζ has the same moment properties as the Dirac measure, up

to r . For a second order approximation we could also define compact support as:

$$\zeta(\mathbf{r} - \mathbf{r}', h) = 0 \quad \text{whenever} \quad \frac{\|\mathbf{r} - \mathbf{r}'\|}{h} > k.$$

A particle-based representation of this interpolation is then obtained by summing over all particles:

$$f(\mathbf{r}) = \sum_{1 \leq b \leq N} f_a W(\mathbf{x} - \mathbf{r}_a, h), \quad (3.3)$$

In particle-based methods, where flow properties are computed via the discrete particles, it is standard practice to introduce a mollified function, denoted ζ_ϵ . One common definition is:

$$\zeta_\epsilon = \frac{1}{\epsilon^3} \zeta\left(\frac{\mathbf{x}}{\epsilon}\right),$$

Here, ζ is frequently chosen to be a Gaussian-type cutoff function, valued for its smoothness, radial symmetry, and rapid decay as $\|\mathbf{x}\| \rightarrow \infty$. The employment of the mollified function is a very useful tool for analyzing the accuracy and stability of vortex methods in general and has been employed in various works. In particular Koumoutsakos and Cottet [8] employ this to prove that on both when the particle velocity is based on a grid solver or on the Biot-Savart law, vortex methods

enjoy the best rate of convergence one might expect: if the derivatives of order up to $r - 1$ of the vorticity are bounded and if one uses a cutoff of order r , the discrepancy between computed and exact velocity decays like ϵ^r , very much as for a grid-based method of the same order.

A central requirement of vortex methods is that particles must overlap (i.e., the cutoff length scale must be at least on the order of the interparticle spacing h). This result was shown by error analysis in the works of Baele and Majda in 1982 [3, 2], who showed that the vortex blob trajectories tend to the exact ones when $h = O(\epsilon)$.

This requirement along with the need to address Lagrangian grid distortions (that give rise remeshing to spatial adaptation and remeshing techniques) have required the exploration of a variety of smooth kernels fo in the literature.

The main group of interpolation kernels that are examined is this work and employed by NTUA's VPM solver are the B-Spline based ones.

The B-Spline kernels are based on the the following smooth functions (illustrated in 3.2):

$$M_p = \frac{1}{2\pi} \int_{-\infty}^{\infty} \left(\frac{\sin(\xi/2)}{\xi/2} e^{ix\xi} \right)$$

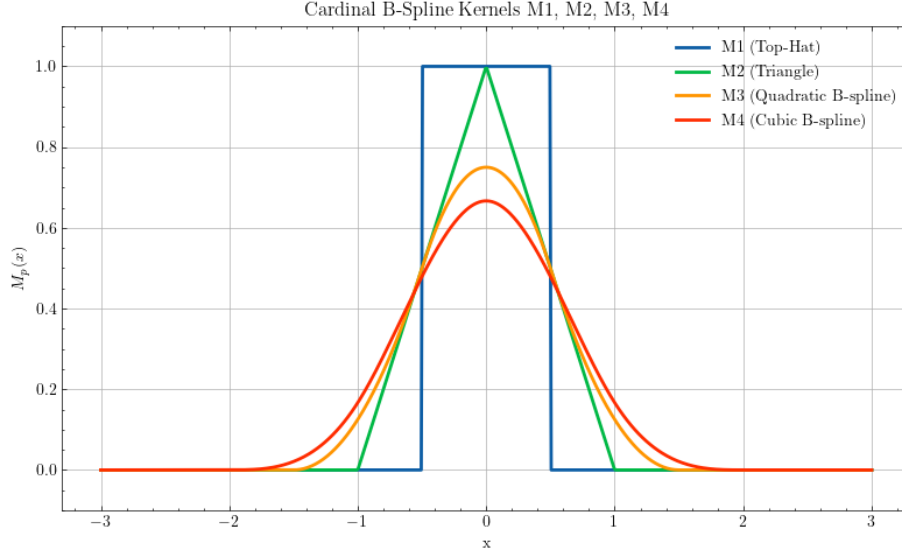


Figure 3.2: *Visualization of a Cardinal B-Spline Kernel*

These kernels, although simple to work with, are limited in their accuracy as they preserve only the first two moments. For this reason, extrapolations of the cardinal kernels (proposed first by Monaghan [22]) have been constructed. These kernels result from linear combinations of M_p kernels, constructed so that the total successive moments are canceled. These functions are referred to as M'_p and are visualized in the following diagram:

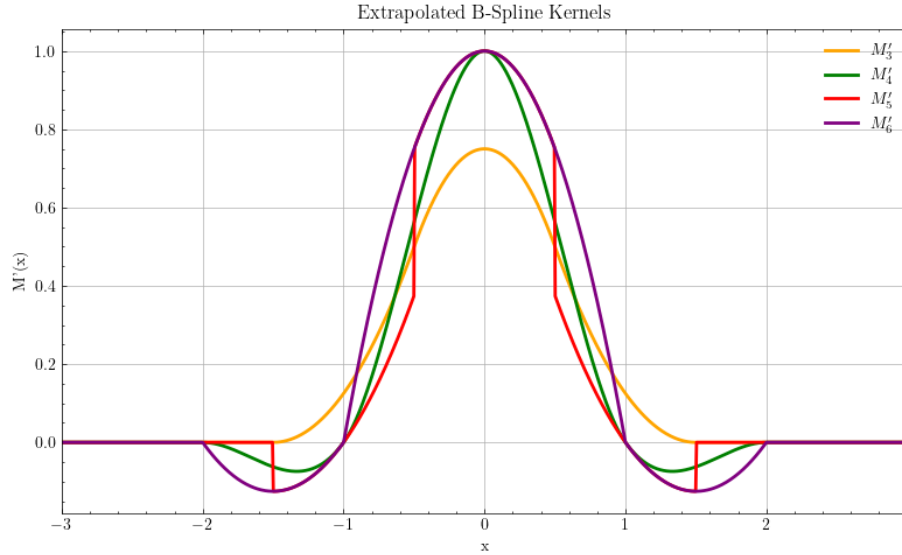


Figure 3.3: *Visualization of the Extrapolated B-Spline Kernels*

In the present work the M'_4 interpolation function is used, which conserves moments up to the 3rd order and in practice is found to be a good compromise between

accuracy, computational cost and regularity. Analytically this function is defined as:

$$W = M'_4(x) = \frac{1}{2} \left(3 M_4(x) + x \frac{d}{dx} M_4(x) \right). \quad (3.4)$$

A piecewise form for W is then given by

$$W = \begin{cases} 1 - \frac{5}{2} |x|^2 + \frac{3}{2} |x|^3, & |x| \leq 1, \\ \frac{1}{2} (1 - |x|) (2 - |x|)^2, & 1 \leq |x| \leq 2, \\ 0, & \text{otherwise.} \end{cases} \quad (3.5)$$

3.2 Flow Equations

Historically, vortex methods for incompressible flows have relied on either the vorticity–velocity ($\mathbf{u} - \boldsymbol{\omega}$) formulation or the streamfunction–vorticity ($\psi - \boldsymbol{\omega}$) formulation of the Euler equations. In this work, as often in vortex methods, we employ the latter approach using the ($\psi - \boldsymbol{\omega}$) form, as described below.

As derived in the previous chapter, the goal of the Lagrangian Methods is to advect the flow markers (particles) based on the following equations:

$$\frac{DM_p}{Dt} = 0. \quad (3.6)$$

$$\frac{d\Omega_p}{dt} = \langle (\boldsymbol{\omega} \cdot \nabla) \mathbf{u} \rangle_p V_p + \langle \nu \nabla^2 \boldsymbol{\omega} \rangle_p V_p \quad (3.7)$$

While the decompositions of the flow field result in the following Poisson equations:

$$\nabla^2 \phi = \nabla \cdot \mathbf{u} = \theta \quad (3.8)$$

$$\nabla^2 \psi = -\nabla \times \mathbf{u} = -\boldsymbol{\omega} \quad (3.9)$$

3.3 Integral Representation and Green's Function.

To solve Poisson-type equations such as (3.8) or (3.9), we often use integral representations based on Green's theorem. Consider a domain Ω bounded by $\partial\Omega = \Gamma$, and let L be a linear differential operator (e.g. the Laplacian, $L = \nabla^2$). A *Green's function* $G(\mathbf{x}, \mathbf{y})$ satisfies

$$L G(\mathbf{x}, \mathbf{y}) = \delta(\mathbf{y} - \mathbf{x}),$$

where δ is the Dirac delta. In the 3D Poisson case ($\nabla^2 u = f$), the fundamental solution (Green's function) is known to be

$$G(\mathbf{x}, \mathbf{y}) = -\frac{1}{4\pi \|\mathbf{x} - \mathbf{y}\|}.$$

Green's Second Identity states that for sufficiently smooth scalar functions u and v defined on a volume Ω with boundary S , we have

$$\int_{\Omega} (u \Delta v - v \Delta u) dV = \int_S \left(u \frac{\partial v}{\partial n} - v \frac{\partial u}{\partial n} \right) dS.$$

Using Green's second identity with $u(\mathbf{y})$ a solution of $\mathcal{L}u(\mathbf{y}) = f(\mathbf{y})$ in Ω and $v(\mathbf{y}) = G(\mathbf{x}, \mathbf{y})$ then yields the classical *Green's representation theorem*:

$$u(\mathbf{x}) = \int_S G(\mathbf{x}, \mathbf{y}) \frac{\partial u}{\partial n(\mathbf{y})} dS(\mathbf{y}) - \int_S \frac{\partial G(\mathbf{x}, \mathbf{y})}{\partial n(\mathbf{y})} u(\mathbf{y}) dS(\mathbf{y}) + \int_{\Omega} G(\mathbf{x}, \mathbf{y}) f(\mathbf{y}) dV(\mathbf{y}).$$

This result may be interpreted through three operators:

- **Single-layer potential:**

$$[\mathcal{V}\phi](\mathbf{x}) = \int_S G(\mathbf{x}, \mathbf{y}) \frac{\partial \phi}{\partial n(\mathbf{y})} dS(\mathbf{y}).$$

- **Double-layer potential:**

$$[\mathcal{K}\psi](\mathbf{x}) = \int_S \frac{\partial G(\mathbf{x}, \mathbf{y})}{\partial n(\mathbf{y})} \psi(\mathbf{y}) dS(\mathbf{y}).$$

- **Newton potential:**

$$[\mathcal{N}f](\mathbf{x}) = \int_{\Omega} G(\mathbf{x}, \mathbf{y}) f(\mathbf{y}) dV(\mathbf{y}).$$

In that notation, one writes the solution compactly as

$$u_{\Omega} = \mathcal{V}u_n - \mathcal{K}u_{\Gamma} + \mathcal{N}f,$$

where u_n is the normal derivative of u on Γ , and u_s is the boundary value of u . Consequently, for both the potential ϕ and the streamfunction ψ in vortex methods, one can adopt such integral solutions—using only boundary and forcing data—to solve (3.8)–(3.9). By defining $\mathbf{r} = \mathbf{x} - \mathbf{y}$ and $\partial_{n(\mathbf{y})}$ to denote the derivative in the outward normal direction at $\mathbf{y} \in S$, these equations thus become:

$$\phi(\mathbf{x}) = \int_{\Omega} \nabla^2 \phi(\mathbf{y}) G(\mathbf{r}) dD(\mathbf{y}) + \int_S \left(\partial_{n(\mathbf{y})} \phi(\mathbf{y}) G(\mathbf{r}) - \phi(\mathbf{y}) \partial_{n(\mathbf{y})} G(\mathbf{r}) \right) dS(\mathbf{y}), \quad (3.10)$$

$$\psi_i(\mathbf{x}) = \int_{\Omega} \nabla^2 \psi_i(\mathbf{y}) G(\mathbf{r}) dD(\mathbf{y}) + \int_S \left(\partial_{n(\mathbf{y})} \psi_i(\mathbf{y}) G(\mathbf{r}) - \psi_i(\mathbf{y}) \partial_{n(\mathbf{y})} G(\mathbf{r}) \right) dS(\mathbf{y}), \quad (3.11)$$

Taking the ∇ of the first and $\nabla \times$ of the second, and combining the results appropriately (including integration by parts and Stokes’ theorem for surface integrals), leads to the velocity-field representation

$$\begin{aligned} \mathbf{u}(\mathbf{x}) = & \mathbf{u}_\infty + \int_{\Omega} \left[\theta(\mathbf{y}) \mathbf{K}(\mathbf{r}) + \boldsymbol{\omega}(\mathbf{y}) \times \mathbf{K}(\mathbf{r}) \right] dD(\mathbf{y}) \\ & + \int_S \left[u_n(\mathbf{y}) \mathbf{K}(\mathbf{r}) + u_t(\mathbf{y}) (\mathbf{K}(\mathbf{r}) \times \mathbf{r}) \right] dS(\mathbf{y}), \end{aligned} \quad (3.12)$$

where $\theta(\mathbf{y})$ and $\boldsymbol{\omega}(\mathbf{y})$ are scalar- and vector-valued disturbance fields (arising from ϕ, ψ_i), while $u_n(\mathbf{y})$ and $u_t(\mathbf{y})$ denote normal and tangential velocity components on S . Here, $\mathbf{K}(\mathbf{r}) = \nabla G(\mathbf{r})$.

Remarks on Equations.

- The volume term in (3.12) stems directly from the volume integrals in (3.11)–(3.10).
- The boundary terms involving $\partial_{n(\mathbf{y})}$ in (3.11)–(3.10) contribute to the u_n and u_t terms in (3.12).

These integral representations have traditionally formed the foundations of early particle methods (and Boundary element methods). If there are no boundaries, one simply advances the transport equations using the velocity from (3.12), but that convolution integral has an $O(N^2)$ cost. Boundaries worsen this, since surface integrals must also be evaluated. In (3.12), S can represent solid boundaries or any domain interface, leading to integral equations for u_n and u_t , which are expensive to solve directly.

3.4 Particle–Mesh (PM)

A well-known approach to reduce the high cost of evaluating the velocity field from (3.12) is to adopt a *hybrid* strategy combining both particle and grid-based techniques. In this setup, one computes the velocity potential ϕ and/or the stream-function ψ on a *Cartesian mesh* using standard techniques (e.g., finite-differences, finite-volumes, or spectral discretizations). This reliance on a structured grid allows the key $O(N^2)$ convolution in (3.12) to be replaced by, for instance, *fast FFT-based* fast-Poisson solver or *Fast Multipole* methods, often reducing the cost to $O(N \log N)$.

A central ingredient of PM schemes is the process of projecting information *from particles to mesh points* (**P2M**) and interpolating back again (**M2P**). This process is illustrated in the figure below 3.4.

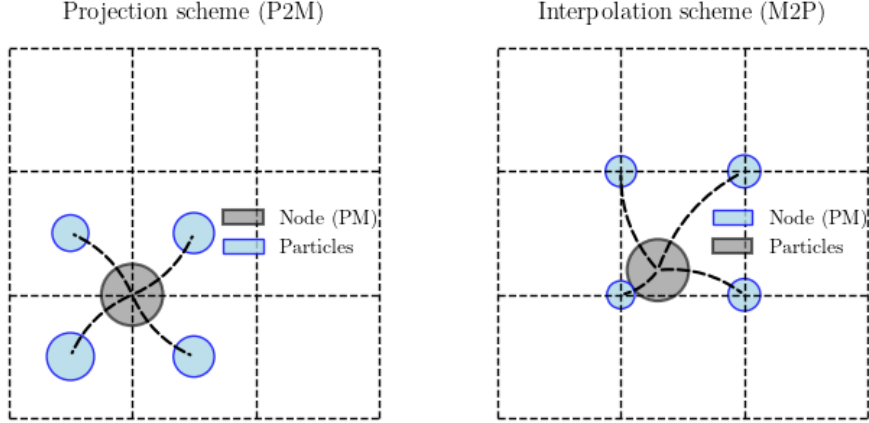


Figure 3.4: Schematic of P2M and M2P transfers in a Particle–Mesh (PM) method. Particles (left) carry physical quantities such as vorticity, which are deposited on a regular mesh (center) via an appropriate kernel or interpolation function. Once solved on the grid, the results (e.g. velocity field) are mapped back to the particle positions (right).

For instance, one typically gathers vorticity (or other flow quantities) from a set of Lagrangian markers $\{x_p\}$ and *deposits* it onto the mesh nodes $\{x_k\}$:

$$V_k \omega_k = \sum_p V_p \Omega_p \phi_k(x_p),$$

where V_k is the volume of the cell surrounding the node x_k , V_p and Ω_p are particle-specific quantities (e.g. volume and vorticity), and $\phi_k(x_p)$ is a chosen *interpolation function* (sometimes a compactly supported kernel). This process draws many analogies to the function approximations developed in section 3.1, since we are once again interested in conserving the integral invariants of quantities and maintain smoothness. For this reason it is often that the same approximation kernels defined in section 3.1 are also used for particle-PM interactions. In the present work both P2M and M2P transfers were computed based on the M'_4 kernel.

Once the flow variables are on the mesh, we solve for ϕ or ψ via fast Poisson solvers (e.g. using an FFT-based approach, multigrid, or direct solvers). Finally, *mesh-to-particle* (**M2P**) interpolation projects the updated flow fields—e.g. velocity or its derivatives—back onto the particle set:

$$\Omega_p = \sum_k \omega_k \phi_k(x_p),$$

which completes one iteration of the hybrid scheme. This combination of a Lagrangian treatment of vorticity transport (where particles naturally track distortion and stretching) with an *Eulerian* Poisson solve drastically lowers computational expense compared to direct $O(N^2)$ convolution, especially in three-dimensional flows.

As a result, Particle–Mesh methods have been widely adopted for complex, large-scale vortex simulations.

3.5 Remeshing

In a purely Lagrangian method, the particle distribution can become distorted as fluid elements cluster or disperse in regions of high strain. This distortion compromises accuracy by weakening the interaction between particles. A representative example is the undisturbed axisymmetric vortex patch studied by Rossinelli and Koumoutsakos [21], where significant shear deforms an initially uniform particle layout. Comparable behaviors—characterized by the emergence of artificial small scales—were also reported by Henshaw et al. and Krasny [14, 18] in two-dimensional, inviscid vortex-sheet calculations.

To mitigate these problems and to ensure proper PM-Particle information sharing, various grid refinement techniques have been developed. The method employed in the current work is that of the frequent physical relocation of the particles unto positions where the overlapping condition is satisfied (remember that we need $h = O(\epsilon)$). This method, called “remeshing”, repositions the particle into regularly ordered positions every N time-steps. This approach, originally proposed by Huberson and Jollès [15] and later extended by Leonard and Koumoutsakos [17], has been widely adopted in vortex methods.

To restore particle regularity, we periodically remesh particles onto a structured grid $\mathcal{G}_h$ with spacing h . The remeshing operator $\mathcal{R} : (\mathbf{X}_p, \Omega_p) \mapsto (\mathbf{X}'_p, \Omega'_p)$ consists of:

1. **Projection (P2M):** Map particle data to grid nodes via convolution:

$$\Gamma_k = \sum_p \Omega_p W \left(\frac{\mathbf{x}_k - \mathbf{X}_p}{h} \right), \quad \mathbf{x}_k \in \mathcal{G}_h \quad (3.13)$$

where W is a compactly supported interpolation kernel.

2. **Interpolation (M2P):** Redistribute to new particles $\mathbf{X}'_p$:

$$\Omega'_p = \sum_k \Gamma_k W \left(\frac{\mathbf{X}'_p - \mathbf{x}_k}{h} \right) \quad (3.14)$$

This process is illustrated on the figure 3.5 (in 2D) for a random initial particle distribution.

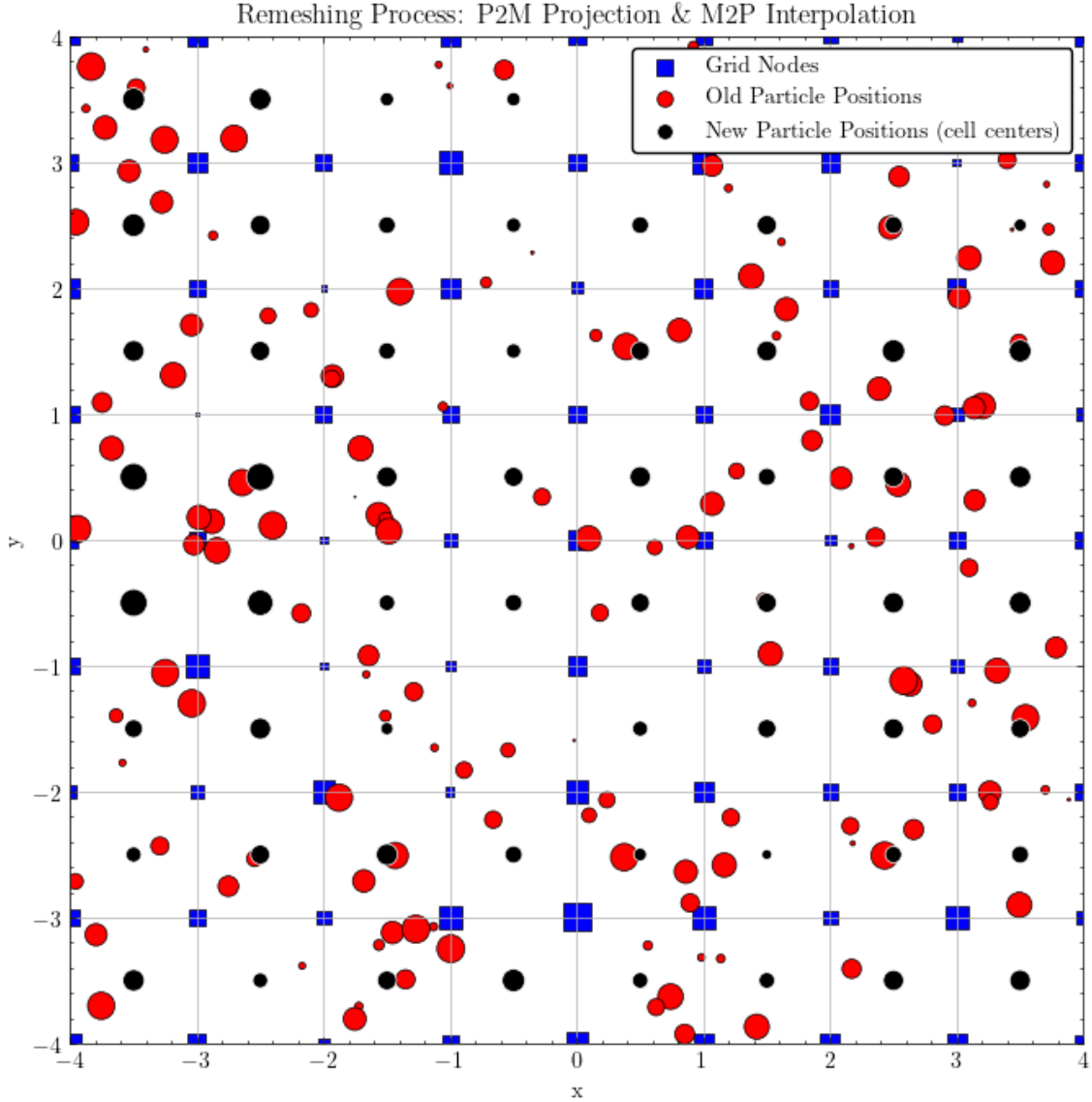


Figure 3.5: Schematic representation of the remeshing process. Red circles indicate the original particle positions. Blue squares represent the grid nodes used in the P2M projection step. Black circles denote the new, regularly spaced particle positions (cell centers), obtained through the M2P interpolation, where the grid values are weighted by the quadratic piecewise projection kernel M_4' . Marker sizes for all shapes are proportional to the field magnitudes.

Another reason for needing to remesh arises when particle volumes are variable. Then, regridding is crucial even in the absence of distortions in the underlying distributions to maintain the overlapping condition. This need arises as interpolation on far-spaced gradients can introduce discontinuities in the flow, as illustrated in figure 3.6. As can be seen a discretization that doesn't match the grid will naturally introduce aliasing that will produce numerical artifacts.

A convenient way to quantify how well a particular interpolation procedure reduces aliasing on a computational mesh, is to define an effectiveness measure, denoted by ε . This measure is the ratio of the amplitude of $\hat{W}$ at an aliased wave number ($k - nk_g$) to its amplitude at the principal wave number k . Mathematically, it can be expressed as

$$\varepsilon = \frac{|\hat{W}(k - nk_g)|}{|\hat{W}(k)|}. \quad (3.15)$$

Interpolation with M3 for Different Particle Spacings

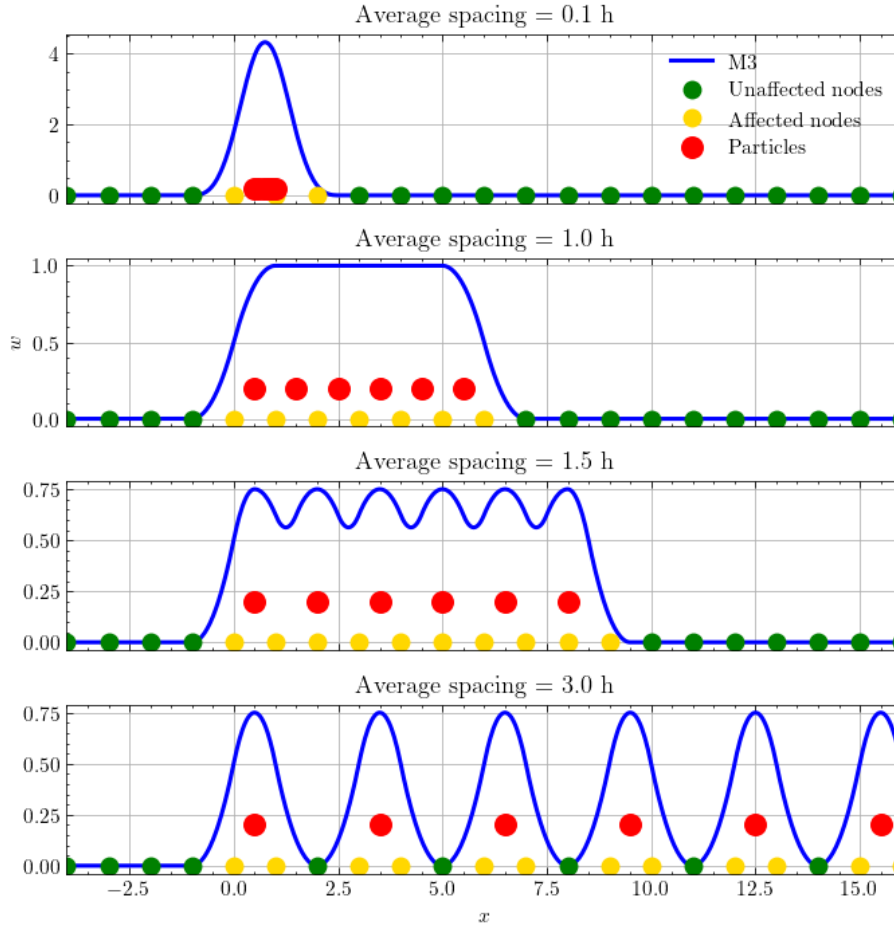


Figure 3.6: Comparison of the M3 interpolation for particle distributions with varying average spacings. The four vertically arranged panels illustrate how the interpolation behaves when particles are uniformly spaced at 0.1, 1, 1.5 units, and 3 units apart. Each panel displays the summed shape function (blue curve), the affected (gold) and unaffected (green) grid nodes, and the particle positions (red circles).

It can be shown [8] that for an m th B-spline:

$$\epsilon = \left(1 - \frac{2\pi N}{k}\right)^{-(m+1)}$$

which show that as m increases, the Fourier transform decays more rapidly at higher wave numbers, yielding smoother functions and thereby reducing aliasing effects.

While remeshing is crucial for maintaining resolution, it can also introduce *numerical diffusion*. For example, a compactly supported kernel W inevitably “spreads” vorticity to some nearby grid points that might previously have had zero vorticity. Over time, repeated remeshing can blur sharp gradients. One way to reduce this artificial diffusion is to perform the remeshing on a *finer* grid than the primary PM mesh as the artificial diffusion introduced is inevitably a function of h and the remeshing frequency Δt_{rm} .

Koumoutsakos and Cottet [8] show that when considering viscous calculations, the remeshing frequency has to satisfy $\nu \Delta t \leq \epsilon^2$, where ϵ is the effectiveness coefficient of the interpolation kernel for each wave-number.

In the present formulation, particles are initialized with volumetric attributes congruent to the underlying PM grid discretization. Let $\mathcal{G}_h = \{\mathbf{x}_k\}_{k=1}^N$ denote the PM grid with cell volume $\Delta V = h^d$, where d is spatial dimension. Each particle p is assigned:

$$V_p(0) = \Delta V, \quad \boldsymbol{\Omega}_p(0) = \boldsymbol{\omega}(\mathbf{X}_p(0), 0) \Delta V \quad (3.16)$$

This ensures discrete conservation of circulation $\Gamma_p = \boldsymbol{\Omega}_p \cdot \mathbf{n}$ and enstrophy $\mathcal{E}_p = \|\boldsymbol{\Omega}_p\|^2 / \Delta V$ through:

$$\sum_p \Gamma_p = \int_{\mathcal{D}} \boldsymbol{\omega}(\mathbf{x}, 0) \cdot \mathbf{n} \, d\mathbf{x} + \mathcal{O}(h^p) \quad (3.17)$$

$$\sum_p \mathcal{E}_p = \int_{\mathcal{D}} \|\boldsymbol{\omega}(\mathbf{x}, 0)\|^2 \, d\mathbf{x} + \mathcal{O}(h^p) \quad (3.18)$$

where $p = 3$ is the kernel order of M'_4 .

To mitigate artificial diffusion effects — particularly interfacial distortion — we implement an enstrophy truncation scheme (based on sorting). Defining the cumulative enstrophy distribution as:

$$F(\mathcal{E}) = \frac{1}{\mathcal{E}_{\text{total}}} \sum_{\{p: \mathcal{E}_p \leq \mathcal{E}\}} \mathcal{E}_p, \quad \mathcal{E}_{\text{total}} = \sum_p \mathcal{E}_p \quad (3.19)$$

Particles are retained if:

$$p \in \mathcal{P}_{\text{active}} \iff F(\mathcal{E}_p) \geq 1 - \epsilon, \quad \epsilon = 10^{-5} \quad (3.20)$$

This removes 0.001% of enstrophy-bearing particles in low-vorticity regions, while preserving:

$$\sum_{p \in \mathcal{P}_{\text{active}}} \mathcal{E}_p \geq (1 - \epsilon) \mathcal{E}_{\text{total}} \quad (3.21)$$

This simple scheme introduces a trade-off between interfacial sharpness and conservation error $\delta \mathcal{E} = \epsilon \mathcal{E}_{\text{total}}$, while also serving as a way to contain particle count and thus all associated computational costs.

3.6 Poisson Solvers

Algorithmically, despite the formulation and particulars of a vortex methods, one finds that the costs are dominated by computing the interactions between particles. Those interactions, as mentioned, naturally lead to an $O(N^2)$ computational cost. The viability and practicality of the Lagrangian methods, therefore, has been determined by designing algorithms and method that can reduce this complexity. In general, the most popular techniques used to address this issue have been to either compute the N-body interactions (Biot-Savart) in an efficient way — as done by Greengard and Rokhlin [12] in 1987 with the introduction of the Fast Multipole Method (FMM) — or by solving the interaction problem on the Particle Mesh. The latter technique, which is used in the current work, tries to take advantage of the underlying mesh by employing the so-called Fast Poisson solvers.

Problem Discretization

Let the computational domain Ω be discretized on a uniform grid

$$\mathcal{G}_h = \{\mathbf{x}_{ijk} = (ih, jh, kh) : i, j, k \in \mathbb{Z}\}$$

with grid spacing h . Using a second-order central difference scheme, the continuous Poisson equation is discretized as

$$\mathbf{L}_h \boldsymbol{\phi} = \mathbf{f}, \quad (3.22)$$

where $\boldsymbol{\phi} \in \mathbb{R}^N$ is the vector of grid-noded potential values, and the discrete Laplacian operator $\mathbf{L}_h \in \mathbb{R}^{N \times N}$ is given by the symmetric 7-point stencil:

$$L_{pq} = \begin{cases} -\frac{6}{h^2}, & p = q, \quad (\text{diagonal}); \\ \frac{1}{h^2}, & p \sim q, \quad (\text{nearest neighbors}); \\ 0, & \text{otherwise.} \end{cases}$$

This discretization incurs a truncation error of $\mathcal{O}(h^2)$, consistent with the second-order finite-difference approximation.

3.6.1 FFT-Based Solvers

In the case of *periodic* boundary conditions, the discrete Fourier transform diagonalizes $\mathbf{L}_h$. Denoting the discrete Fourier transform of the right-hand side f_{ijk} by

$$\tilde{f}_{mnp} = \sum_{i=0}^{N_x-1} \sum_{j=0}^{N_y-1} \sum_{k=0}^{N_z-1} f_{ijk} \exp\left(-2\pi i \left(\frac{im}{N_x} + \frac{jn}{N_y} + \frac{kp}{N_z}\right)\right).$$

Then, the corresponding transform of the potential, $\tilde{\phi}_{mnp}$, is obtained by dividing with the eigenvalue of the discrete Laplacian,

$$\lambda_{mnp} = -\frac{4}{h^2} \left[\sin^2\left(\frac{\pi m}{N_x}\right) + \sin^2\left(\frac{\pi n}{N_y}\right) + \sin^2\left(\frac{\pi p}{N_z}\right) \right],$$

so that

$$\tilde{\phi}_{mnp} = \frac{\tilde{f}_{mnp}}{\lambda_{mnp}}.$$

The solution in physical space follows from the inverse discrete Fourier transform. The algorithmic cost is $\mathcal{O}(N \log N)$, where $N = N_x N_y N_z$. For non-periodic boundaries (Dirichlet, Neumann), one replaces the standard FFT with discrete sine/cosine transforms that enforce the desired homogeneous boundary conditions, and the procedure is analogous but uses the corresponding eigenfunctions.

3.6.2 Cyclic Reduction Algorithm

FFT-based methods require constant coefficients and coordinate-aligned boundaries. In more general *separable* settings, *cyclic reduction* (CR) provides a direct way to solve the resulting linear system. To illustrate the main idea, consider a two-dimensional elliptic PDE of the form

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + b(y) \frac{\partial u}{\partial y} + c(y) u = g(x, y), \quad (3.23)$$

discretized so that for each grid index j in the x -direction we have an equation

$$u_{j-1} + T u_j + u_{j+1} = g_j, \quad j = 1, 2, \dots, J-1, \quad (3.24)$$

where u_j is now a *vector* of unknowns in the y -direction, g_j is the corresponding discrete right-hand side, and T is a (typically tridiagonal) matrix encoding the finite-difference operators in y . The boundary values u_0 and u_J (at $x = 0$ and $x = J$) are assumed to be known.

Step 1: Reduction of Unknowns. Consider three consecutive equations indexed by $j - 1$, j , and $j + 1$. Writing them as

$$\begin{aligned} u_{j-2} + T u_{j-1} + u_j &= g_{j-1}, \\ u_{j-1} + T u_j + u_{j+1} &= g_j, \\ u_j + T u_{j+1} + u_{j+2} &= g_{j+1}. \end{aligned}$$

Multiplying the middle equation by $-T$ and summing all three, after algebraic cancellation one obtains a new equation involving only u_{j-2} , u_j , u_{j+2} :

$$u_{j-2} + T^{(1)} u_j + u_{j+2} = g_j^{(1)},$$

where

$$T^{(1)} = 2I - T^2 \quad \text{and} \quad g_j^{(1)} = \Delta^2 [g_{j-1} - T g_j + g_{j+1}].$$

Hence, *every second unknown* in the x -direction is eliminated, effectively halving the number of equations.

Step 2: Repetition and Coarsest Equation. Repeating this halving process (using the newly formed equations in each stage) continues until only *one* equation remains (often at the grid midpoint $j = J/2$, if J is a power of two). This single equation can be solved directly by standard tridiagonal methods, because $T^{(f)}$ (the matrix after f levels of reduction) is typically also tridiagonal.

Step 3: Back-substitution. After solving for the unknown at the coarsest level, one proceeds *upwards* the levels of reduction. At each intermediate level, the values of the newly found unknowns provide boundary-like data for a small tridiagonal system at that level. Solving it recovers the eliminated unknowns. One continues until all unknowns $u_1, \dots, u_{J-1}$ are determined.

Complexity and Remarks. Cyclic reduction requires solving $(J - 1)$ tridiagonal systems of size corresponding to the vector dimension in y . It is thus very efficient for separable or near-separable problems. If the PDE also has constant coefficients in one direction, one can combine CR with a Fourier transform in the other direction (the *FACR* method) for even faster solutions [24, 4].

3.6.3 Multigrid Solvers

Consider the linear system

$$A_h u_h = b_h,$$

where $A_h \in \mathbb{R}^{n \times n}$ is a symmetric positive definite matrix arising from the discretization of an elliptic partial differential equation (PDE) on a fine grid with mesh size h . Denote by u the exact solution and by u_h its numerical approximation. Then,

the error e is defined as

$$e = u - u_h,$$

and it satisfies

$$A_h e = r_h, \quad \text{with} \quad r_h = b_h - A_h u_h,$$

where r_h is the residual.

Smoothing and the Motivation for Coarse-Grid Correction Standard iterative methods such as weighted Jacobi or Gauss–Seidel are highly effective in damping high-frequency (oscillatory) components of the error. After performing a fixed number ν_1 of pre-smoothing iterations, the remaining error e is predominantly composed of low-frequency (smooth) components, which are known to decay slowly under these iterations.

To address this deficiency, the *multigrid method* transfers the low-frequency error components to a coarser grid, where they appear as higher-frequency errors and can thus be reduced efficiently.

Intergrid Transfer Operators Let the coarse grid have mesh size $2h$. Define:

- A *restriction operator* $R_h^{2h} \in \mathbb{R}^{m \times n}$ (with $m \approx n/2$) that maps fine-grid vectors to the coarse grid. A common choice is the full-weighting operator.
- An *interpolation operator* (or prolongation) $I_{2h}^h \in \mathbb{R}^{n \times m}$ that maps coarse-grid vectors back to the fine grid; typical choices include linear interpolation.

The coarse-grid residual is computed by

$$r_{2h} = R_h^{2h} r_h.$$

The Galerkin Coarse-Grid Operator The coarse-grid operator A_{2h} is defined via the *Galerkin condition*:

$$A_{2h} = R_h^{2h} A_h I_{2h}^h.$$

This definition ensures that the coarse-grid correction is consistent with the fine-grid operator, and it preserves properties such as symmetry and positive definiteness (when A_h possesses them).

Coarse-Grid Correction and the Two-Grid Method On the coarse grid, we solve (or approximately solve) the error equation

$$A_{2h} E_{2h} = r_{2h},$$

to obtain an approximation E_{2h} of the error. This coarse-grid error is then interpolated back to the fine grid:

$$E_h = I_{2h}^h E_{2h}.$$

The fine-grid approximation is updated as

$$u_h \leftarrow u_h + E_h.$$

Defining the *coarse-grid correction operator* S as

$$S = I_{2h}^h (R_h^{2h} A_h I_{2h}^h)^{-1} R_h^{2h} A_h.$$

It can be verified that S is a projection; that is,

$$S^2 = S.$$

Consequently, the updated error after coarse-grid correction is

$$e_{\text{new}} = e - S e = (I - S)e.$$

V-Cycle and Overall Algorithm The two-grid method described above is recursively extended to a hierarchy of grids via the V-cycle (or, alternatively, the W-cycle or full multigrid cycle). A typical V-cycle consists of the following steps (as illustrated in (3.7)):

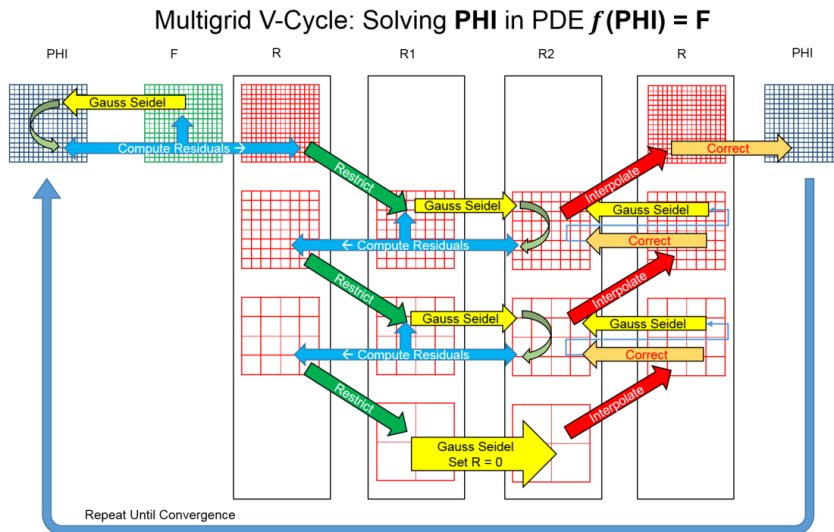


Figure 3.7: Visualization of Multigrid V-Cycle Algorithm

1. **Pre-smoothing:** Apply ν_1 iterations of a smoother $\mathcal{M}_h$ to u_h , reducing high-frequency errors.
2. **Residual Computation:** Compute the residual $r_h = b_h - A_h u_h$.
3. **Restriction:** Transfer the residual to the coarse grid via $r_{2h} = R_h^{2h} r_h$.

4. **Coarse-Grid Correction:** Solve (exactly or approximately) the coarse-grid system

$$A_{2h} E_{2h} = r_{2h},$$

and interpolate the correction to obtain $E_h = I_{2h}^h E_{2h}$. Update the fine-grid solution:

$$u_h \leftarrow u_h + E_h.$$

5. **Post-smoothing:** Apply ν_2 iterations of the smoother to further reduce any high-frequency errors introduced by the interpolation.

Under suitable conditions on the smoother and the intergrid operators, one can prove that the combined multigrid iteration satisfies

$$\|e_{\text{new}}\| \leq \rho \|e\|,$$

with a convergence factor $\rho < 1$ that is independent of h . In practice, values such as $\rho \approx 1/10$ are observed, leading to an overall computational cost of $O(n)$ for problems of size n .

Summary and Solver Selection. In this work, three specialized Poisson solvers were tested and integrated into the VPM code:

- **Fishpack (NCAR)** and **Intel's Fast Poisson Cartesian Solver** both employ *FACL*-type algorithms, combining Fourier Analysis with Cyclic Reduction for efficient handling of separable PDEs. Their performance was found to be similar although rigorous testing was not performed.
- **Mudpack (NCAR)** implements a classical multigrid approach (notably a V-cycle), but requires careful domain subdivision at problem solving. Its parameters (e.g., number of levels and subdomain sizes) must be manually configured for optimal performance, making it somewhat more difficult to automate. In general, the multigrid solver showed better performance compared to the FFT-based ones.

All of these routines are already highly optimized, both in algorithmic design and low-level implementation. Consequently, further gains in performance typically hinge on parallelization strategies that distribute the computation and memory requirements across multiple processors. In such a setup, one can maintain scalability while tackling larger problem sizes, thereby pushing the computational envelope beyond what is possible in a strictly serial environment.

Finally, domain decomposition methods are especially advantageous for unbounded or very large domains, where partitioning the domain across multiple processing units can yield significant parallel speedups. In the current work, a parallel algorithm for splitting and solving the Poisson problem was also employed.

To split the Poisson problem, we begin by partitioning the domain D into N non-

overlapping blocks, where:

$$D = \bigcup_{i=1}^N D_i, \quad f(\mathbf{x}) = \sum_{i=1}^N f_i(\mathbf{x}), \quad w(\mathbf{x}) = \sum_{i=1}^N w_i(\mathbf{x}).$$

In this setup, f_i and w_i are taken as zero-extensions of the corresponding functions restricted to D_i . For each subdomain, it follows that:

$$\nabla^2 w_i(\mathbf{x}) = f_i(\mathbf{x}), \quad \mathbf{x} \in D_i.$$

The steps to solve the system, which are laid out in great detail in [23] are summarized as follows:

- **Step 0: Domain Decomposition:** Subdivide the domain D into N disjoint subdomains D_i . Each subdomain D_i is then treated (conceptually) as an independent region, with forcing f_i and solution component w_i .
- **Step 1a: Free-Space Problems.** Solve the free-space Poisson problem in each extended subdomain D_i using the James–Lackner algorithm [16, 19].
- **Step 1b: Coarse-Grid Solution.** Solve the full (global) problem on a suitably coarse grid.
- **Step 2: Combine and Set Dirichlet Data.** Combine the coarse-grid solution and subgrid solutions to define the Dirichlet boundary data on ∂D for every subdomain D_i .
- **Step 3: Sub-Domain Dirichlet Problems.** Solve the Poisson Dirichlet problem in every sub-domain and obtain the final solutions that can be stitched together.

This process allows us to distribute the problem into different processing units allowing us to exploit robust parallelism at scale, reduce time-to-solution, and deliver maximum performance on modern multi-core processor and HPC / cloud-based architectures.

3.7 Viscous Splitting

The *viscous splitting algorithm* partitions the time integration into two sub-steps, treating advection and diffusion separately. The procedure reads as follows:

1. Advection:

$$\frac{d\mathbf{x}}{dt} = \mathbf{u}(\mathbf{x}_p), \tag{3.25}$$

$$\frac{d\omega}{dt} = 0. \tag{3.26}$$

During this phase, particle positions $\mathbf{x}_p$ are updated according to the local flow velocity $\mathbf{u}$, which is derived from the Biot–Savart integral.

2. Diffusion:

$$\frac{d\mathbf{x}}{dt} = 0, \quad (3.27)$$

$$\frac{d\omega}{dt} = \nu \nabla^2 \omega. \quad (3.28)$$

Here, the vorticity ω is evolved under the action of viscosity, while particle positions remain fixed.

In this manner, the advection step moves particles along characteristic paths determined by the velocity field, whereas the diffusion step captures viscous effects via a separate evolution of ω . Convergence of the viscous splitting method was established in rigorous mathematical terms by Beale and Majda [25].

3.8 Pressure Recovery

A notable feature of the Lagrangian formulation for incompressible flow is that the pressure term does not explicitly appear in the vorticity transport equation. The reason for this is that *pressure cannot create or destroy vorticity*, and it naturally vanishes from the vorticity equation. To see why, recall that the vorticity

$$\boldsymbol{\omega} = \nabla \times \mathbf{u}$$

obeys the transport equation

$$\frac{D\boldsymbol{\omega}}{Dt} = (\boldsymbol{\omega} \cdot \nabla) \mathbf{u} - (\mathbf{u} \cdot \nabla) \boldsymbol{\omega} + \nu \nabla^2 \boldsymbol{\omega},$$

where $\mathbf{u}$ is the velocity field, ν is the kinematic viscosity, and $\frac{D}{Dt}$ denotes the material (Lagrangian) derivative. The pressure appears in the Navier–Stokes equations as a gradient,

$$-\nabla p,$$

but taking the curl of a gradient yields zero:

$$\nabla \times (-\nabla p) = \mathbf{0}.$$

Hence, it vanishes. The physical intuition behind this is that because the pressure force acts uniformly in all directions, it is symmetric and does not induce shear or rotation in the fluid. In more intuitive terms, a purely normal (or volumetric) force cannot impart a net “twisting” motion to fluid elements; it only compresses or expands them uniformly. Consequently, the presence of pressure gradients alone does not generate vorticity.

In classical vortex methods, the evolution of the vorticity (or the positions of vortex particles) is computed without explicit reference to pressure. Once the velocity field is recovered, pressure can be obtained *a posteriori* by solving an appropriate Poisson equation derived from the momentum or continuity constraints. However, with the emergence of hybrid or coupled solvers, such as those described in [23], it becomes necessary to transfer pressure information from the Lagrangian framework to an Eulerian one that explicitly requires it.

In this work, the pressure field is recovered by a two-step process that first involves solving a three-dimensional Poisson equation for an auxiliary field q and then applying an algebraic relation to obtain the pressure p . The procedure is as follows:

– **Step 1: Solve for the q Field.**

The auxiliary field q is determined by solving the 3D Poisson equation

$$\frac{\nabla^2 q}{\rho} = -\nabla \cdot (\mathbf{u} \times \boldsymbol{\omega}) - \frac{\mu}{\rho} \nabla \cdot \nabla^2 \mathbf{u}, \quad (3.29)$$

which can equivalently be written as

$$\frac{\nabla^2 q}{\rho} = -\boldsymbol{\omega}^2 + (\nabla \times \boldsymbol{\omega}) \cdot \mathbf{u} - \frac{\mu}{\rho} \nabla \cdot \nabla^2 \mathbf{u}.$$

This equation is subject to the boundary conditions

$$\frac{\nabla_n q}{\rho} = \vec{n} \left(\frac{\partial \cdot \mathbf{u} \boldsymbol{\omega}}{\partial t} - (\mathbf{u} \times \boldsymbol{\omega}) - \frac{\mu}{\rho} \nabla^2 \mathbf{u} \right), \quad (3.30)$$

– **Step 2: Recover the Pressure Field.**

Once the q field has been computed, the pressure p is obtained from the following algebraic equation:

$$\frac{\partial \phi}{\partial t} + \frac{\mathbf{u}^2}{2} + \frac{p}{\rho} + \frac{q}{\rho} = \text{const.}$$

Since this is an algebraic relation, it can be directly solved for p after evaluating the time derivative of the velocity potential ϕ and the computed q .

It is worth noting that, for bounded problems, solving Equation (3.29) requires the computation of boundary information, which is equivalent to performing surface convolutions, a procedure analogous to the specification of boundary conditions for the vorticity equation (in the determination of ψ). Moreover, because the q equation depends on both the velocity $\mathbf{u}$ and the vorticity $\boldsymbol{\omega}$, it cannot be solved in strict parallel with the primary systems governing ψ and ϕ . Nevertheless, since the equation is a Poisson equation, it can be readily integrated into any vortex particle method that solves such problems. Finally, there is no need to project the q field

onto the particles, as its dynamics are determined in a post-processing step and do not evolve according to their own set of ordinary differential equations.

3.9 Vorticity Correction

Recall vorticity equation (2.30):

$$\frac{\partial \boldsymbol{\omega}}{\partial t} - \nabla \times (\mathbf{u} \times \boldsymbol{\omega}) = \nabla \times \frac{1}{\rho} \nabla \cdot (-pI + \boldsymbol{\sigma}). \quad (3.31)$$

In our formulation in order to handle the term $\nabla \times (\mathbf{u} \times \boldsymbol{\omega})$ we used the vector identity

$$\nabla \times (\mathbf{u} \times \boldsymbol{\omega}) = \mathbf{u}(\nabla \cdot \boldsymbol{\omega}) - \boldsymbol{\omega}(\nabla \cdot \mathbf{u}) + (\boldsymbol{\omega} \cdot \nabla)\mathbf{u} - (\mathbf{u} \cdot \nabla)\boldsymbol{\omega}$$

The first term and second terms in the expansion are both considered zero as $\nabla \cdot \boldsymbol{\omega} = 0$ and $\nabla \cdot \mathbf{u} = 0$ (incompressibility). The fourth term is the convective acceleration part of the material derivative and thus the third term captures the material vortex stretching. This term in the conservative formulation is then rewritten as:

$$\nabla \cdot (\boldsymbol{\omega} \otimes \mathbf{u}) = (\boldsymbol{\omega} \cdot \nabla)\mathbf{u} - \mathbf{u}(\nabla \cdot \boldsymbol{\omega}) = (\boldsymbol{\omega} \cdot \nabla)\mathbf{u}$$

If we compare the two expressions we get that:

$$\mathbf{u}(\nabla \cdot \boldsymbol{\omega}) + (\boldsymbol{\omega} \cdot \nabla)\mathbf{u} = \nabla \cdot (\boldsymbol{\omega} \otimes \mathbf{u}) + 2\mathbf{u}(\nabla \cdot \boldsymbol{\omega})$$

In this expression the terms involving $(\nabla \cdot \boldsymbol{\omega})$ should be zero, giving validity to the conservative scheme. However, in practice, discretization errors lead to a small but nonzero divergence of the vorticity field, that can even accumulate through time, and thereby introduce errors into the calculation of the stretching term. This inaccuracy can adversely affect the calculations, as the standard formulation does not account for these numerical artifacts.

As part of this work it was tried to employ a vorticity correction scheme that corrects these numerical artifacts by removing the divergent part of $\boldsymbol{\omega}$. Despite the theoretical benefits, our testing revealed that the implementation of this approach produced unsatisfactory results, with errors exceeding those observed using the standard conservative formulation. The reason behind this, is that the process of removing the divergence of the vorticity field is fundamentally an ill-posed problem for finite-difference schemes. This point will be addressed with greater rigor in the following chapter.

3.10 The VPM Algorithm

The Vortex-in-Cell (VIC) approaches, on which the current VPM implementation is based, propose a *hybrid* method combining Lagrangian vortex elements (particles) with an underlying *Cartesian grid*. In each fractional-step (or operator-splitting) cycle, the (incompressible) algorithm uses:

1. Poisson equation (velocity computation).

- (a) *Particle-to-mesh (P2M) step*: Assign particle vorticities ω_p to the grid nodes via a P2M projection.
- (b) *Velocity solve*: On the grid, solve

$$\nabla^2 \phi = \nabla \cdot \mathbf{u} = \theta = 0$$

$$\nabla^2 \psi = -\nabla \times \mathbf{u} = -\omega$$

obtaining $\mathbf{u} = \nabla \phi + \nabla \times \psi$ at each node

2. Pressure Recovery.

- (a) *Auxiliary Field Solve*: On the grid, compute the auxiliary pressure field q by solving the 3D Poisson equation

$$\frac{\nabla^2 q}{\rho} = -\nabla \cdot (\mathbf{u} \times \boldsymbol{\omega}) - \frac{\mu}{\rho} \nabla \cdot \nabla^2 \mathbf{u},$$

subject to the appropriate boundary conditions.

- (b) *Pressure Field*: Recover the pressure p from the algebraic relation

$$\frac{\partial \phi}{\partial t} + \frac{\mathbf{u}^2}{2} + \frac{p}{\rho} + \frac{q}{\rho} = \text{const},$$

which can be directly solved for p once the time derivative of the velocity potential ϕ and the computed q are available.

3. Advection.

- (a) *Mesh-to-particle (M2P) step*: Interpolate the velocity field $\mathbf{u}$ from the grid back to the particle locations $\{x_p\}$.
- (b) *Lagrangian update*: Move the particles according to

$$\frac{d\mathbf{X}_p}{dt} = \mathbf{U}_p, \quad \frac{d\boldsymbol{\Omega}_p}{dt} = 0,$$

where $\mathbf{U}_p$ is each particle's velocity (interpolated from the PM). Since there is no stretching or diffusion in this step, the particle vorticity remains constant.

- (c) Optionally remesh the updated particles.

4. Vortex Stretching.

- (a) *Grid-based stretch calculation:* Use the velocity gradient on the grid to compute

$$(\boldsymbol{\omega} \cdot \nabla) \mathbf{u},$$

which reflects the vortex-stretching term. Using the vector calculus identity:

$$\nabla \cdot (\mathbf{A} \otimes \mathbf{T}) = \mathbf{T}(\nabla \cdot \mathbf{A}) + (\mathbf{A} \cdot \nabla) \mathbf{T}$$

we can rewrite the above equation in conservative form (since $\nabla \cdot \boldsymbol{\omega} = 0$) as:

$$\nabla(\boldsymbol{\omega} \otimes \mathbf{u}) = (\boldsymbol{\omega} \cdot \nabla) \mathbf{u} - \mathbf{u}(\nabla \cdot \boldsymbol{\omega}) = (\boldsymbol{\omega} \cdot \nabla) \mathbf{u}$$

This conservative scheme is preferential for conserving flow quantities as explained in [8]. However the underlying assumption that the vorticity field is zero may not hold true numerically and introduce error.

- (b) *M2P step:* Interpolate the vortex stretching term from the grid back to the particle locations.
- (c) *Lagrangian update:* Move the particles according to

$$\frac{d\mathbf{X}_p}{dt} = 0, \quad \frac{d\Omega_p}{dt} = \langle (\boldsymbol{\omega} \cdot \nabla) \mathbf{u} \rangle_p$$

5. Diffusion.

- (a) *Grid-based diffusion solve:* If viscosity is present, calculate the diffusion term on the PM

$$\nu \nabla^2 \boldsymbol{\omega},$$

- (b) *M2P step:* Interpolate the new diffusion term from the grid back to the particle locations.
- (c) *Lagrangian update:* Move the particles according to

$$\frac{d\mathbf{X}_p}{dt} = 0, \quad \frac{d\Omega_p}{dt} = \langle \nu \nabla^2 \boldsymbol{\omega} \rangle_p$$

This fractional-step framework leverages the strengths of both descriptions: *Lagrangian* particles naturally track advection, while the *Eulerian* grid efficiently handles velocity solves, vortex stretching, and diffusion. VIC methods thus combine the robustness of vortex methods with the computational efficiency of mesh-based Poisson solutions, making them effective for large-scale or three-dimensional simulations.

Chapter 4

Numerical Analysis of the PM Method

Numerical schemes are the backbone of computational fluid dynamics (CFD) and other scientific computing applications that involve solving partial differential equations (PDEs). The choice of numerical discretization significantly impacts the accuracy, stability, and efficiency of the simulation. This chapter introduces the fundamental concepts of numerical schemes, while explaining their ramification on the vortex particle method (VPM). The analysis culminates in a Fourier analysis of discrete operators, demonstrating their influence on numerical stability and error propagation.

Convergence and Stability of Vortex Methods

From a theoretical standpoint, the pioneering analyses of Hald [13] and Beale and Majda [2] established rigorous convergence results for the two-dimensional vortex blob approach. Their work examined the interplay between the smoothing radius ϵ and the underlying mesh spacing h , ultimately demonstrating that ϵ must scale with h . This relationship yields an $O(\epsilon)$ convergence order.

Although the three-dimensional case has not been explored as thoroughly, the existing extensions still exhibit a convergence of $O(\epsilon)$ under certain conditions. Efforts by Anderson and Greengard [1] demonstrated that vortex methods, for the 2D and 3D Euler equations, avoid many of the stringent Courant-Friedrichs-Lewy conditions found in conventional schemes. Instead, the primary restriction on the time step emerges from ensuring stable advection of the vortex elements in the velocity field. Their study highlighted that the stability constraint for vortex methods on the time step only depends on the velocity gradient:

$$\Delta t \leq \frac{C}{\|\nabla \mathbf{u}\|_{\infty}},$$

where C is sometimes referred to as the Lagrangian CFL (LCFL) constant. For many practical simulations, C must remain at or below unity, but even then, this condition is generally more relaxed than the classical CFL constraints associated with grid-based methods.

Although these theoretical analyses provide valuable insights into the stability and convergence of vortex methods, they rely on continuous tracking of particle trajec-

tories in a Lagrangian framework and assume inviscid flow. In practice, if particles are frequently relocated (e.g., re-meshed or redistributed), the original Lagrangian assumptions may become less applicable.

For instance, if the particles are re-meshed at every single time step, the method must be treated from a predominantly Eulerian perspective, where standard CFL constraints tend to govern the allowable time step. The remeshing, thus, limits the benefits of a purely Lagrangian approach. For this reason, and in order to examine the viscous effects, this chapter is dedicated to examining the stability and error of calculations on the PM from an Eulerian perspective.

4.1 Fourier and Von Neumann Analysis

Fourier and Von Neumann analysis constitute a fundamental framework for examining the stability and error dynamics of numerical schemes employed in numerical calculations. At its core, Fourier analysis involves decomposing a discrete numerical solution into a spectrum of Fourier modes. By studying the evolution of each individual mode under the discrete operator, one can characterize the amplification and phase errors that the numerical scheme introduces.

Von Neumann stability analysis builds on this idea by assessing the growth factor of each Fourier mode over time. Specifically, it involves replacing the numerical solution with a Fourier mode of the form $e^{i(kx - \omega t)}$ and analyzing the resulting amplification factor, often denoted as $G(k)$, for each wavenumber k . A necessary condition for the stability of the scheme is that the magnitude of $G(k)$ remains bounded by unity (or within a specific acceptable range) for all wavenumbers represented in the discretization.

This approach not only provides insight into the stability characteristics of the scheme but also elucidates the nature of numerical dispersion and diffusion, which are critical in understanding error propagation. In the context of the vortex particle method, such analysis is indispensable, as the discrete operators used to model fluid flow directly influence the accuracy of the computed vorticity and velocity fields.

By performing Fourier/Von Neumann analysis on the discrete operators, we can rigorously determine the conditions under which the numerical scheme remains stable and quantify the potential trade-offs between accuracy and computational efficiency. This chapter ultimately demonstrates how these theoretical tools are applied to assess and enhance the performance of the PM method in vortex particle simulations.

4.2 Fourier Symbol of a Discrete Operator

In numerical analysis of partial differential equations, the *Fourier symbol* of a discrete operator characterizes how the operator acts on individual Fourier modes. It plays a role analogous to the symbol of a differential operator in the continuous

Fourier transform, where differentiation becomes multiplication by a polynomial in the wave-number k .

Definition.

Consider a linear finite-difference operator L acting on a grid function u_i defined on a uniform mesh with spacing Δx . Suppose we represent the solution as a Fourier mode:

$$u_i = e^{ikx_i}, \quad \text{with } x_i = i\Delta x.$$

When the operator L is applied to this mode, linearity implies

$$L e^{ikx_i} = \lambda(k) e^{ikx_i},$$

where $\lambda(k)$ is the *Fourier symbol* of the operator L . In other words, the operator acts as a multiplication by $\lambda(k)$ in Fourier space.

For instance, consider the standard second-order central difference approximation of the second derivative:

$$Lu_i = \frac{u_{i+1} - 2u_i + u_{i-1}}{\Delta x^2}.$$

Substitute the Fourier mode $u_i = e^{ikx_i}$:

$$\begin{aligned} u_{i+1} &= e^{ik(x_i + \Delta x)} = e^{ik\Delta x} e^{ikx_i}, \\ u_{i-1} &= e^{ik(x_i - \Delta x)} = e^{-ik\Delta x} e^{ikx_i}. \end{aligned}$$

Thus,

$$L e^{ikx_i} = \frac{e^{ik\Delta x} - 2 + e^{-ik\Delta x}}{\Delta x^2} e^{ikx_i}.$$

The Fourier symbol is then given by

$$\lambda(k) = \frac{e^{ik\Delta x} - 2 + e^{-ik\Delta x}}{\Delta x^2}.$$

Using the Euler identity

$$e^{ik\Delta x} + e^{-ik\Delta x} = 2 \cos(k\Delta x),$$

this can be simplified to

$$\lambda(k) = -\frac{2[1 - \cos(k\Delta x)]}{\Delta x^2}.$$

Accuracy: The discrepancy between $\lambda(k)$ and $-k^2$ (continuous) for small $k\Delta x$ indicates the truncation error of the finite-difference approximation.

Stability: In explicit time-integration schemes, the time step is often limited by the largest (in magnitude) eigenvalue of the discrete spatial operator. Since the

maximum eigenvalue corresponds to the maximum value of $|\lambda(k)|$ for $k\Delta x \in [0, \pi]$, analyzing the Fourier symbol directly reveals the stability restrictions.

4.3 Diffusion Equation

Let $u : \Omega \times (0, T) \rightarrow \mathbb{R}$ be a sufficiently smooth function representing a scalar field on a spatial domain $\Omega \subset \mathbb{R}^n$ and over the time interval $(0, T)$. The general form of the diffusion (or heat) equation is given by

$$\frac{\partial u(\mathbf{r}, t)}{\partial t} = \nabla \cdot \left[D(u, \mathbf{r}) \nabla u(\mathbf{r}, t) \right],$$

where $D(u, \mathbf{r})$ is the diffusivity which, generally, may depend on the field u and the spatial variable $\mathbf{r}$. Assuming that the diffusivity is constant (as is the case under the VPM assumptions), that is,

$$D(u, \mathbf{r}) \equiv D \quad \text{with } D > 0.$$

the diffusion equation reduces to

$$\frac{\partial u}{\partial t} = D \nabla^2 u,$$

In one spatial dimension, using the coordinate x , the Laplacian simplifies to the second derivative:

$$\nabla^2 u = \frac{\partial^2 u}{\partial x^2}.$$

Thus, the one-dimensional diffusion equation is expressed as

$$\frac{\partial u}{\partial t} = D \frac{\partial^2 u}{\partial x^2}. \quad (4.1)$$

In three-dimensional Cartesian coordinates (x, y, z) , the Laplacian takes the form

$$\nabla^2 u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2}.$$

Therefore, the three-dimensional diffusion equation becomes

$$\frac{\partial u}{\partial t} = D \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} \right). \quad (4.2)$$

1D Second Order Central Difference Scheme.

Using the second-order central difference approximation for the second derivative, the time-stepping scheme is:

$$u_i^{n+1} = u_i^n + \mu (u_{i+1}^n - 2u_i^n + u_{i-1}^n), \quad \text{where } \mu = D\Delta t / \Delta x^2.$$

Transforming to Fourier space (as illustrated above) gives:

$$\hat{u}^{n+1} = \hat{u}^n [1 - 2\mu(1 - \cos(k\Delta x))] = \hat{u}^n (1 + \mu \lambda(k)).$$

The amplification factor of the scheme is then

$$G(k) = \frac{\hat{u}^{n+1}}{\hat{u}^n} = 1 + \mu \lambda(k).$$

$$\Rightarrow G(k) = 1 - 2\mu(1 - \cos(k\Delta x)).$$

The function $1 - \cos(k\Delta x)$ reaches its maximum value 2 when $k\Delta x = \pi$. Hence, the worst-case (most restrictive) amplification factor is

$$G_{\min} = 1 - 2\mu(2) = 1 - 4\mu.$$

For stability (requiring $|G| \leq 1$), we need

$$1 - 4\mu \geq -1 \quad \Rightarrow \quad \mu \leq \frac{1}{2},$$

i.e.

$$\Delta t \leq \frac{\Delta x^2}{2D}.$$

1D Fourth Order Central Difference Scheme

A fourth-order accurate approximation for the second derivative is

$$\left. \frac{\partial^2 u}{\partial x^2} \right|_{x_i} \approx \frac{-u_{i+2}^n + 16u_{i+1}^n - 30u_i^n + 16u_{i-1}^n - u_{i-2}^n}{12\Delta x^2}.$$

Thus the time-stepping scheme is

$$u_i^{n+1} = u_i^n + \mu \left[-\frac{1}{12}u_{i+2}^n + \frac{16}{12}u_{i+1}^n - \frac{30}{12}u_i^n + \frac{16}{12}u_{i-1}^n - \frac{1}{12}u_{i-2}^n \right].$$

Transforming to Fourier space gives:

$$\hat{u}^{n+1} = \hat{u}^n \left\{ 1 + \mu \frac{-e^{2ik\Delta x} + 16e^{ik\Delta x} - 30 + 16e^{-ik\Delta x} - e^{-2ik\Delta x}}{12} \right\}.$$

and

$$\lambda(k) = \frac{-e^{2ik\Delta x} + 16e^{ik\Delta x} - 30 + 16e^{-ik\Delta x} - e^{-2ik\Delta x}}{12\Delta x^2}.$$

A standard calculation (included in appendix A) shows that this symbol can be rewritten as

$$\lambda(k) = -\frac{4}{\Delta x^2} \sin^2\left(\frac{k\Delta x}{2}\right) \left[1 + \frac{1}{3} \sin^2\left(\frac{k\Delta x}{2}\right)\right].$$

Thus, the amplification factor is

$$G(k) = 1 + D\Delta t\lambda(k) = 1 - 4\mu \sin^2\left(\frac{k\Delta x}{2}\right) \left[1 + \frac{1}{3} \sin^2\left(\frac{k\Delta x}{2}\right)\right].$$

Denoting

$$s = \sin^2\left(\frac{k\Delta x}{2}\right),$$

we have

$$G(k) = 1 - 4\mu s \left(1 + \frac{s}{3}\right).$$

Since $s \leq 1$, the worst-case occurs when $s = 1$, giving

$$G_{\min} = 1 - 4\mu \left(1 + \frac{1}{3}\right) = 1 - \frac{16\mu}{3}.$$

Stability requires

$$1 - \frac{16\mu}{3} \geq -1 \implies \mu \leq \frac{3}{8}$$

or

$$\Delta t \leq \frac{3}{8} \frac{\Delta x^2}{D}.$$

3D Second Order Central Difference Scheme

The second order central difference approximation for the Laplacian in three dimensions is:

$$\frac{u_{i,j,k}^{n+1} - u_{i,j,k}^n}{\Delta t} = D \left[\frac{u_{i+1,j,k}^n - 2u_{i,j,k}^n + u_{i-1,j,k}^n}{h^2} + \frac{u_{i,j+1,k}^n - 2u_{i,j,k}^n + u_{i,j-1,k}^n}{h^2} + \frac{u_{i,j,k+1}^n - 2u_{i,j,k}^n + u_{i,j,k-1}^n}{h^2} \right].$$

Since we know the Fourier symbol for the one-dimensional Laplacian, we can directly write down the symbol for the three-dimensional Laplacian as the sum of the

one-dimensional symbols:

$$\lambda(k_x, k_y, k_z) = \lambda(k_x) + \lambda(k_y) + \lambda(k_z)$$

$$\lambda(k_x, k_y, k_z) = -\frac{2}{h^2} \left(1 - \cos(k_x h)\right) - \frac{2}{h^2} \left(1 - \cos(k_y h)\right) - \frac{2}{h^2} \left(1 - \cos(k_z h)\right).$$

Defining

$$\mu = \frac{D \Delta t}{h^2},$$

the corresponding amplification factor becomes

$$G(k_x, k_y, k_z) = 1 - 2\mu \left[\left(1 - \cos(k_x h)\right) + \left(1 - \cos(k_y h)\right) + \left(1 - \cos(k_z h)\right) \right].$$

The most restrictive mode is when

$$\cos(k_x h) = \cos(k_y h) = \cos(k_z h) = -1 \implies 1 - \cos(k h) = 2,$$

so that

$$G_{\min} = 1 - 2\mu (2 + 2 + 2) = 1 - 12\mu.$$

Stability (i.e. $G_{\min} \geq -1$) requires

$$1 - 12\mu \geq -1 \implies \mu \leq \frac{1}{6}$$

or

$$\Delta t \leq \frac{h^2}{6D}.$$

3D Fourth Order Central Difference Scheme

For the fourth-order scheme the one-dimensional approximation is applied in each coordinate:

$$\frac{\partial^2 u}{\partial x^2} \Big|_{(i,j,k)} \approx \frac{-u_{i+2,j,k}^n + 16 u_{i+1,j,k}^n - 30 u_{i,j,k}^n + 16 u_{i-1,j,k}^n - u_{i-2,j,k}^n}{12 h^2},$$

with analogous formulas in y and z . Thus, the full scheme is

$$\begin{aligned} \frac{u_{i,j,k}^{n+1} - u_{i,j,k}^n}{\Delta t} = D \Bigg[& \frac{-u_{i+2,j,k}^n + 16 u_{i+1,j,k}^n - 30 u_{i,j,k}^n + 16 u_{i-1,j,k}^n - u_{i-2,j,k}^n}{12 h^2} \\ & + \frac{-u_{i,j+2,k}^n + 16 u_{i,j+1,k}^n - 30 u_{i,j,k}^n + 16 u_{i,j-1,k}^n - u_{i,j-2,k}^n}{12 h^2} \\ & + \frac{-u_{i,j,k+2}^n + 16 u_{i,j,k+1}^n - 30 u_{i,j,k}^n + 16 u_{i,j,k-1}^n - u_{i,j,k-2}^n}{12 h^2} \Bigg]. \end{aligned}$$

The Fourier symbol for the Laplacian in one dimension was derived as:

$$\lambda_x(k_x) = -\frac{4}{h^2} \sin^2\left(\frac{k_x h}{2}\right) \left[1 + \frac{1}{3} \sin^2\left(\frac{k_x h}{2}\right)\right].$$

Similarly, we have

$$\lambda_y(k_y) = -\frac{4}{h^2} \sin^2\left(\frac{k_y h}{2}\right) \left[1 + \frac{1}{3} \sin^2\left(\frac{k_y h}{2}\right)\right],$$

$$\lambda_z(k_z) = -\frac{4}{h^2} \sin^2\left(\frac{k_z h}{2}\right) \left[1 + \frac{1}{3} \sin^2\left(\frac{k_z h}{2}\right)\right].$$

Thus, the overall Fourier symbol for the Laplacian is

$$\lambda(k_x, k_y, k_z) = \lambda_x(k_x) + \lambda_y(k_y) + \lambda_z(k_z).$$

The amplification factor is then

$$G(k_x, k_y, k_z) = 1 + D \Delta t \lambda(k_x, k_y, k_z).$$

That is

$$\begin{aligned} G(k_x, k_y, k_z) = 1 - 4\mu & \left[\sin^2\left(\frac{k_x h}{2}\right) \left(1 + \frac{1}{3} \sin^2\left(\frac{k_x h}{2}\right)\right) \right. \\ & + \sin^2\left(\frac{k_y h}{2}\right) \left(1 + \frac{1}{3} \sin^2\left(\frac{k_y h}{2}\right)\right) \\ & \left. + \sin^2\left(\frac{k_z h}{2}\right) \left(1 + \frac{1}{3} \sin^2\left(\frac{k_z h}{2}\right)\right) \right]. \end{aligned}$$

The worst-case occurs when each $\sin^2(\cdot) = 1$. Then, each bracket becomes

$$1 \left(1 + \frac{1}{3}\right) = \frac{4}{3},$$

and the sum over the three directions is

$$\frac{4}{3} + \frac{4}{3} + \frac{4}{3} = 4.$$

Thus,

$$G_{\min} = 1 - 4\mu (4) = 1 - 16\mu.$$

Stability requires

$$1 - 16\mu \geq -1 \quad \implies \quad \mu \leq \frac{1}{8}$$

or

$$\Delta t \leq \frac{h^2}{8D}.$$

Summary of Stability Limits, Amplification Factors, and their Ramifications

Given that $\mu = \frac{D\Delta t}{\Delta x^2}$ in this section we have derived the following:

1D Second-order accuracy:

$$G(k) = 1 - 2\mu [1 - \cos(k\Delta x)], \quad \Delta t \leq \frac{\Delta x^2}{2D}$$

1D Fourth-order accuracy:

$$G(k) = 1 - 4\mu \sin^2\left(\frac{k\Delta x}{2}\right) \times \left[1 + \frac{1}{3} \sin^2\left(\frac{k\Delta x}{2}\right)\right], \quad \Delta t \leq \frac{3}{8} \frac{\Delta x^2}{D}$$

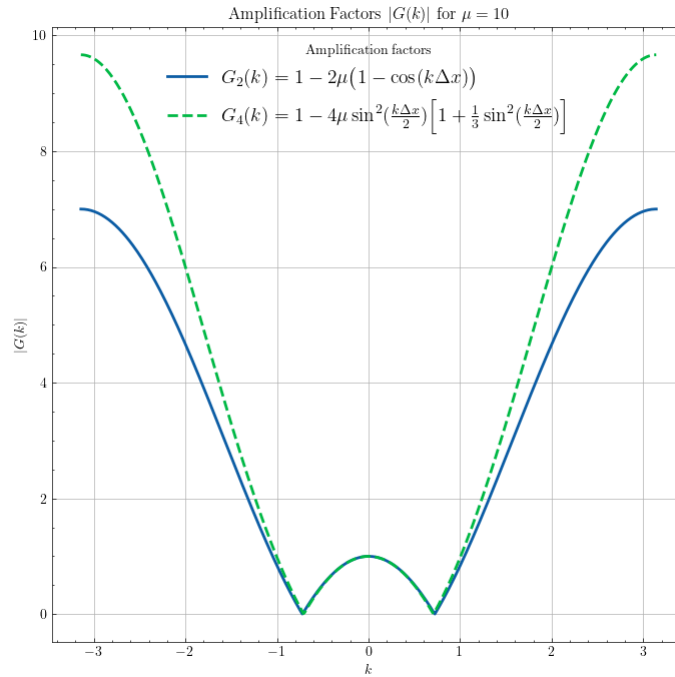


Figure 4.1: Plots of the amplification factors $G(k)$ for second-order $G_2(k)$ and fourth-order $G_4(k)$ accuracy schemes in the one-dimensional case, illustrating the severe amplification of numerical errors at higher frequencies when the stability criterion is violated ($\mu = 10$).

As can be seen in figure 4.1 the amplification is more dominant at higher frequencies, especially when the stability limit is exceeded.

Three-Dimensional Domain

Assuming uniform mesh spacing $h = \Delta x = \Delta y = \Delta z$, the conditions become more restrictive:

Second-order accuracy:

$$G(k_x, k_y, k_z) = 1 - 2\mu [(1 - \cos(k_x h)) + (1 - \cos(k_y h)) + (1 - \cos(k_z h))], \quad \Delta t \leq \frac{h^2}{6D}, \quad \text{where } \mu = \frac{D\Delta t}{h^2}$$

Fourth-order accuracy:

$$G(k_x, k_y, k_z) = 1 - 4\mu \left[\sin^2\left(\frac{k_x h}{2}\right) \left(1 + \frac{1}{3} \sin^2\left(\frac{k_x h}{2}\right)\right) + \sin^2\left(\frac{k_y h}{2}\right) \left(1 + \frac{1}{3} \sin^2\left(\frac{k_y h}{2}\right)\right) + \sin^2\left(\frac{k_z h}{2}\right) \left(1 + \frac{1}{3} \sin^2\left(\frac{k_z h}{2}\right)\right) \right], \quad \Delta t \leq \frac{h^2}{8D}$$

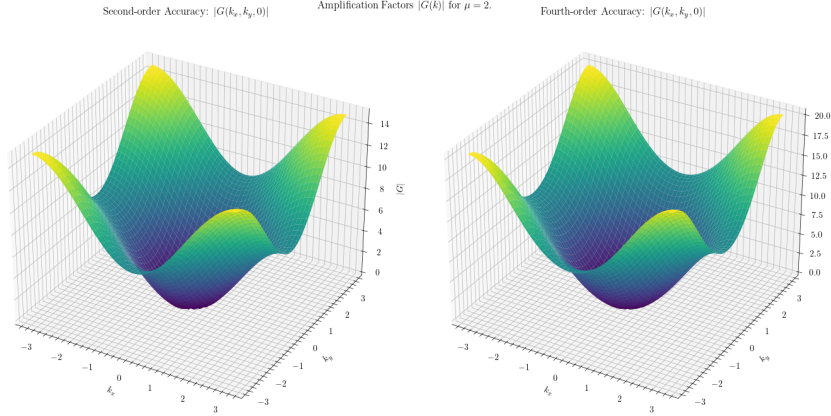


Figure 4.2: Plots of the amplification factors $G(k)$ for second-order $G_2(k)$ and fourth-order $G_4(k)$ accuracy schemes in the three-dimensional case, illustrating the severe amplification of numerical errors at higher frequencies when the stability criterion is violated ($\mu = 10$).

The amplification factors clearly demonstrate that numerical errors are most severely amplified at high frequencies (maximums are at $k = \pi$), particularly in three dimen-

sions, due to additive contributions from each spatial direction. This effect becomes especially critical at large viscosities, characteristic of low Reynolds number flows, where diffusion dominates and high-frequency numerical inaccuracies rapidly amplify. Consequently, strict adherence to stability criteria during the PM solve, is essential for maintaining accuracy and stability within the VPM framework.

4.4 Vorticity Correction Schemes

In the context of the Vortex Particle Method (VPM), accurate preservation of the solenoidal property of the vorticity field is critical. During the numerical integration of the vorticity transport equation, discretization errors inevitably accumulate, causing the computed vorticity field $\boldsymbol{\omega}$ to lose its divergence-free property. Consequently, the numerical solution no longer strictly satisfies the solenoidal condition $\nabla \cdot \boldsymbol{\omega} = 0$, potentially impacting the accuracy and stability of subsequent calculations.

To remedy this numerical divergence, we perform a Helmholtz decomposition of the computed vorticity field:

$$\boldsymbol{\omega} = \boldsymbol{\omega}_s + \boldsymbol{\omega}_c = \nabla \Omega_s + \nabla \times \boldsymbol{\Omega}_c. \quad (4.3)$$

Taking the divergence of this expression, we arrive at the following Poisson equation:

$$\nabla \cdot \boldsymbol{\omega} = \nabla^2 \Omega_s, \quad (4.4)$$

from which we can explicitly obtain the scalar potential Ω_s representing the non-solenoidal component of the vorticity. With this scalar field known, the solenoidal part of the vorticity can subsequently be recovered through the following correction:

$$\boldsymbol{\omega}^* = \boldsymbol{\omega} - \nabla \Omega_s. \quad (4.5)$$

The above correction scheme, although theoretically sound, exhibited unsatisfactory performance upon numerical testing. Specifically, it was observed that the updated vorticity field continued to retain a non-negligible residual divergence, thus compromising the effectiveness of the correction. To elucidate this phenomenon, we analyze our numerical scheme through the lens of Fourier analysis.

Our aim in this section is to rigorously investigate the properties of the discrete gradient ∇_d , divergence $\nabla_d \cdot$, and Laplacian Δ_d operators. Fourier analysis will reveal that the composite operator $\nabla_d \cdot \nabla_d$ does not exactly coincide with the discrete Laplacian Δ_d . This subtle yet significant mismatch between the discrete operators explains the observed residual divergence, thus identifying a fundamental numerical limitation inherent in the correction scheme.

In particular, for these operators we consider second-order finite-difference schemes on a uniform three-dimensional grid with spacing h in each coordinate direction.

Grid points are defined as:

$$x_i = i h, \quad y_j = j h, \quad z_k = k h,$$

with the discrete function values given by $f_{i,j,k} = f(x_i, y_j, z_k)$.

Discrete Gradient

For a scalar function f , the second-order central-difference approximation for the gradient is given by

$$\begin{aligned} (\nabla_d f)_x|_{i,j,k} &= \frac{f_{i+1,j,k} - f_{i-1,j,k}}{2h}, \\ (\nabla_d f)_y|_{i,j,k} &= \frac{f_{i,j+1,k} - f_{i,j-1,k}}{2h}, \\ (\nabla_d f)_z|_{i,j,k} &= \frac{f_{i,j,k+1} - f_{i,j,k-1}}{2h}. \end{aligned}$$

For the x -component, applying the central difference gives:

$$\begin{aligned} (\nabla_d f)_x|_{i,j,k} &= \frac{1}{2h} \left(e^{i\xi_x h} - e^{-i\xi_x h} \right) \hat{f} e^{i(\xi_x i h + \xi_y j h + \xi_z k h)} \\ &= \frac{i \sin(\xi_x h)}{h} \hat{f} e^{i(\xi_x i h + \xi_y j h + \xi_z k h)}. \end{aligned}$$

Similarly for the other components. Hence, the Fourier symbol for the gradient is

$$\sigma_{\nabla}(\boldsymbol{\xi}) = \left(\frac{i \sin(\xi_x h)}{h}, \frac{i \sin(\xi_y h)}{h}, \frac{i \sin(\xi_z h)}{h} \right).$$

Discrete Divergence of the Gradient

For a vector field $\mathbf{g} = (g_x, g_y, g_z)$, the discrete divergence is approximated by

$$(\nabla_d \cdot \mathbf{g})|_{i,j,k} = \frac{g_{x,i+1,j,k} - g_{x,i-1,j,k}}{2h} + \frac{g_{y,i,j+1,k} - g_{y,i,j-1,k}}{2h} + \frac{g_{z,i,j,k+1} - g_{z,i,j,k-1}}{2h}.$$

Taking the divergence of the discrete gradient, we have

$$\sigma_{\nabla_d \cdot \nabla_d}(\boldsymbol{\xi}) = \frac{i \sin(\xi_x h)}{h} \cdot \frac{i \sin(\xi_x h)}{h} + \frac{i \sin(\xi_y h)}{h} \cdot \frac{i \sin(\xi_y h)}{h} + \frac{i \sin(\xi_z h)}{h} \cdot \frac{i \sin(\xi_z h)}{h}.$$

Since $i^2 = -1$, this becomes

$$\sigma_{\nabla_d \cdot \nabla_d}(\boldsymbol{\xi}) = -\frac{1}{h^2} \left[\sin^2(\xi_x h) + \sin^2(\xi_y h) + \sin^2(\xi_z h) \right].$$

Using the double-angle formula

$$\sin^2(\xi h) = 4 \sin^2\left(\frac{\xi h}{2}\right) \cos^2\left(\frac{\xi h}{2}\right),$$

we rewrite the divergence of the gradient as

$$\begin{aligned} \sigma_{\nabla_d \cdot \nabla_d}(\boldsymbol{\xi}) = & -\frac{4}{h^2} \left[\sin^2\left(\frac{\xi_x h}{2}\right) \cos^2\left(\frac{\xi_x h}{2}\right) \right. \\ & + \sin^2\left(\frac{\xi_y h}{2}\right) \cos^2\left(\frac{\xi_y h}{2}\right) \\ & \left. + \sin^2\left(\frac{\xi_z h}{2}\right) \cos^2\left(\frac{\xi_z h}{2}\right) \right]. \end{aligned} \quad (4.6)$$

Discrete Laplacian

For the discrete Laplacian, applying the Fourier transform yields

$$(\Delta_d f)_{i,j,k} = \frac{1}{h^2} \left(e^{i\xi_x h} + e^{-i\xi_x h} + e^{i\xi_y h} + e^{-i\xi_y h} + e^{i\xi_z h} + e^{-i\xi_z h} - 6 \right) \hat{f} e^{i(\xi_x i h + \xi_y j h + \xi_z k h)}.$$

Noting that $e^{i\xi h} + e^{-i\xi h} = 2 \cos(\xi h)$, we obtain

$$\sigma_{\Delta_d}(\boldsymbol{\xi}) = \frac{2}{h^2} \left[\cos(\xi_x h) + \cos(\xi_y h) + \cos(\xi_z h) - 3 \right].$$

Using the identity

$$\cos(\xi h) - 1 = -2 \sin^2\left(\frac{\xi h}{2}\right),$$

this can be rewritten as

$$\sigma_{\Delta_d}(\boldsymbol{\xi}) = -\frac{4}{h^2} \left[\sin^2\left(\frac{\xi_x h}{2}\right) + \sin^2\left(\frac{\xi_y h}{2}\right) + \sin^2\left(\frac{\xi_z h}{2}\right) \right].$$

Comparing the Two Symbols

We compare:

$$\sigma_{\Delta_d}(\boldsymbol{\xi}) = -\frac{4}{h^2} \left[\sin^2\left(\frac{\xi_x h}{2}\right) + \sin^2\left(\frac{\xi_y h}{2}\right) + \sin^2\left(\frac{\xi_z h}{2}\right) \right].$$

with

$$\begin{aligned} \sigma_{\nabla_d \cdot \nabla_d}(\boldsymbol{\xi}) = & -\frac{4}{h^2} \left[\sin^2 \left(\frac{\xi_x h}{2} \right) \cos^2 \left(\frac{\xi_x h}{2} \right) + \sin^2 \left(\frac{\xi_y h}{2} \right) \cos^2 \left(\frac{\xi_y h}{2} \right) \right. \\ & \left. + \sin^2 \left(\frac{\xi_z h}{2} \right) \cos^2 \left(\frac{\xi_z h}{2} \right) \right]. \end{aligned} \quad (4.7)$$

For these symbols to be identical for all wave numbers, we would need

$$\cos^2 \left(\frac{\xi_\alpha h}{2} \right) = 1, \quad \text{for } \alpha = x, y, z,$$

which holds only in the low-frequency limit ($\xi_\alpha h \approx 0$). In general however, $\cos^2(\xi_\alpha h/2) \neq 1$, and thus

$$\sigma_{\nabla_d \cdot \nabla_d}(\boldsymbol{\xi}) \neq \sigma_{\Delta_d}(\boldsymbol{\xi}).$$

Implications for the Helmholtz Correction Scheme

In the Helmholtz decomposition correction, we solve the discrete Poisson equation

$$\Delta_d \Omega_s = (\nabla_d \cdot \omega)_d,$$

and then define the corrected vorticity as

$$\omega^* = \omega - \nabla_d \Omega_s.$$

Taking the divergence of the corrected field in Fourier space gives:

$$\widehat{\nabla_d \cdot \omega^*}(\boldsymbol{\xi}) = (\widehat{\nabla_d \cdot \omega})(\boldsymbol{\xi}) - \sigma_{\nabla_d \cdot \nabla_d}(\boldsymbol{\xi}) \hat{\Omega}_s(\boldsymbol{\xi}).$$

Since by construction

$$\sigma_{\Delta_d}(\boldsymbol{\xi}) \hat{\Omega}_s(\boldsymbol{\xi}) = (\widehat{\nabla_d \cdot \omega})(\boldsymbol{\xi}),$$

it follows that

$$\widehat{\nabla_d \cdot \omega^*}(\boldsymbol{\xi}) = [\sigma_{\Delta_d}(\boldsymbol{\xi}) - \sigma_{\nabla_d \cdot \nabla_d}(\boldsymbol{\xi})] \hat{\Omega}_s(\boldsymbol{\xi}).$$

Since in general, $\sigma_{\Delta_d}(\boldsymbol{\xi}) \neq \sigma_{\nabla_d \cdot \nabla_d}(\boldsymbol{\xi})$, a residual divergence remains:

$$\widehat{\nabla_d \cdot \omega^*}(\boldsymbol{\xi}) \neq 0.$$

The analysis conducted in this section underscores a fundamental numerical limitation inherent in the discrete Helmholtz decomposition correction scheme. Specifically, we have demonstrated that the discrete divergence of the discrete gradient operator, $\nabla_d \cdot \nabla_d$, does not exactly coincide with the discrete Laplacian operator, Δ_d ,

beyond the low-frequency regime. Consequently, this discrepancy results in a persistent residual divergence in the corrected vorticity field, even after the Helmholtz decomposition correction is applied. Such residual divergence, arising inherently from numerical discretization errors, implies that the standard Helmholtz-based correction approach is fundamentally limited and incapable of fully eliminating divergence induced by numerical noise. Therefore, alternative correction strategies or higher-order discretization schemes must be explored.

Chapter 5

Compact Finite-Difference Schemes: Improved Spectral Accuracy

Many physical phenomena—fluid mechanics in particular—span a broad spectrum of spatial and temporal scales, necessitating that numerical simulations accurately capture all relevant dynamics. While finite difference schemes are traditionally assessed by their formal order of accuracy (e.g., truncation error), they also exhibit inherent resolution characteristics. These characteristics determine how effectively the finite difference approximation replicates the exact operator across the entire range of scales supported by a given mesh. Fourier analysis offers a rigorous framework for this evaluation, revealing that certain wave-number ranges are poorly resolved—a region commonly referred to as the *nonapproximation range*. In the previous chapter, we showed that this nonapproximation range makes the vorticity correction problem ill posed.

Additionally, standard explicit finite-difference approximations for the Laplacian—such as the second-order or classical fourth-order schemes—tend to have large maximum eigenvalues. These large eigenvalues, which correspond to high-frequency components in discrete Fourier space, enforce limited timesteps when using explicit time integration methods.

To overcome this limitation, we first consider the compact finite difference for approximating the derivatives. Compact schemes are a class of implicit finite-difference methods that achieve high-order accuracy and excellent stability properties while using a relatively small stencil. In these schemes, the derivative at a grid point is approximated not only by function values at nearby points but also involves the derivative (or second derivative) values at neighboring grid points.

A minor disadvantage is that compact schemes are implicit and require solving a banded matrix system for the evaluation of derivatives at all grid points. This however, is not as severe since the solution can be obtained in $O(n)$ time using, for instance, Thomas' algorithm.

5.1 Overview of Compact Finite Difference Schemes

A compact finite-difference scheme approximates a derivative by establishing a linear relation between the derivative at a grid point and the function values (or derivatives)

at a small number of neighboring points. A compact scheme for approximating $u''(x_i)$ on a uniform grid $\{x_i = i \Delta x\}$ may be written in the general implicit form (see, e.g., [20]):

$$\alpha (u''_{i+1} + u''_{i-1}) + u''_i = \frac{a (u_{i+1} - 2u_i + u_{i-1})}{\Delta x^2} + \frac{b (u_{i+2} - 2u_i + u_{i-2})}{\Delta x^2} + \frac{c (u_{i+3} - 2u_i + u_{i-3})}{\Delta x^2}. \quad (5.1)$$

Here, a , b , c , and α are real coefficients chosen so that the finite-difference approximation on the right-hand side matches the Taylor expansion of u'' to the desired order. By restricting the highest offset to $(i \pm 2)$, one obtains a *tri-diagonal* system once the left-hand side is written out for all i . A comprehensive practical review of compact schemes is found in Lele's work [20].

In this section, we present a **sixth-order** tri-diagonal compact scheme for the second derivative. The resulting linear system remains banded with bandwidth 3, so it can be inverted efficiently using the Thomas algorithm in $O(n)$ operations. Moreover, its Fourier symbol closely approximates the exact symbol $-k^2$ of the continuous Laplacian, thereby reducing high-frequency dissipation errors and relaxing the stability constraint.

In the specific scheme we employ, the following coefficients give **sixth-order** accuracy for $u''(x_i)$ while keeping the system tri-diagonal:

$$\alpha = \frac{2}{11}, \quad a = \frac{12}{11}, \quad b = \frac{3}{11}, \quad c = 0. \quad (5.2)$$

Hence, the final compact stencil becomes:

$$\frac{2}{11} (u''_{i+1} + u''_{i-1}) + u''_i = \frac{12}{11 \Delta x^2} (u_{i+1} - 2u_i + u_{i-1}) + \frac{3}{11 \Delta x^2} (u_{i+2} - 2u_i + u_{i-2}). \quad (5.3)$$

Solving (5.3) for $\{u''_i\}$ across all interior points yields u''_i as an implicit combination of u_i , $u_{i\pm 1}$, $u_{i\pm 2}$. Since no $(i \pm 3)$ terms appear, the coefficient matrix is banded with bandwidth 3 and can be inverted in $O(n)$ time. To verify that (5.3) indeed achieves sixth-order accuracy, one typically expands $u(x_i \pm \Delta x)$ and $u(x_i \pm 2\Delta x)$ in Taylor series about x_i . Matching coefficients of Δx^n on both sides confirms that

$$u''(x_i) = u''_i + C \Delta x^6 u^{(8)}(x_i) + \mathcal{O}(\Delta x^8),$$

where C is some constant. This shows that the leading local truncation error term arises at $\mathcal{O}(\Delta x^6)$, consistent with a *sixth-order* scheme.

5.2 Fourier Symbol and Spectral Accuracy

An alternative and illuminating perspective comes from examining the *Fourier symbol* of the discrete operator. Let k be the continuous wavenumber and set $\theta = k \Delta x$. Suppose $u_i = e^{ikx_i}$, so that the exact second derivative is $-k^2 e^{ikx_i}$. We write the numerical approximation as $u_i'' = D_{\text{comp}}(k) e^{ikx_i}$, for some function $D_{\text{comp}}(k)$ we wish to identify.

Substitution into the Fourier representation into the the left-hand side (LHS) of equation (5.3) yields:

$$\begin{aligned} \text{LHS} &= \frac{2}{11} (u_{i+1}'' + u_{i-1}'') + u_i'' \\ &= D_{\text{comp}}(k) e^{ikx_i} \left[1 + \frac{2}{11} (e^{i\theta} + e^{-i\theta}) \right] \\ &= D_{\text{comp}}(k) e^{ikx_i} \left[1 + \frac{4}{11} \cos \theta \right]. \end{aligned}$$

Similarly the the right-hand side (RHS) becomes:

$$\begin{aligned} u_{i+1} - 2u_i + u_{i-1} &= e^{ikx_i} (e^{i\theta} - 2 + e^{-i\theta}) = 2e^{ikx_i} [\cos \theta - 1], \\ u_{i+2} - 2u_i + u_{i-2} &= 2e^{ikx_i} [\cos(2\theta) - 1]. \end{aligned}$$

Hence,

$$\begin{aligned} \text{RHS} &= \frac{12}{11 \Delta x^2} [2e^{ikx_i} (\cos \theta - 1)] + \frac{3}{11 \Delta x^2} [2e^{ikx_i} (\cos(2\theta) - 1)] \\ &= \frac{2e^{ikx_i}}{11 \Delta x^2} [12 (\cos \theta - 1) + 3 (\cos(2\theta) - 1)]. \end{aligned}$$

Equating LHS = RHS and canceling e^{ikx_i} gives

$$D_{\text{comp}}(k) \left[1 + \frac{4}{11} \cos \theta \right] = \frac{2}{11 \Delta x^2} [12 (\cos \theta - 1) + 3 (\cos(2\theta) - 1)]. \quad (5.4)$$

Thus,

$$D_{\text{comp}}(k) = -\frac{\frac{2}{11 \Delta x^2} [12 (1 - \cos \theta) + 3 (1 - \cos(2\theta))]}{1 + \frac{4}{11} \cos \theta}.$$

A standard small- θ expansion of $\cos \theta$ (with $\theta = k \Delta x$) shows that

$$D_{\text{comp}}(k) = -k^2 + \mathcal{O}(k^4 \Delta x^2).$$

In other words, *at low wavenumbers* $k \Delta x \ll 1$, the scheme's symbol agrees with $-k^2$ up to leading error terms of order θ^4 , confirming **fourth-order** spectral equivalence

to the continuous second derivative. The resolution characteristics of this scheme are visualized in figure 5.1. As can be seen the compact scheme encompasses much lower resolution error in the high frequency content of the data.

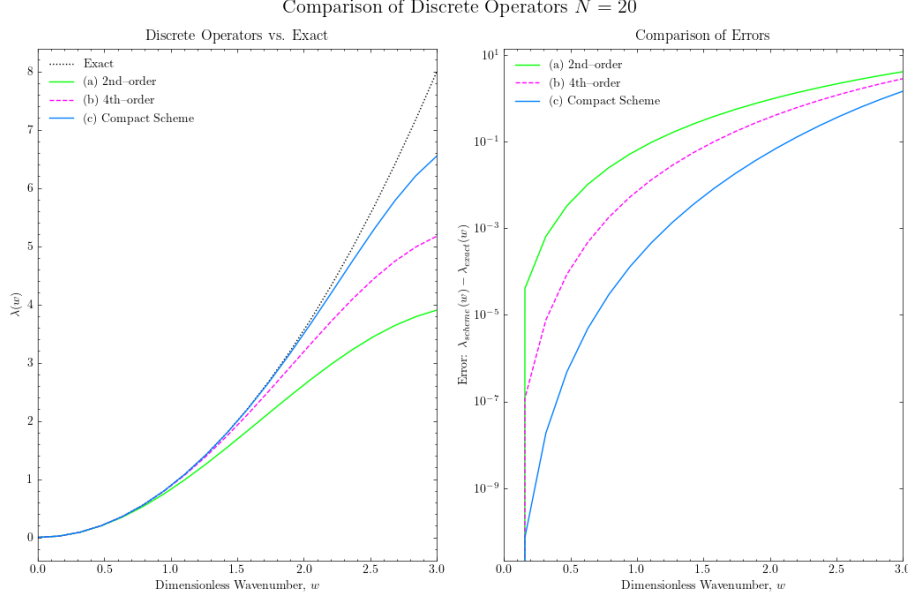


Figure 5.1: Comparison of discrete scheme resolution characteristics across a range of dimensionless wavenumbers, showing the differencing error for various schemes: (a) second-order central differences; (b) fourth-order central differences; (c) Presented Compact Scheme (5.3).

5.3 Modified Stencil Compact Schemes

The methodology outlined above illustrates the effectiveness and versatility of compact finite difference schemes. A natural extension of these concepts is to consider their applicability beyond single-direction derivatives to multi-directional derivatives or, more generally, arbitrary linear differential operators. By appropriately choosing finite-difference stencil coefficients in multiple spatial dimensions, one can develop compact approximations to virtually any linear operator, such as gradients, divergences, curls, or even more complex differential expressions encountered in PDE systems.

To illustrate this generalization, consider a linear differential operator in three-dimensional space of the form:

$$\mathcal{L}(u) = a \frac{\partial^2 u}{\partial x^2} + b \frac{\partial^2 u}{\partial y^2} + c \frac{\partial^2 u}{\partial z^2} + d \frac{\partial^2 u}{\partial x \partial y} + e \frac{\partial^2 u}{\partial x \partial z} + f \frac{\partial^2 u}{\partial y \partial z} + (\text{lower-order terms}).$$

Each derivative in this operator can be approximated using finite-difference stencils designed specifically for the spatial arrangement of grid points. By using compact

stencils that incorporate neighboring points in multiple directions, one achieves a high-order accurate representation of these derivatives.

However, this generalization comes at a computational cost. Unlike the previously described tri-diagonal systems arising from one-dimensional compact schemes, multi-dimensional compact stencils naturally produce larger and more complex coefficient matrices. Specifically, the linear systems obtained from these approximations are no longer tri-diagonal but become banded or even block-banded, significantly widening their bandwidth. Consequently, efficient algorithms such as the Thomas algorithm, optimal for tri-diagonal matrices, are not directly applicable.

To address this challenge, one typically resorts to specialized linear algebraic solvers capable of efficiently handling banded matrices or adopts iterative solution methods (e.g., multigrid, Krylov subspace methods) that exploit the structure of the operator and the sparsity of the resulting matrix.

In this section, we explore an alternative and potentially efficient strategy. Specifically, we consider the possibility of splitting the original complex differential operator equation into a set of simpler equations. By carefully structuring these simpler equations to ensure that each has a Laplacian operator on its left-hand side, we can leverage fast Fourier transform (FFT)-based methods, which are solvable in $O(N \log N)$ operations. However, this strategy places increased complexity on the right-hand side (RHS), which must absorb all the non-Laplacian terms originally present in the operator. Consequently, the RHS might become contaminated losing its physical meaning while necessitating additional treatment or approximations to maintain overall computational efficiency.

To concretely illustrate this methodology, we develop a compact finite-difference algorithm to approximate the second derivative u'' in one dimension. We split the approximation into three separate problems using the decomposition

$$u'' = u''^1 + u''^2 + u''^3,$$

where each component u''^j is obtained by solving an individual modified Poisson-type equation:

$$-u''^j_{i-1} + 2u''^j_i - u''^j_{i+1} = \text{RHS}_j, \quad j = 1, 2, 3.$$

Each RHS component RHS_j is carefully constructed using finite-difference approximations involving the values u_i . The final discrete second derivative approximation is then assembled by summing these intermediate solutions:

$$u''_i = u''^1_i + u''^2_i + u''^3_i.$$

Our goal is to design the RHS terms so that the combined discrete operator approximates the continuous (1D) Laplacian operator—whose Fourier symbol is $-k^2$ —with high-order accuracy and enhanced spectral properties, particularly at high frequencies.

Thus, extending compact finite-difference schemes to multi-dimensional and multi-directional contexts, while increasing complexity, significantly enhances their versatility and potential applicability across a wide range of scientific and engineering problems.

5.3.1 Fourier Analysis

Consider a Fourier mode $u_i = e^{ikx_i}$ with $x_i = i\Delta x$. Suppose that

$$u_i'' = D_j(k) e^{ikx_i}.$$

Substitute into the modified stencil:

$$\begin{aligned} -u_{i-1}'' + 2u_i'' - u_{i+1}'' &= -D_j(k)e^{ik(i-1)\Delta x} + 2D_j(k)e^{ik i\Delta x} - D_j(k)e^{ik(i+1)\Delta x} \\ &= D_j(k)e^{ik i\Delta x} \left[-e^{-ik\Delta x} + 2 - e^{ik\Delta x} \right]. \end{aligned}$$

Noting that

$$-e^{-ik\Delta x} + 2 - e^{ik\Delta x} = 2 - (e^{ik\Delta x} + e^{-ik\Delta x}) = 2[1 - \cos(k\Delta x)],$$

we have

$$-u_{i-1}'' + 2u_i'' - u_{i+1}'' = 2D_j(k)[1 - \cos(k\Delta x)]e^{ikx_i}.$$

If we express $\text{RHS}_j = R_j(k)e^{ikx_i}$, then the equation implies

$$2D_j(k)[1 - \cos(k\Delta x)] = R_j(k)$$

or equivalently

$$D_j(k) = \frac{R_j(k)}{2[1 - \cos(k\Delta x)]}.$$

Thus, the total Fourier symbol of the approximation is

$$D_{\text{total}}(k) = \sum_{j=1}^3 D_j(k) = \frac{1}{2[1 - \cos(k\Delta x)]} \sum_{j=1}^3 R_j(k).$$

For the scheme to approximate the continuous second derivative, we require

$$D_{\text{total}}(k) \approx -k^2,$$

so that

$$\sum_{j=1}^3 R_j(k) \approx -2[1 - \cos(k\Delta x)] k^2.$$

Let $\theta = k\Delta x$. Since

$$1 - \cos \theta = \frac{\theta^2}{2} - \frac{\theta^4}{24} + \frac{\theta^6}{720} + O(\theta^8),$$

we have

$$2[1 - \cos \theta] = \theta^2 - \frac{\theta^4}{12} + \frac{\theta^6}{360} + O(\theta^8).$$

Thus, the target condition becomes

$$\sum_{j=1}^3 R_j(k) \approx -\frac{1}{\Delta x^2} \left(\theta^4 - \frac{\theta^6}{12} + \frac{\theta^8}{360} + \cdots \right).$$

Note that the leading order term here is $O(\theta^4)$. This is because the new operator in the left-hand side annihilates any quadratic component, and the approximation of the second derivative is recovered only after division by $2[1 - \cos \theta] \sim \theta^2$ for small θ .

5.3.2 Designing the Right-Hand Sides

One may parameterize the $R_j(k)$ in terms of basis functions corresponding to stencils of various widths. For instance, one might write

$$\begin{aligned} \text{RHS}_1 &= a \frac{u_{i+1} - 2u_i + u_{i-1}}{\Delta x^2}, \\ \text{RHS}_2 &= b \frac{u_{i+2} - 2u_i + u_{i-2}}{4\Delta x^2}, \\ \text{RHS}_3 &= c \frac{u_{i+3} - 2u_i + u_{i-3}}{9\Delta x^2}. \end{aligned}$$

This, in general, can be expressed as follows:

$$R_j(k) = \sum_m a_{j,m} f_m(\theta), \quad \text{with} \quad f_m(\theta) = \frac{2[\cos(m\theta) - 1]}{(m\Delta x)^2}.$$

Each RHS proposed above represents 2nd order finite difference approximations at a different. This parametrization can be thought of as a *numerical multigrid*. Choosing the stencils corresponding to $m = 1, 2, 3$, their Fourier symbols are computed as:

$$\begin{aligned} R_1(k) &= a \frac{e^{i\theta} - 2 + e^{-i\theta}}{\Delta x^2} = a \frac{2[\cos \theta - 1]}{\Delta x^2}, \\ R_2(k) &= b \frac{e^{i2\theta} - 2 + e^{-i2\theta}}{4\Delta x^2} = b \frac{\cos(2\theta) - 1}{2\Delta x^2}, \\ R_3(k) &= c \frac{e^{i3\theta} - 2 + e^{-i3\theta}}{9\Delta x^2} = c \frac{2[\cos(3\theta) - 1]}{9\Delta x^2}. \end{aligned}$$

A Taylor expansion for small θ gives:

$$\begin{aligned} R_1(k) &= a \left(-\frac{\theta^2}{\Delta x^2} + \frac{\theta^4}{12\Delta x^2} - \frac{\theta^6}{360\Delta x^2} + O(\theta^8) \right), \\ R_2(k) &= b \left(-\frac{\theta^2}{\Delta x^2} + \frac{\theta^4}{3\Delta x^2} - \frac{2\theta^6}{45\Delta x^2} + O(\theta^8) \right), \\ R_3(k) &= c \left(-\frac{\theta^2}{\Delta x^2} + \frac{3\theta^4}{4\Delta x^2} - \frac{9\theta^6}{40\Delta x^2} + O(\theta^8) \right). \end{aligned}$$

Summing these yields:

$$\sum_{j=1}^3 R_j(k) = -\frac{\theta^2}{\Delta x^2}(a+b+c) + \frac{\theta^4}{\Delta x^2} \left(\frac{a}{12} + \frac{b}{3} + \frac{3c}{4} \right) - \frac{\theta^6}{\Delta x^2} \left(\frac{a}{360} + \frac{2b}{45} + \frac{9c}{40} \right) + O(\theta^8).$$

For the overall operator to approximate $-k^2 = -\theta^2/\Delta x^2$ after division by $2[1 - \cos \theta] \sim \theta^2$, we require

$$\frac{1}{2[1 - \cos \theta]} \sum_{j=1}^3 R_j(k) \approx -\frac{\theta^2}{\Delta x^2}.$$

This leads to the target series

$$\sum_{j=1}^3 R_j(k) \approx -\frac{2[1 - \cos \theta]\theta^2}{\Delta x^2} = -\frac{1}{\Delta x^2} \left(\theta^4 - \frac{\theta^6}{12} + \frac{\theta^8}{360} + \dots \right).$$

Matching coefficients term by term then gives:

$$\begin{aligned} \theta^2 : \quad & -(a+b+c) = 0 \quad \implies \quad a+b+c = 0, \\ \theta^4 : \quad & \frac{a}{12} + \frac{b}{3} + \frac{3c}{4} = -1, \\ \theta^6 : \quad & -\left(\frac{a}{360} + \frac{2b}{45} + \frac{9c}{40} \right) = \frac{1}{12} \quad \implies \quad \frac{a}{360} + \frac{2b}{45} + \frac{9c}{40} = -\frac{1}{12}. \end{aligned}$$

Multiplying the second equation by 12 and the third by 360 yields:

$$\begin{aligned} a + 4b + 9c &= -12, \\ a + 16b + 81c &= -30. \end{aligned}$$

Subtracting the first from the second and using $a + b + c = 0$ (so that $a = -b - c$) gives:

$$(a + 16b + 81c) - (a + 4b + 9c) = 12b + 72c = -18,$$

$$\text{and } (a + 4b + 9c) - (a + b + c) = 3b + 8c = -12.$$

Solving these two equations yields

$$c = \frac{3}{4}, \quad b = -6, \quad \text{and} \quad a = -b - c = \frac{21}{4}.$$

Thus, the right-hand sides are defined by:

$$\begin{aligned} \text{RHS}_1 &= \frac{21}{4} \frac{u_{i+1} - 2u_i + u_{i-1}}{\Delta x^2}, \\ \text{RHS}_2 &= -6 \frac{u_{i+2} - 2u_i + u_{i-2}}{4\Delta x^2}, \\ \text{RHS}_3 &= \frac{3}{4} \frac{u_{i+3} - 2u_i + u_{i-3}}{9\Delta x^2}. \end{aligned}$$

5.3.3 The Algorithm

The steps of this algorithm clearly illustrate the central idea behind the proposed methodology:

1. First, calculate each RHS component RHS_j individually, using carefully chosen finite-difference stencils designed to approximate specific frequency content and accuracy requirements.
2. Solve the corresponding simplified linear equations independently, leveraging efficient FFT-based algorithms or optimized solvers for Poisson-type equations.
3. Finally, recombine the solutions $u_i''^j$ obtained from these simpler equations to form the complete approximation of the second derivative:

$$u_i'' = u_i''^1 + u_i''^2 + u_i''^3.$$

By employing this splitting and recombination strategy, the computational complexity is reduced significantly, enabling high-order accurate approximations while maintaining computational efficiency and advantageous spectral properties. This approach effectively balances accuracy, efficiency, and numerical stability.

Implications on the VPM method Fundamentally this is a method worth exploring in particular to tackle spectral deficiencies of the finite different methods as those presented in the previous chapter regarding the vorticity correction schemes.

In particular in order to solve the vorticity correction we have shown mathematically

that the correction scheme discussed in the previous chapter is insufficient due to $\sigma_{\Delta_d}(\boldsymbol{\xi}) \neq \sigma_{\nabla_d \cdot \nabla_d}(\boldsymbol{\xi})$. This result can be interpreted from the error characteristics presented in 5.1. The approximation error of the classical finite-difference schemes acts as a form of relaxation in the high-frequency spectrum that imposes an iterative process.

The compact finite-difference formulation presented in this chapter is well suited to correct this mismatch, and it can be further specialized or optimized if needed. Additionally, since VIC methods introduce complex spectral characteristics due to the projection/interpolation operators, it would be interesting to test the performance of this formulation in solving the Poisson problems ψ and q (pressure formulation), where the spectral fidelity of the approximation is of particular importance.

Chapter 6

Validation and Verification

6.1 Validation of Diffusion Schemes

In this section, we construct a non-trivial analytical solution for the vector diffusion problem to rigorously benchmark the diffusive step of the vortex particle method (VPM) algorithm. Our strategy is to build a vector solution derived from the three-dimensional Ricker (Mexican hat) wavelet. Although the Ricker wavelet is originally a scalar function, its intrinsic properties allow us to extend it to a vectorial framework by considering its gradient.

The Ricker wavelet is the second Hermite function and, as such, forms an orthogonal basis. This orthogonality implies that any sufficiently smooth function can, in principle, be decomposed into a linear combination of Ricker wavelets. Moreover, due to the linearity of the diffusion equation, fundamental solutions based on the Ricker wavelet can be superimposed to analytically examine their mutual interactions. This natural extension would allow for a spectral analysis, exploring the coupling of various scales in a controlled manner to benchmark numerical diffusion solvers at different resolutions. However, in the present work, we restrict our study to a single, representative case, leaving a more comprehensive spectral investigation for future work.

Specifically, the vorticity diffusion equation in the viscous splitting framework of the VPM algorithm is given by

$$\frac{\partial \boldsymbol{\omega}}{\partial t} = \nu \nabla^2 \boldsymbol{\omega}, \quad (6.1)$$

where ν denotes the diffusivity. Our objective is to verify that the numerical solver can accurately resolve this equation by comparing its output against our derived analytical solution.

6.1.1 Ricker Wavelet (Mexican Hat)

Let us consider the normalized three-dimensional Gaussian kernel defined by

$$G(\mathbf{r}) = \frac{1}{(2\pi\sigma^2)^{3/2}} \exp\left(-\frac{r^2}{2\sigma^2}\right), \quad r = \|\mathbf{r}\|, \quad (6.2)$$

which naturally extends the one-dimensional Gaussian (with zero mean) to $\mathbb{R}^3$.

The three-dimensional Ricker wavelet is defined as the negative Laplacian of the Gaussian, multiplied by σ^2 , i.e.,

$$\psi(\mathbf{r}) = -\sigma^2 \nabla^2 G(r). \quad (6.3)$$

For any radially symmetric function $G(r)$, the Laplacian in spherical coordinates is given by

$$\nabla^2 G(r) = G''(r) + \frac{2}{r} G'(r), \quad (6.4)$$

where the prime denotes differentiation with respect to r .

We thus begin by differentiating Eq. (6.2) with respect to r , obtaining:

$$\begin{aligned} G'(r) &= \frac{dG(r)}{dr} = \frac{d}{dr} \left[\frac{1}{(2\pi\sigma^2)^{3/2}} \exp\left(-\frac{r^2}{2\sigma^2}\right) \right] \\ &= \frac{1}{(2\pi\sigma^2)^{3/2}} \cdot \exp\left(-\frac{r^2}{2\sigma^2}\right) \cdot \left(-\frac{r}{\sigma^2}\right) \\ &= -\frac{r}{\sigma^2} G(r). \end{aligned} \quad (6.5)$$

We observe that the derivative of a Gaussian is also a Gaussian in itself. Differentiating again, we get:

$$\begin{aligned} G''(r) &= \frac{d}{dr} \left(-\frac{r}{\sigma^2} G(r) \right) \\ &= -\frac{1}{\sigma^2} G(r) + \frac{r^2}{\sigma^4} G(r) \\ &= \left(\frac{r^2}{\sigma^4} - \frac{1}{\sigma^2} \right) G(r). \end{aligned} \quad (6.6)$$

Substituting Eqs. (6.5) and (6.6) into Eq. (6.4) we derive the formula for the Laplacian of a Gaussian:

$$\begin{aligned} \nabla^2 G(r) &= \left(\frac{r^2}{\sigma^4} - \frac{1}{\sigma^2} \right) G(r) + \frac{2}{r} \left(-\frac{r}{\sigma^2} G(r) \right) \\ &= \left(\frac{r^2}{\sigma^4} - \frac{1}{\sigma^2} - \frac{2}{\sigma^2} \right) G(r) \\ &= \left(\frac{r^2}{\sigma^4} - \frac{3}{\sigma^2} \right) G(r). \end{aligned} \quad (6.7)$$

The Ricker wavelet is thus expressed as:

$$\begin{aligned}
 \psi(\mathbf{r}) &= -\sigma^2 \left(\frac{r^2}{\sigma^4} - \frac{3}{\sigma^2} \right) G(r) \\
 &= \left(3 - \frac{r^2}{\sigma^2} \right) G(r) \\
 &= \left(3 - \frac{r^2}{\sigma^2} \right) \frac{1}{(2\pi\sigma^2)^{3/2}} \exp\left(-\frac{r^2}{2\sigma^2}\right).
 \end{aligned} \tag{6.8}$$

6.1.2 Diffusion of the Ricker Wavelet in Time

Now, let's consider the diffusion (or heat) equation in three dimensions:

$$\frac{\partial \phi(\mathbf{r}, t)}{\partial t} = \nu \nabla^2 \phi(\mathbf{r}, t), \tag{6.9}$$

where ν is the diffusivity coefficient. The formal solution with the initial condition $\phi(\mathbf{r}, 0)$ is given by the convolution with the heat kernel:

$$\phi(\mathbf{r}, t) = \int_{\mathbb{R}^3} \phi(\mathbf{r}', 0) K(\mathbf{r} - \mathbf{r}', t) d\mathbf{r}', \tag{6.10}$$

,with the heat kernel in three dimensions expressed as

$$K(\mathbf{r}, t) = \frac{1}{(4\pi\nu t)^{3/2}} \exp\left(-\frac{r^2}{4\nu t}\right). \tag{6.11}$$

In our application, the initial condition is the Ricker wavelet from Eq. (6.8), i.e.,

$$\psi(\mathbf{r}, 0) = \left(3 - \frac{r^2}{\sigma^2} \right) \frac{1}{(2\pi\sigma^2)^{3/2}} \exp\left(-\frac{r^2}{2\sigma^2}\right).$$

A direct convolution of this initial condition with the heat kernel is algebraically intensive due to the polynomial factor. In order to simplify the analysis, consider that the diffusion equation is linear and the Laplacian commutes with the time evolution. Hence, we first consider the evolution of the Gaussian:

$$G_r(\mathbf{r}, 0) = \frac{1}{(2\pi\sigma^2)^{3/2}} \exp\left(-\frac{r^2}{2\sigma^2}\right). \tag{6.12}$$

The solution of Eq. (6.9) with this initial condition is given by

$$G_r(\mathbf{r}, t) = \frac{1}{[2\pi(\sigma^2 + 2\nu t)]^{3/2}} \exp\left(-\frac{r^2}{2(\sigma^2 + 2\nu t)}\right). \tag{6.13}$$

This result follows from the convolution of two Gaussians, noting that the variance of the heat kernel is $2\nu t$.

Since the Laplacian operator commutes with the diffusion process, we can write the time evolution of the Ricker wavelet as

$$\psi(\mathbf{r}, t) = -\sigma^2 \nabla^2 G_r(\mathbf{r}, t). \quad (6.14)$$

For the diffused Gaussian given by Eq. (6.13), we compute its Laplacian in spherical coordinates. Differentiating with respect to r , we have

$$\frac{\partial G_r(r, t)}{\partial r} = G_r(r, t) \left(-\frac{r}{\sigma^2 + 2\nu t} \right), \quad (6.15)$$

$$\begin{aligned} \frac{\partial^2 G_r(r, t)}{\partial r^2} &= \frac{\partial}{\partial r} \left[-\frac{r}{\sigma^2 + 2\nu t} G_r(r, t) \right] \\ &= -\frac{1}{\sigma^2 + 2\nu t} G_r(r, t) + \frac{r^2}{(\sigma^2 + 2\nu t)^2} G_r(r, t). \end{aligned} \quad (6.16)$$

Thus, the Laplacian of $G(\mathbf{r}, t)$ is

$$\begin{aligned} \nabla^2 G_r(r, t) &= \frac{\partial^2 G_r(r, t)}{\partial r^2} + \frac{2}{r} \frac{\partial G_r(r, t)}{\partial r} \\ &= \left[\frac{r^2}{(\sigma^2 + 2\nu t)^2} - \frac{1}{\sigma^2 + 2\nu t} \right] G_r(r, t) - \frac{2}{\sigma^2 + 2\nu t} G_r(r, t) \\ &= \left[\frac{r^2}{(\sigma^2 + 2\nu t)^2} - \frac{3}{\sigma^2 + 2\nu t} \right] G_r(r, t). \end{aligned} \quad (6.17)$$

Substituting Eq. (6.17) into Eq. (6.14) gives

$$\begin{aligned} \psi(\mathbf{r}, t) &= -\sigma^2 \nabla^2 G_r(r, t) \\ &= -\sigma^2 \left[\frac{r^2}{(\sigma^2 + 2\nu t)^2} - \frac{3}{\sigma^2 + 2\nu t} \right] G_r(r, t) \\ &= \sigma^2 \left[\frac{3}{\sigma^2 + 2\nu t} - \frac{r^2}{(\sigma^2 + 2\nu t)^2} \right] G_r(r, t). \end{aligned} \quad (6.18)$$

Finally, substituting the explicit form of $G_r(r, t)$ from Eq. (6.13) into Eq. (6.18), we arrive at the analytical expression for the time-evolved Ricker wavelet:

$$\psi(\mathbf{r}, t) = \sigma^2 \left[\frac{3}{\sigma^2 + 2\nu t} - \frac{r^2}{(\sigma^2 + 2\nu t)^2} \right] \frac{1}{[2\pi(\sigma^2 + 2\nu t)]^{3/2}} \exp\left(-\frac{r^2}{2(\sigma^2 + 2\nu t)}\right). \quad (6.19)$$

In summary, we have derived the three-dimensional Ricker wavelet as a modified

Gaussian (via the negative Laplacian operation) in Eq. (6.8). Moreover, by exploiting the commutativity of the Laplacian with the diffusion operator, we have obtained an explicit analytical expression for the time evolution of the Ricker wavelet under diffusion (see Eq. (6.19)).

Extension to a Vector Field

To extend the scalar Ricker wavelet to a vector field suitable for vectorial diffusion problems, we have two natural approaches. One may either employ a linear operator—such as the gradient—to generate the vector field, or alternatively, define the vector field by modulating the scalar function with a prescribed vector function. In the present study, we adopt the first method.

Specifically, we define the vector field by:

$$\mathbf{\Psi}(\mathbf{r}, t) = \nabla \psi(\mathbf{r}, t), \quad (6.20)$$

where $\psi(\mathbf{r}, t)$ is given by Eq. (6.19).

In spherical coordinates, for a radially symmetric scalar field $\psi(r, t)$, the gradient simplifies to

$$\nabla \psi(r, t) = \frac{d\psi(r, t)}{dr} \hat{\mathbf{r}}, \quad (6.21)$$

with $\hat{\mathbf{r}} = \mathbf{r}/r$ being the radial unit vector. Hence, the vector field is purely radial:

$$\mathbf{\Psi}(\mathbf{r}, t) = \psi_r(r, t) \hat{\mathbf{r}}, \quad (6.22)$$

where

$$\psi_r(r, t) = \frac{d\psi(r, t)}{dr}. \quad (6.23)$$

This construction ensures that the divergence of the vector field is directly related to the Laplacian of ψ , preserving the diffusion dynamics. Moreover, since the Laplacian and the gradient commute with the diffusion operator in linear problems, the time evolution of $\mathbf{\Psi}(\mathbf{r}, t)$ can be analyzed by first evolving $\psi(\mathbf{r}, t)$ via Eq. (6.19) and then computing its gradient.

In appendix B we show that the time evolution of $\mathbf{\Psi}$ under diffusion is:

$$\mathbf{\Psi}(\mathbf{r}, t) = \nabla \psi(\mathbf{r}, t) = -\sigma^2 \frac{r}{\tau^2} \left(5 - \frac{r^2}{\tau} \right) G(r, t) \hat{\mathbf{r}}, \quad (6.24)$$

with

$$G(r, t) = \frac{1}{[2\pi\tau]^{3/2}} \exp\left(-\frac{r^2}{2\tau}\right) \quad \text{and} \quad \tau = \sigma^2 + 2\nu t.$$

In order to thoroughly assess the performance of a diffusion solver, we extend our validation framework by tracking the time evolution of key wavelet properties. The

Ricker wavelet, with its analytically derived evolution, provides a rich test case to verify not only the accuracy of numerical diffusion but also the conservation and symmetry properties intrinsic to the underlying physics.

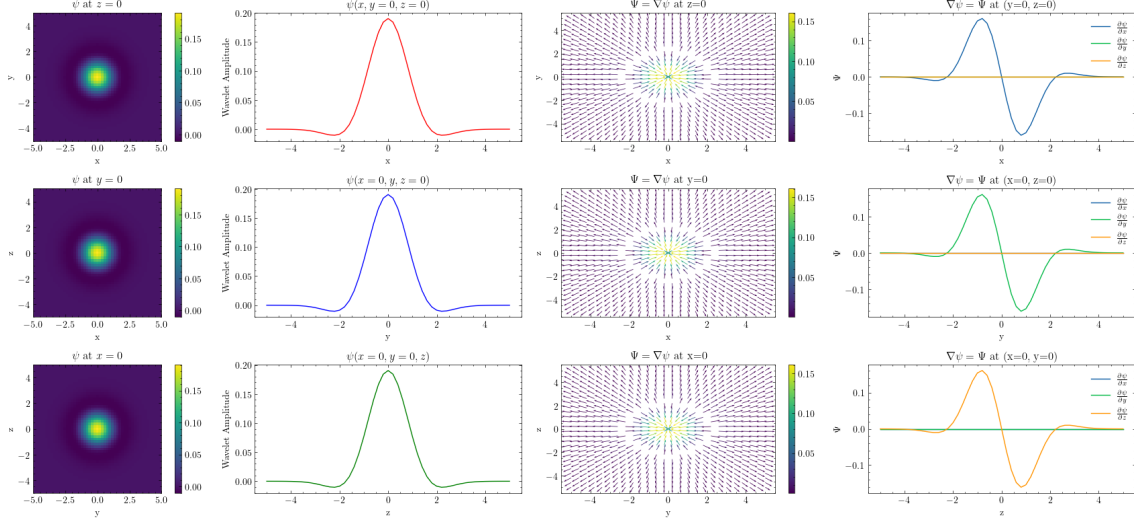


Figure 6.1: Visualization of Ricker's wavelet ψ and the manufactured field $\Psi = \nabla\psi$ at $t = T_0$. **First column:** A two-dimensional slice of the scalar wavelet ψ , color-coded to highlight its amplitude distribution. **Second column:** A one-dimensional cross-section of ψ along the one axis (i.e., $z = 0$, $x = 0$, $y = 0$). **Third column:** The vector field $\Psi = \nabla\psi$, visualized with arrows denoting the local direction and color indicating the magnitude. **Fourth column:** Line plots of the individual components of Ψ (e.g., Ψ_x , Ψ_y , Ψ_z) along on axis.

As demonstrated in 6.1 ψ is initially localized and radially symmetric, as expected from the analytic form of the Ricker wavelet, while the gradient field Ψ captures its spatial derivatives.

Energy Decay. In Appendix B.2 we derive that if we define the vector field energy as:

$$E_{\Psi}(t) = \int_{\mathbb{R}^3} |\Psi(\mathbf{r}, t)|^2 dV. \quad (6.25)$$

Then that energy will naturally decay through the diffusion obtaining:

$$E_{\Psi}(t) = \frac{105 \sigma^4}{64 \pi^{3/2} (\sigma^2 + 2\nu t)^{9/2}}. \quad (6.26)$$

This property is equivalent to the enstrophy of a flow field and thus quantifies the rotational-energy metric, offering a measure of the dissipation in the system.

Definition of Enstrophy. In fluid dynamics, *enstrophy* is a measure of the intensity of rotational motion. For a velocity field $\mathbf{u}(\mathbf{r}, t)$ with vorticity $\boldsymbol{\omega}(\mathbf{r}, t) =$

$\nabla \times \mathbf{u}(\mathbf{r}, t)$, the enstrophy is given by

$$\mathcal{E}(t) = \int_{\mathbb{R}^3} |\boldsymbol{\omega}(\mathbf{r}, t)|^2 dV. \quad (6.27)$$

, which coincides with the energy definition in Equation (6.26).

Incompressible Flow Perspective. In the case of an incompressible flow ($\nabla \cdot \mathbf{u} = 0$), the enstrophy $\mathcal{E}(t)$ can be described as the integral of the squared vorticity:

$$\mathcal{E}(\boldsymbol{\omega}) = \int_{\Omega} |\boldsymbol{\omega}|^2 dV \quad \text{or, equivalently,} \quad \mathcal{E}(\mathbf{u}) = \int_{\Omega} |\nabla \times \mathbf{u}|^2 dV.$$

Enstrophy plays a key role in quantifying how kinetic energy decays due to viscosity. A well-known result is

$$\frac{d}{dt} \left(\frac{1}{2} \int_{\Omega} |\mathbf{u}|^2 dV \right) = -\nu \int_{\Omega} |\nabla \times \mathbf{u}|^2 dV,$$

, where the left-hand side represents the rate of change of kinetic energy in the flow and the right-hand side shows that energy is dissipated at a rate proportional to the kinematic viscosity ν times the enstrophy. Thus, enstrophy not only serves as a measure of rotational intensity but also governs the rate at which kinetic energy decays in incompressible flows.

Synthesis and Application. Collectively, the energy decay formula and field solution provide a comprehensive framework for evaluating the performance of diffusion solvers. By tracking the time evolution of these properties in the numerical solution and comparing them with the analytical results derived from the Ricker wavelet, one gains deep insight into the accuracy, stability, and fidelity of the solver.

6.1.3 Numerical Results

Computational Implementation

- **Grid Resolution:** $\Delta x \leq \sigma/3$ to adequately resolve the core of the wavelet.
- **Domain Size:** Domain size $L \geq 5\sigma$ to capture approximately 99.9% of the wavelet energy.
- **Time Stepping:** $\Delta t \leq \Delta x^2/(6\nu)$ to maintain stability.

Solver Comparisons. We now verify the performance of two numerical solvers applied within the VPM framework to solve the diffusion equation (6.1) with the initial condition $\boldsymbol{\omega}(\mathbf{r}, 0) = \boldsymbol{\Psi}(\mathbf{r}, 0)$. Specifically, we employ:

- A multigrid solver based on *MUDPACK*.

- A domain-decomposition approach implemented in *YAPS* (using the *FISHPACK* library).

Both methods aim to solve the Poisson-type subproblems that arise in the viscous splitting step of the VPM. Our goal is to confirm that each solver accurately captures the analytical time evolution of the Ricker wavelet’s vorticity field.

Energy Decay. We track the total “energy” (or equivalently, enstrophy for a vorticity field) of the wavelet over time, defined by

$$E_{\Psi}(t) = \int_{\mathbb{R}^3} |\Psi(\mathbf{r}, t)|^2 dV.$$

Figure 6.2 compares the analytical evolution of $E_{\Psi}(t)$ (6.26) to the numerical solutions obtained by both MUDPACK and YAPS. The simulated decay closely matches the analytical curve for each method, indicating that the diffusive dynamics are well resolved.

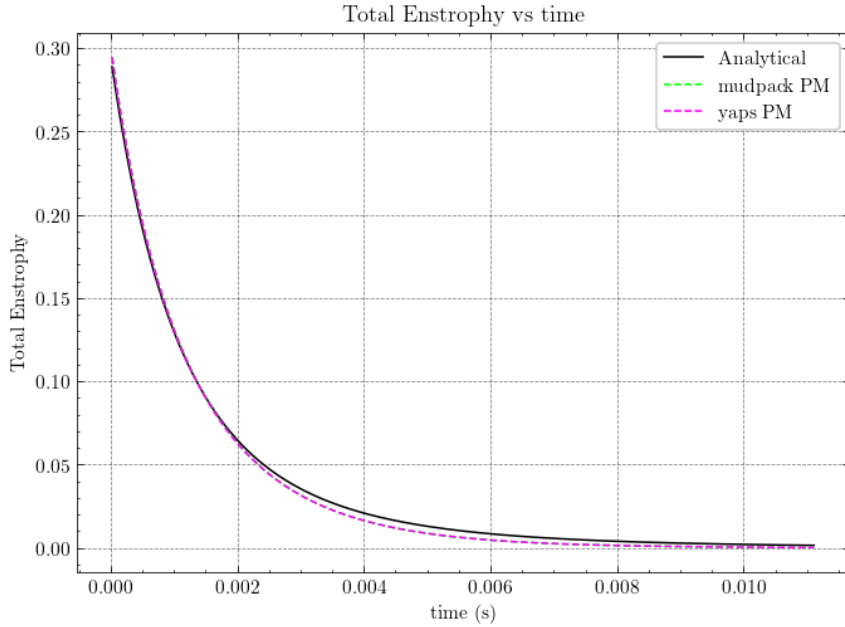


Figure 6.2: Comparison of total enstrophy (energy) $E_{\Psi}(t)$ between the analytical solution (6.26) and the numerical results from MUDPACK and YAPS. Both methods exhibit good agreement with the theoretical decay over the simulated time interval.

To scrutinize finer details, Figure 6.3 presents the same data in a logarithmic scale. This view reveals that the VPM-based solutions capture diffusion in an essentially *exponential* fashion, though there is a small discrepancy in the tail region where higher-order (nonlinear) effects manifest.

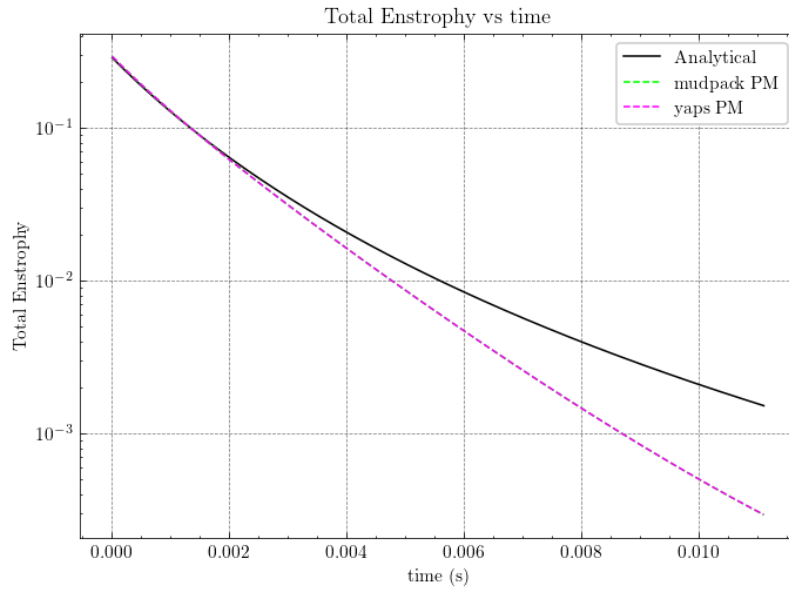


Figure 6.3: Log-scale plot of $E_\Psi(t)$ vs. time, highlighting subtle deviations in the long-time tail. The VPM solutions (MUDPACK and YAPS) remain very close to the analytical curve, though minor departures may indicate missed nonlinear effects due to the time-integration strategy.

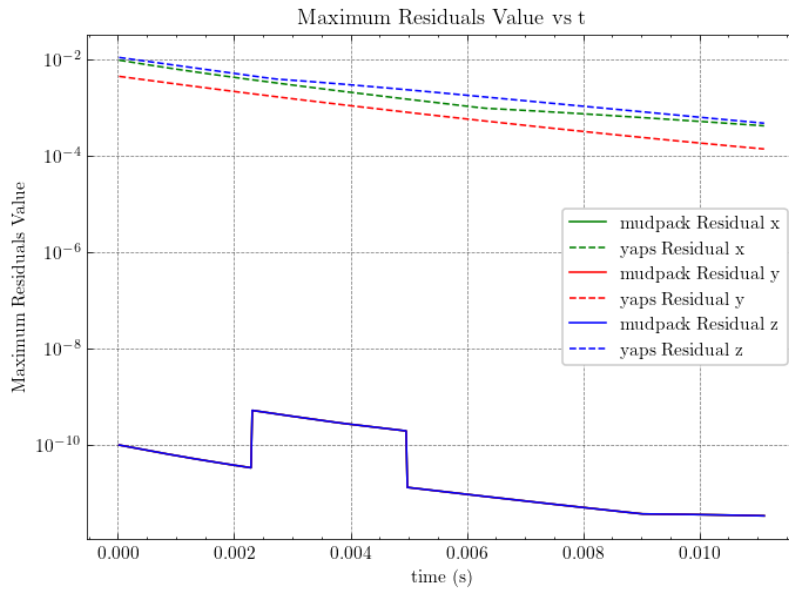


Figure 6.4: Maximum solver residual at each time step and spatial component for MUDPACK and YAPS. The multigrid method (MUDPACK) achieves lower residuals, whereas YAPS displays higher but still diminishing residuals over time.

Solver Residuals. A crucial check of the numerical solution is the internal consistency of each Poisson-solve step. In Figure 6.4, we plot the maximum residual over

all grid points for each solver at each time step. We observe that MUDPACK consistently attains significantly lower residuals than YAPS, reflecting the robustness of the multigrid approach for this problem. Nevertheless, both methods converge to acceptable levels for practical applications.

In summary, by comparing both energy decay (enstrophy) and solver residuals, we confirm that the VPM adequately captures the diffusion of the Ricker wavelet vorticity field. MUDPACK exhibits slightly superior convergence properties in terms of residuals, whereas YAPS remains competitive for practical purposes. Although the energy-decay curves closely match the analytical solution in absolute terms, it is crucial to investigate why the VPM exhibits a linear behavior in the logarithmic plot. This indicates that some nonlinear effects may not be fully captured by the current integration scheme. These findings nonetheless support the reliability of the viscous splitting approach in the VPM framework and provide a foundation for further exploration of more complex, multi-wavelet scenarios in future work.

6.2 Validation of the VPM algorithm

In this chapter, we present a rigorous validation of the vortex particle method (VPM) algorithm. Our objective is to assess the accuracy and robustness of the VPM in resolving complex fluid dynamics problems involving coupled convection and diffusion. To this end, we utilize well-established analytical solutions as benchmarks, thereby enabling a quantitative comparison between the numerical and theoretical predictions.

A central component of our validation strategy is the use of Hill’s spherical vortex, an exact solution of the inviscid, incompressible, and axisymmetric Euler equations. Hill’s vortex, with its clearly defined interior region (where vorticity is nonzero) and exterior region (which is irrotational), provides a stringent test case for verifying the fidelity of the VPM algorithm. By expressing the velocity field in terms of a Stokes streamfunction and deriving the corresponding vorticity field, we establish an analytical framework that captures the essential features of the vortex dynamics.

In our simulations, key integral quantities such as total momentum, kinetic energy, enstrophy, and the maximum divergence of the vorticity field are meticulously monitored. These metrics serve not only as indicators of the numerical accuracy but also as benchmarks for assessing the conservation properties and the physical fidelity of the algorithm. Furthermore, by varying simulation parameters—such as the Reynolds number and the remeshing frequency—we are able to systematically investigate the sensitivity and robustness of the VPM.

6.2.1 Hill's Vortex

Hill's spherical vortex is an exact solution of the inviscid, incompressible, axisymmetric Euler equations. In spherical coordinates

$$(r, \theta, \phi), \quad r = \sqrt{x^2 + y^2 + z^2}, \quad \theta \in [0, \pi], \quad \phi \in [0, 2\pi),$$

the velocity field for a swirl-free flow may be written in terms of a Stokes streamfunction $\Psi(r, \theta)$ as

$$u_r = \frac{1}{r^2 \sin \theta} \frac{\partial \Psi}{\partial \theta}, \quad u_\theta = -\frac{1}{r \sin \theta} \frac{\partial \Psi}{\partial r}, \quad u_\phi = 0.$$

We consider the flow in two regions: the interior ($r < a$), where the vorticity is nonzero, and the exterior ($r > a$), where the flow is irrotational.

The Interior Flow ($r < a$). is given by the streamfunction:

$$\Psi_{\text{in}}(r, \theta) = \frac{3U}{4} \left(r^2 - \frac{r^4}{a^2} \right) \sin^2 \theta,$$

where U is a characteristic velocity. Differentiation yields

$$\frac{\partial \Psi_{\text{in}}}{\partial \theta} = \frac{3U}{2} \left(r^2 - \frac{r^4}{a^2} \right) \sin \theta \cos \theta,$$

so that

$$u_r = \frac{1}{r^2 \sin \theta} \frac{\partial \Psi_{\text{in}}}{\partial \theta} = \frac{3U}{2} \cos \theta \left(1 - \frac{r^2}{a^2} \right).$$

Similarly,

$$\frac{\partial \Psi_{\text{in}}}{\partial r} = \frac{3U}{4} \sin^2 \theta \left(2r - \frac{4r^3}{a^2} \right),$$

implying

$$u_\theta = -\frac{1}{r \sin \theta} \frac{\partial \Psi_{\text{in}}}{\partial r} = -\frac{3U}{4} \frac{\sin \theta}{r} \left(2r - \frac{4r^3}{a^2} \right) = -\frac{3U}{2} \sin \theta \left(1 - \frac{2r^2}{a^2} \right).$$

The exterior flow ($r > a$) is given by an irrotational field that matches a uniform flow at infinity. Its stream-function is

$$\Psi_{\text{out}}(r, \theta) = -\frac{U a^3}{2r} \sin^2 \theta,$$

which yields $\nabla \times \mathbf{u} = \mathbf{0}$ for $r > a$.

Vorticity Computation. For an axisymmetric swirl-free flow, the only nonzero vorticity component is in the $\hat{\phi}$ direction:

$$\omega_\phi = \frac{1}{r} \frac{\partial}{\partial r} (r u_\theta) - \frac{1}{r} \frac{\partial u_r}{\partial \theta}.$$

Within $r < a$, using the expressions for u_r and u_θ we have

$$r u_\theta = -\frac{3U}{2} r \sin \theta \left(1 - \frac{2r^2}{a^2} \right).$$

Differentiating with respect to r gives

$$\frac{\partial}{\partial r} (r u_\theta) = -\frac{3U}{2} \sin \theta \left[\left(1 - \frac{2r^2}{a^2} \right) - \frac{4r^2}{a^2} \right] = -\frac{3U}{2} \sin \theta \left(1 - \frac{6r^2}{a^2} \right).$$

Thus,

$$\frac{1}{r} \frac{\partial}{\partial r} (r u_\theta) = -\frac{3U \sin \theta}{2r} \left(1 - \frac{6r^2}{a^2} \right).$$

Next, since

$$u_r = \frac{3U}{2} \cos \theta \left(1 - \frac{r^2}{a^2} \right),$$

its θ -derivative is

$$\frac{\partial u_r}{\partial \theta} = -\frac{3U}{2} \sin \theta \left(1 - \frac{r^2}{a^2} \right),$$

so that

$$-\frac{1}{r} \frac{\partial u_r}{\partial \theta} = \frac{3U \sin \theta}{2r} \left(1 - \frac{r^2}{a^2} \right).$$

Combining these results, the azimuthal vorticity is

$$\omega_\phi = \frac{3U \sin \theta}{2r} \left[-\left(1 - \frac{6r^2}{a^2} \right) + \left(1 - \frac{r^2}{a^2} \right) \right] = \frac{3U \sin \theta}{2r} \left(\frac{5r^2}{a^2} \right) = \frac{15U r \sin \theta}{2a^2}.$$

Hence, the vorticity field inside the vortex is

$$\boxed{\boldsymbol{\omega}(r, \theta) = \frac{15U r \sin \theta}{2a^2} \hat{\phi}, \quad r < a.}$$

while for $r > a$ we have $\boldsymbol{\omega}(r, \theta) = \mathbf{0}$.

In summary, Hill's vortex is characterized by the velocity components

$$u_r = \frac{3U}{2} \cos \theta \left(1 - \frac{r^2}{a^2} \right), \quad u_\theta = -\frac{3U}{2} \sin \theta \left(1 - \frac{2r^2}{a^2} \right),$$

and a confined azimuthal vorticity given by

$$\omega_\phi = \frac{15U r \sin \theta}{2a^2}, \quad r < a.$$

To provide a visual context for the evolution of Hill's vortex, we include a snapshot of the flow field at an intermediate time $t = T_0$. This image illustrates the vortex structure as it develops and interacts with the surrounding fluid, serving as a useful reference for the subsequent quantitative analysis.

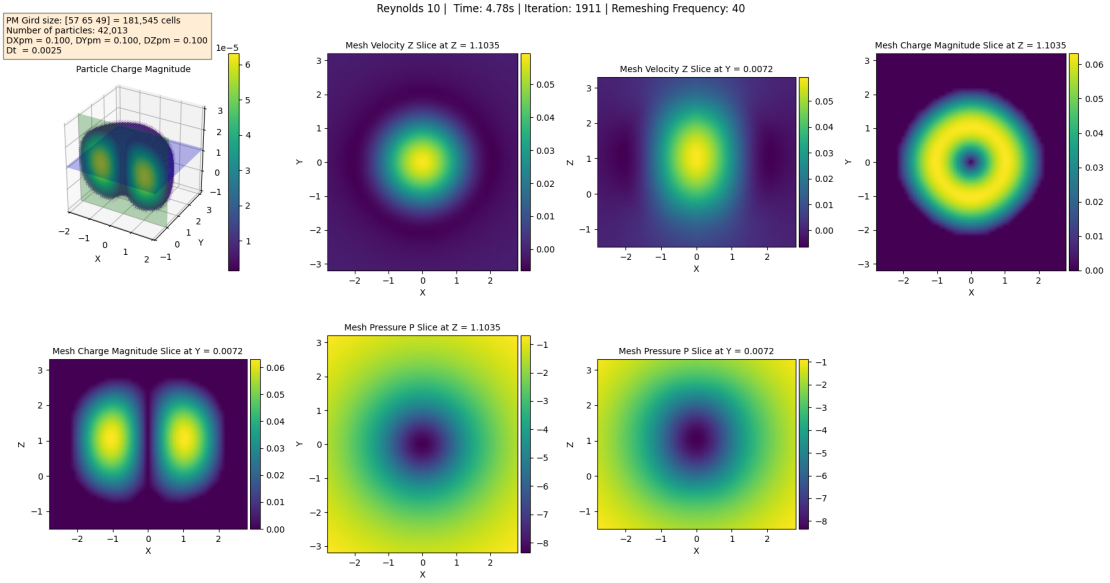


Figure 6.5: Snapshot of Hill's vortex at $t = T_0$, corresponding to $Re = 10$, time $t = 4.78$ s. **Top-left:** A three-dimensional rendering of the particle vorticity magnitude. **Top-center** Two-dimensional slices of the mesh-based velocity magnitude $\|\mathbf{u}\|$. The left slice is horizontal ($z = 1.1035$) while the right one is vertical ($y \approx 0$). **Top-right:** A slice of the same horizontal plane, highlighting the vorticity magnitude. **Bottom-left** A slice of the same vertical plane, highlighting the vorticity magnitude. **Bottom-Center:** Horizontal and Vertical slices of the mesh-based pressure field p .

Generalized Beltrami Flows. A flow is termed a *generalized Beltrami flow* if it satisfies the condition

$$\nabla \times (\mathbf{u} \times \boldsymbol{\omega}) = \mathbf{0}.$$

This requirement ensures that the nonlinear interactions associated with vortex stretching and advection are arranged in a manner that promotes the persistence of coherent vortical structures. Although Hill's vortex does not fulfill the classical pointwise alignment $\boldsymbol{\omega} \parallel \mathbf{u}$ for a Beltrami flow, one can show that it indeed satisfies the generalized Beltrami condition. In this sense, Hill's vortex is naturally embedded

in the broader framework of generalized Beltrami flows, a connection that has been explored in the literature.

6.2.2 Parametric Numerical Study on the Evolution of Hill's Vortex

We investigate the temporal evolution of Hill's vortex over a total simulation time $T = 5s$ using the VPM algorithm.

Simulation Parameters

The computational domain is defined by a sphere of radius:

$$R = 2.0.$$

The dynamic viscosity is prescribed as:

$$\nu = U_\infty \frac{R}{Re},$$

where $U_\infty = 1$ is a characteristic far-field velocity and Re denotes the Reynolds number. The spatial discretization is given by the uniform mesh spacing

$$\Delta x = \Delta y = \Delta z = 0.1,$$

and the time step is set to

$$\Delta t = 0.0025.$$

The fluid density is normalized to

$$\rho = 1.$$

These parameters are chosen to satisfy the stability conditions required by the convection and diffusion schemes.

Parameter Variation

The study is performed over a range of Reynolds numbers,

$$Re \in \{10, 100, 500, 1000, \infty\},$$

to examine the influence of viscous diffusion on vortex evolution. For each Reynolds number, two remeshing strategies are implemented: remeshing is performed every 5 timesteps in one set of simulations and every 40 timesteps in another. The remeshing frequency is critical for controlling particle clustering and numerical diffusion, and its variation allows us to assess the sensitivity and robustness of the particle mesh method.

Summary of Simulation Cases

Case	Re	Remeshing Frequency (timesteps)	Vorticity Correction
1	10	5	Yes
2	10	40	Yes
3	10	5	No
4	10	40	No
5	100	5	No
6	100	40	No
7	500	5	No
8	500	40	No
9	1000	5	No
10	1000	40	No
11	∞	5	Yes
12	∞	No	No
13	∞	1	No
14	∞	5	No
15	∞	40	No
16	∞	100	No

Table 6.1: *Summary of the Primary Simulation Cases.*

The simulation cases summarized in Table 6.1 are designed to investigate the combined effects of viscosity, remeshing frequency, and the application of a vorticity correction procedure on the evolution of Hill’s vortex. In these simulations, the Reynolds number Re is varied over a wide range—from $Re = 10$, representing a regime of high viscosity to $Re = \infty$, corresponding to the inviscid limit—thus allowing us to capture the extreme effects of viscous diffusion.

In addition, the frequency at which the particles are remeshed is varied. Remeshing is performed at different intervals (every 1, 5, 40, or 100 timesteps) to assess its influence on numerical accuracy and the preservation of coherent vortex structures.

Furthermore, the vorticity correction is applied selectively. Specifically, we examine the impact of the vorticity correction only in the extreme cases of high viscosity ($Re = 10$) and in the inviscid limit ($Re = \infty$). This selective application allows us to isolate and analyze the effect of the correction procedure under conditions where viscous effects are either dominant or entirely absent.

6.2.3 Numerical Results

We now present the simulation results for the evolution of Hill’s vortex, computed via the VPM algorithm.

Total Momentum

The total momentum components in the x , y , and z directions are defined as

$$M_i = \int_{\Omega} \rho u_i dV, \quad i \in \{x, y, z\},$$

where $\rho = 1$ and Ω denotes the computational domain. These integrals quantify the conservation of momentum in the system. In particular, if $M_i(t)$ denotes the momentum at time t , any significant deviations from constant values (when appropriate) can indicate numerical dissipation or accumulation of errors. Figures 6.6, 6.7, and 6.8 display the time evolution of M_x , M_y , and M_z , respectively.

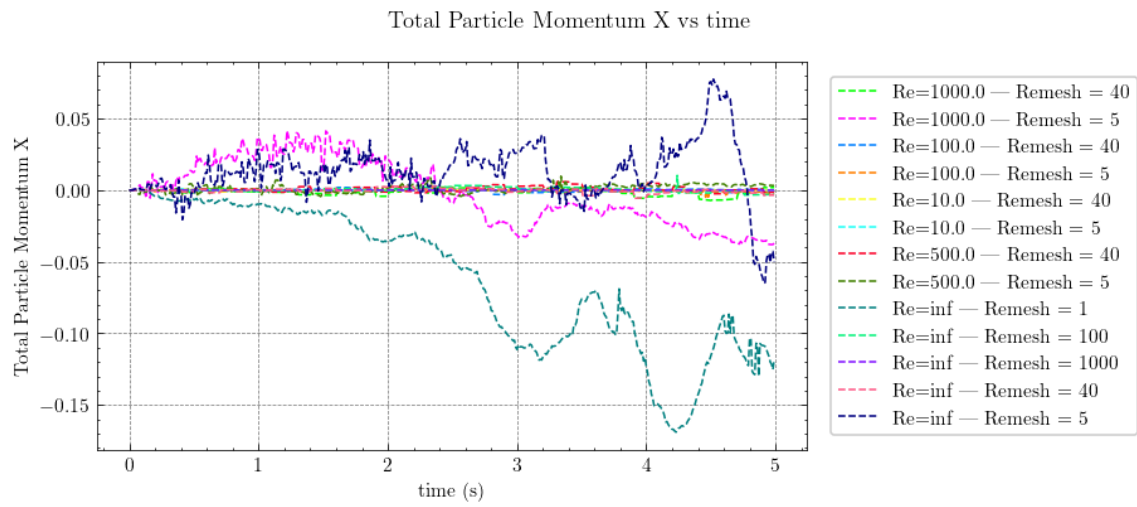


Figure 6.6: Time evolution of the total momentum in the x -direction, M_x .

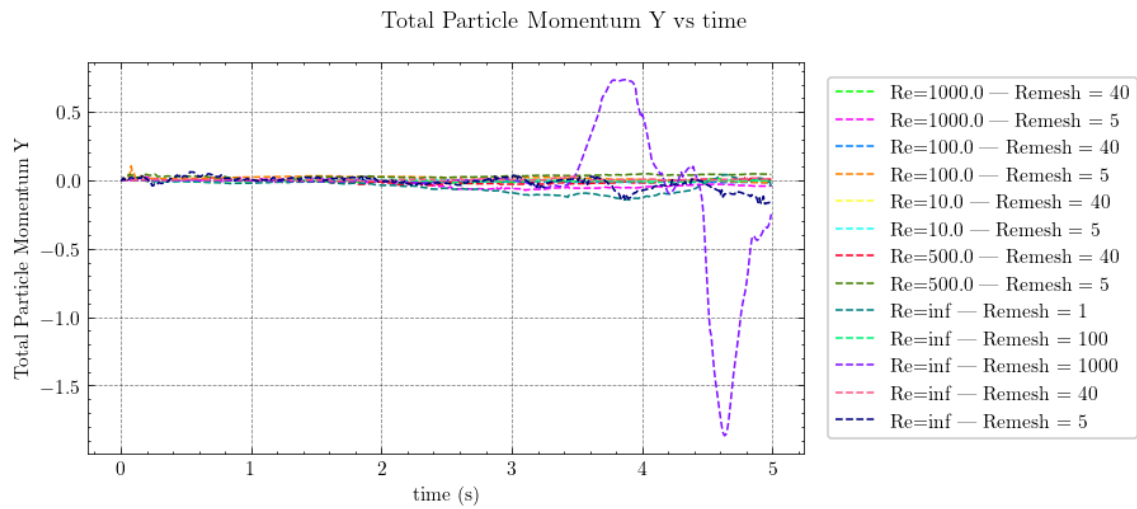


Figure 6.7: Time evolution of the total momentum in the y -direction, M_y .

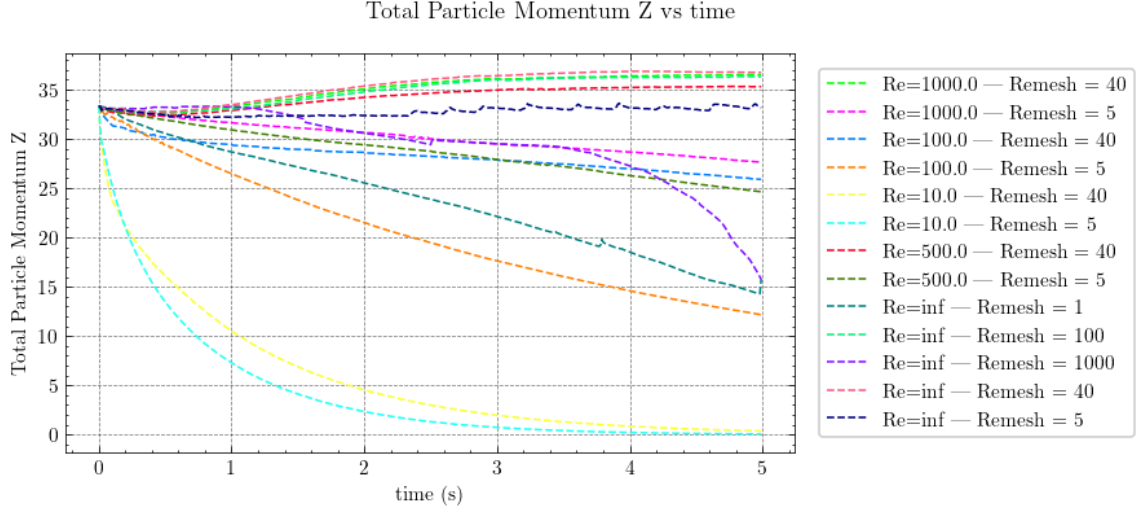


Figure 6.8: Time evolution of the total momentum in the z -direction, M_z .

In Hill's vortex all of the momentum is gathered in the z direction. As expected we see that most setups present a specific decay that is due to diffusion. Additionally, we see that the effect of remeshing is clear. For each Reynold's number, we see that as we increase the remeshing frequency, we introduce artificial diffusion and therefore, the momentum decay-rate increases.

Enstrophy

Enstrophy, defined by $E = \int_{\Omega} \|\omega\|^2 dV$, provides a measure of the intensity of small-scale rotational motions. It is particularly sensitive to viscous dissipation and numerical diffusion. Figure 6.9 displays the temporal evolution of $E(t)$ for the simulation cases.

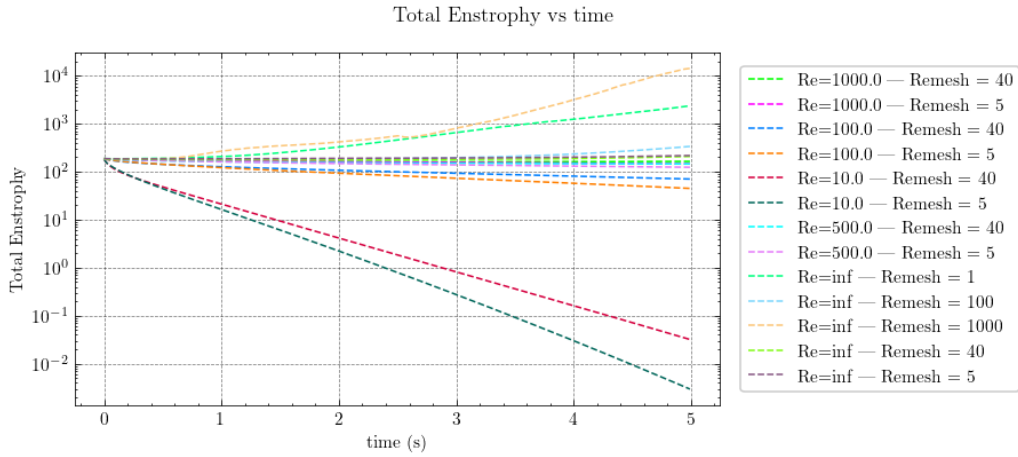


Figure 6.9: Time evolution of enstrophy, E .

Noticeable differences are observed in high Reynolds number regimes and in cases with inviscid behavior, where remeshing frequency has a pronounced impact. In particular, inviscid calculations display a persistent growth in enstrophy. This could stem from the use of fixed time increments in conjunction with the inherent fact that Euler equations blow-up (become numerically stiff).

Kinetic Energy

The total kinetic energy is computed as $K = \frac{1}{2} \int_{\Omega} \rho \|\mathbf{u}\|^2 dV$. This quantity reflects the energetic state of the system and is influenced by both convective transport and viscous dissipation. Figure 6.11 presents $K(t)$ as computed in the Particle Mesh while 6.10 as computed on the particles. In cases with lower Reynolds numbers, a more rapid decay of kinetic energy is observed due to enhanced viscous effects, whereas inviscid cases maintain higher energy levels. Variations with remeshing frequency are also evident.

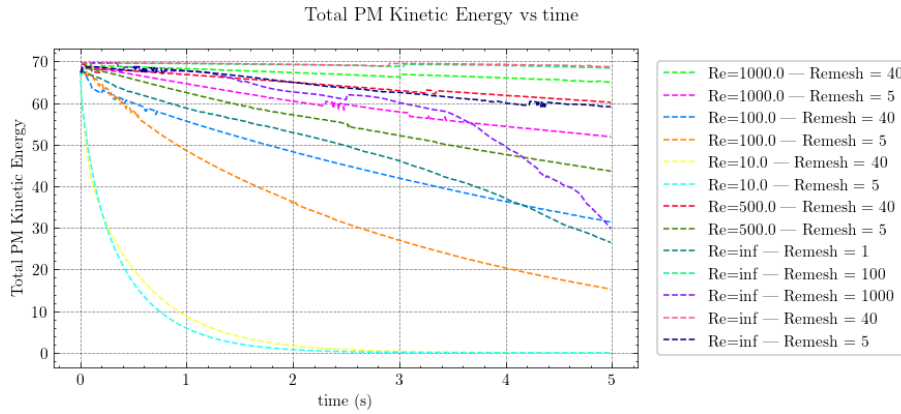


Figure 6.10: Time evolution of kinetic energy, K , as computed on the particle mesh.

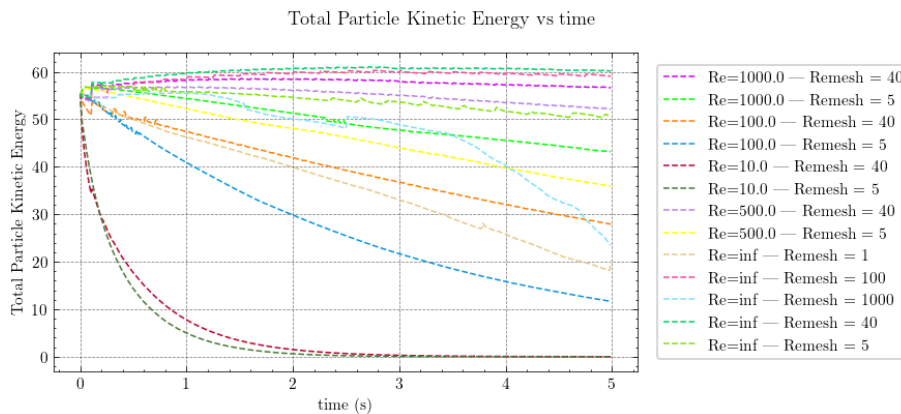


Figure 6.11: Time evolution of kinetic energy, K , as computed on the particles.

The decay of kinetic energy in incompressible flows is strongly tied to the flow

enstrophy as:

$$\frac{d}{dt} \left(\frac{1}{2} \int_{\Omega} |\mathbf{u}|^2 \right) = -\nu E(\mathbf{u}).$$

The behavior observed in 6.11 and 6.10 is consistent with this result, as we see cases exhibit K decay in proportion to their viscosity. The difference between the kinetic energy stored in the Particles and in the Particle Mesh lies in the fact that since we use a vorticity formulation of the equations, regions with zero vorticity are not discretized. This means that particles only account for the solenoidal part of the velocity field.

Maximum Vorticity Divergence

Although the vorticity field in an incompressible flow should ideally be divergence-free, numerical discretization may induce slight deviations. The maximum divergence is given by

$$\delta_{\omega} = \max_{\mathbf{x} \in \Omega} |\nabla \cdot \boldsymbol{\omega}(\mathbf{x})|.$$

This metric is critical for assessing how well the numerical scheme maintains the divergence-free condition of the vorticity, which is a fundamental property of incompressible flows. Figure 6.12 shows the evolution of $\delta_{\omega}(t)$.

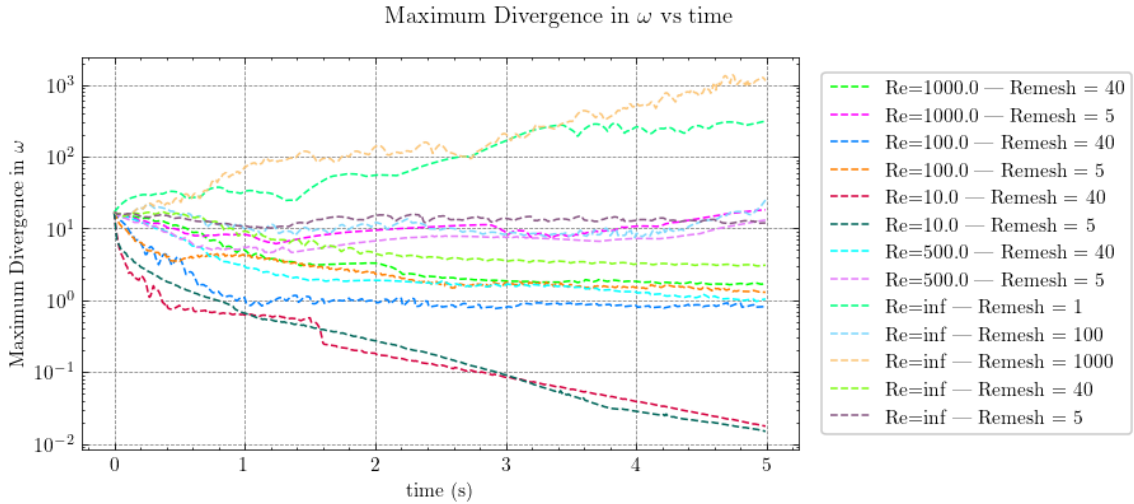


Figure 6.12: Time evolution of the maximum vorticity divergence, δ_{ω} .

In general, we observe that the divergence of ω gets greater as we increase the Reynolds number (decrease viscosity). This figure showcases that in many cases, the underlying assumption we made in the formulation is not only invalid but can also become numerically significant.

Effect of Vorticity Correction

Below, we present a comparative analysis of flow simulations both with and without vorticity correction, a numerical scheme designed to reduce the divergence of ω . While this approach is intended to mitigate the accumulation of $\nabla \cdot \omega$, it inherently struggles with high-wavenumber modes, where most of the spurious divergence emerges. Figure 6.13 illustrates the temporal evolution of the maximum divergence δ_ω , whereas Figure 6.14 displays the corresponding evolution of kinetic energy K , as evaluated on the particle mesh.

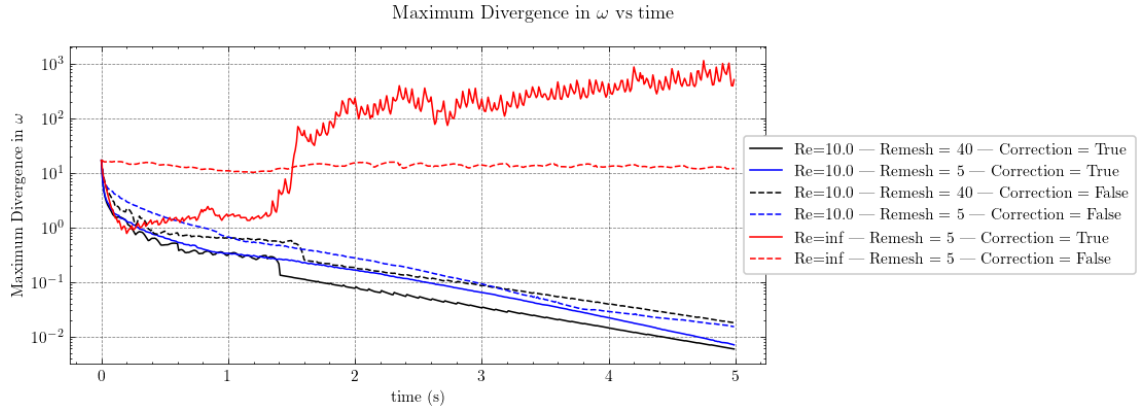


Figure 6.13: Time evolution of the maximum vorticity divergence δ_ω , including cases where we correct the vorticity.

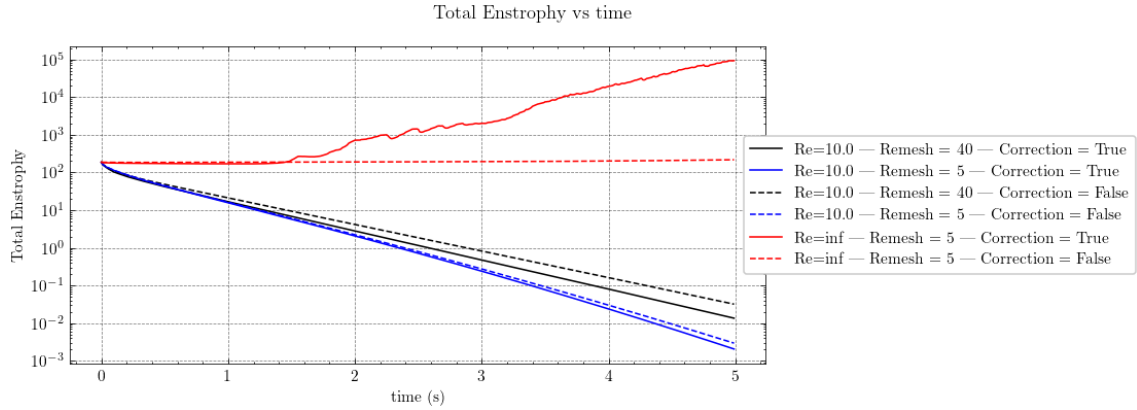


Figure 6.14: Time evolution of enstrophy E , as computed on the particles, including cases where we correct the vorticity.

From these results, one observes that for the higher viscosity regime, exemplified here by $Re = 10$, the vorticity correction is notably effective: it reduces the maximum divergence to approximately half an order of magnitude below the uncorrected cases. However, in the inviscid scenario ($Re = \infty$), the initially favorable behavior of the correction eventually gives way to numerical instabilities. These instabilities

manifest in a surge of kinetic energy (and enstrophy similarly) over time, signaling that the correction—despite its early success—can significantly disturb the flow dynamics in the absence of diffusive damping. Consequently, although vorticity correction holds promise, as implemented it is not robust.

6.3 Validation of Pressure Field Calculation

6.3.1 The Lamb-Oseen Vortex

The pressure field calculation is validated using the Lamb-Oseen vortex. In cylindrical coordinates (r, θ, z) , the Lamb-Oseen vortex is characterized by the sole nonzero velocity component

$$u_\theta(r, t) = \frac{\Gamma}{2\pi r} \left[1 - \exp\left(-\frac{r^2}{4\nu t}\right) \right],$$

with $u_r = u_z = 0$. Here, Γ denotes the circulation and ν is the kinematic viscosity.

For an incompressible viscous flow, the two-dimensional Navier–Stokes equations in polar coordinates are

$$\frac{\partial u_r}{\partial t} + u_r \frac{\partial u_r}{\partial r} - \frac{u_\theta^2}{r} = -\frac{1}{\rho} \frac{\partial p}{\partial r} + \nu \left[\frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial u_r}{\partial r} \right) - \frac{u_r}{r^2} \right], \quad (6.28)$$

$$\frac{\partial u_\theta}{\partial t} + u_r \frac{\partial u_\theta}{\partial r} + \frac{u_r u_\theta}{r} = -\frac{1}{\rho r} \frac{\partial p}{\partial \theta} + \nu \left[\frac{1}{r} \frac{\partial}{\partial r} \left(r \frac{\partial u_\theta}{\partial r} \right) - \frac{u_\theta}{r^2} \right], \quad (6.29)$$

$$\frac{1}{r} \frac{\partial}{\partial r} (r u_r) + \frac{1}{r} \frac{\partial u_\theta}{\partial \theta} = 0. \quad (6.30)$$

Since $u_r \equiv 0$ and the flow is axisymmetric, the radial momentum equation reduces to

$$-\frac{u_\theta^2}{r} = -\frac{1}{\rho} \frac{\partial p}{\partial r}, \quad (6.31)$$

or equivalently

$$\frac{\partial p}{\partial r} = \rho \frac{u_\theta^2}{r}. \quad (6.32)$$

Although the flow is time-dependent through $u_\theta(r, t)$, the unsteady term does not contribute to the radial balance due to the vanishing of u_r .

Substitution of the expression for $u_\theta(r, t)$ into equation (6.32) yields

$$\frac{\partial p}{\partial r} = \rho \frac{1}{r} \left(\frac{\Gamma}{2\pi r} \left[1 - \exp\left(-\frac{r^2}{4\nu t}\right) \right] \right)^2 = \rho \frac{\Gamma^2}{4\pi^2 r^3} \left[1 - \exp\left(-\frac{r^2}{4\nu t}\right) \right]^2. \quad (6.33)$$

Integrating equation (6.33) with respect to r gives

$$p(r, t) = p_\infty(t) + \rho \frac{\Gamma^2}{4\pi^2} \int_r^\infty \frac{1}{r'^3} \left[1 - \exp\left(-\frac{r'^2}{4\nu t}\right) \right]^2 dr', \quad (6.34)$$

where $p_\infty(t)$ denotes the pressure at infinity.

Introducing the substitution

$$u = \frac{r'^2}{4\nu t}, \quad \text{so that} \quad r' = 2\sqrt{\nu t u}, \quad dr' = \frac{\sqrt{\nu t}}{\sqrt{u}} du,$$

and noting that

$$\frac{1}{r'^3} = \frac{1}{(2\sqrt{\nu t u})^3} = \frac{1}{8(\nu t)^{3/2} u^{3/2}},$$

the lower limit of integration transforms to

$$u_0 = \frac{r^2}{4\nu t}.$$

Accordingly, the integral in equation (6.34) becomes

$$\begin{aligned} I(r, t) &= \int_r^\infty \frac{1}{r'^3} \left[1 - \exp\left(-\frac{r'^2}{4\nu t}\right) \right]^2 dr' \\ &= \frac{1}{8(\nu t)^{3/2}} \int_{u_0}^\infty \frac{(1 - e^{-u})^2}{u^{3/2}} \frac{\sqrt{\nu t}}{\sqrt{u}} du \\ &= \frac{1}{8\nu t} \int_{u_0}^\infty \frac{(1 - e^{-u})^2}{u^2} du. \end{aligned} \quad (6.35)$$

Thus, the pressure field is expressed as

$$p(r, t) = p_\infty(t) + \rho \frac{\Gamma^2}{32\pi^2\nu t} \int_{u_0}^\infty \frac{(1 - e^{-u})^2}{u^2} du. \quad (6.36)$$

Evaluation of the Integral

The squared term expands as

$$(1 - e^{-u})^2 = 1 - 2e^{-u} + e^{-2u},$$

which permits the decomposition

$$\int_{u_0}^\infty \frac{(1 - e^{-u})^2}{u^2} du = \int_{u_0}^\infty \frac{1}{u^2} du - 2 \int_{u_0}^\infty \frac{e^{-u}}{u^2} du + \int_{u_0}^\infty \frac{e^{-2u}}{u^2} du. \quad (6.37)$$

The first integral is elementary:

$$\int_{u_0}^{\infty} \frac{1}{u^2} du = \frac{1}{u_0}.$$

For the remaining terms, the following relation is used:

$$\int_{u_0}^{\infty} \frac{e^{-au}}{u^2} du = \frac{e^{-au_0}}{u_0} + a E_1(au_0),$$

where the exponential integral $E_1(z)$ is defined as

$$E_1(z) = \int_z^{\infty} \frac{e^{-s}}{s} ds.$$

Thus, for $a = 1$ and $a = 2$,

$$\int_{u_0}^{\infty} \frac{e^{-u}}{u^2} du = \frac{e^{-u_0}}{u_0} + E_1(u_0), \quad (6.38)$$

$$\int_{u_0}^{\infty} \frac{e^{-2u}}{u^2} du = \frac{e^{-2u_0}}{u_0} + 2E_1(2u_0). \quad (6.39)$$

Combining these results yields

$$\begin{aligned} \int_{u_0}^{\infty} \frac{(1 - e^{-u})^2}{u^2} du &= \frac{1}{u_0} - 2 \left[\frac{e^{-u_0}}{u_0} + E_1(u_0) \right] + \left[\frac{e^{-2u_0}}{u_0} + 2E_1(2u_0) \right] \\ &= \frac{(1 - e^{-u_0})^2}{u_0} - 2E_1(u_0) + 2E_1(2u_0). \end{aligned} \quad (6.40)$$

Final Expression for the Pressure Field

Substitution of the evaluated integral into equation (6.36) leads to the final expression for the pressure field:

$$p(r, t) = p_{\infty}(t) + \rho \left[\frac{\Gamma^2}{8\pi^2 r^2} \left(1 - \exp \left(-\frac{r^2}{4\nu t} \right) \right)^2 - \frac{\Gamma^2}{16\pi^2 \nu t} \left(E_1 \left(\frac{r^2}{4\nu t} \right) - E_1 \left(\frac{r^2}{2\nu t} \right) \right) \right]. \quad (6.41)$$

Here, the relation

$$\frac{1}{u_0} = \frac{4\nu t}{r^2}$$

has been utilized to express the first term in an equivalent form. Moreover, the exponential integral $E_1(z)$ may be identified with the upper incomplete gamma

function as

$$E_1(z) = \Gamma(0, z), \quad \text{where} \quad \Gamma(s, z) = \int_z^\infty t^{s-1} e^{-t} dt.$$

Equation (6.41) constitutes the analytical pressure field for the Lamb–Oseen vortex and serves as a benchmark for numerical pressure field calculations in viscous vortex flows.

6.3.2 Numerical Results

As a final step in validating our numerical pressure field calculation, we compare in this section the computed velocity and pressure fields obtained via the Vortex Particle Method (VPM) against the analytical Lamb–Oseen solution derived in the preceding sections.

Figure 6.15 compares the numerical pressure field to the analytical Lamb–Oseen vortex at $t = 0.2$ s, and Figure 6.16 illustrates the corresponding velocity profiles. In both cases, the results agree well in the core region. Minor discrepancies develop near the edges of the domain. These arise due to the fact that, in order to approximate free-space behavior in z , we have extended the computational domain but still apply standard Dirichlet/Neumann conditions (rather than fully periodic treatment). This was done, because our VPM implementation does not currently support periodicity. The simulation domain and mesh increments are summarized here:

$$\text{Bounds: } D_{\min} = [-0.8, -0.8, -40], \quad D_{\max} = [0.8, 0.8, 40],$$

$$\text{Spatial steps: } \Delta x = 0.02, \quad \Delta y = 0.02, \quad \Delta z = 0.2.$$

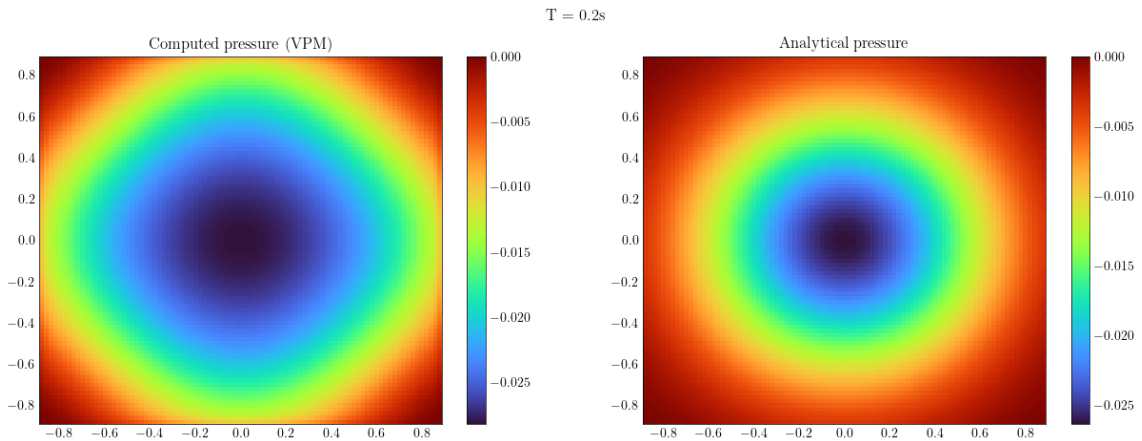


Figure 6.15: Comparison of computed and analytical pressure fields for the Lamb–Oseen vortex at $t = 0.2$ s.

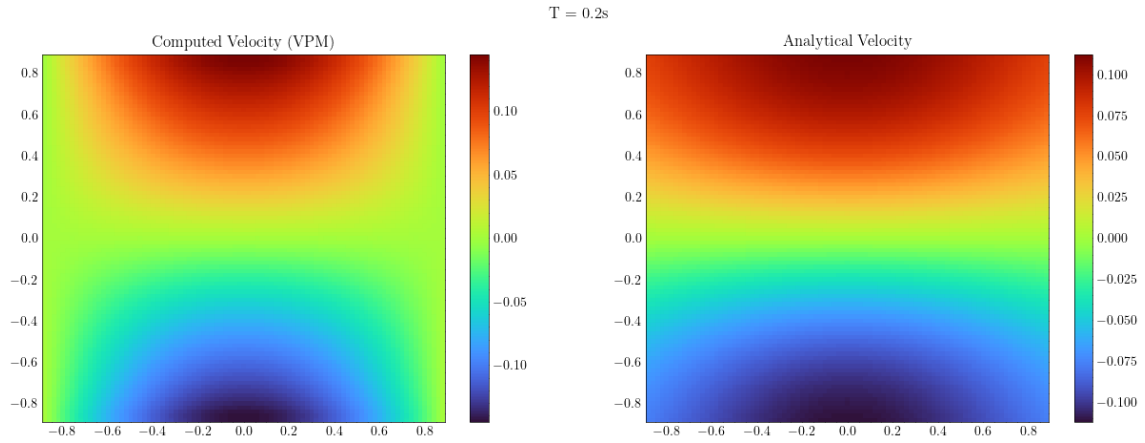


Figure 6.16: *Comparison of computed and analytical velocity fields for the Lamb–Oseen vortex at $t = 0.2$ s.*

Overall, these results corroborate that the derived pressure formulation (see (6.41)) and its numerical implementation in the VPM framework perform reliably for viscous vortex flows, subject to the caveat that finite boundaries and periodic conditions necessarily approximate the unbounded vortex domain.

Bibliography

- [1] Christopher Anderson and Claude Greengard. “On Vortex Methods”. In: *SIAM Journal on Numerical Analysis* 22.3 (1985), pp. 413–440. DOI: 10.1137/0722025. eprint: <https://doi.org/10.1137/0722025>. URL: <https://doi.org/10.1137/0722025>.
- [2] J. Thomas Beale and Andrew Majda. “Vortex methods. I: Convergence in three dimensions”. English (US). In: *Mathematics of Computation* 39.159 (July 1982), pp. 1–27. ISSN: 0025-5718. DOI: 10.1090/S0025-5718-1982-0658212-5.
- [3] J. Thomas Beale and Andrew Majda. “Vortex methods. ii: Higher order accuracy in two and three dimensions”. English (US). In: *Mathematics of Computation* 39.159 (July 1982), pp. 29–52. ISSN: 0025-5718. DOI: 10.1090/S0025-5718-1982-0658213-7.
- [4] B. L. Buzbee, G. H. Golub, and C. W. Nielson. “On Direct Methods for Solving Poisson’s Equations”. In: *SIAM Journal on Numerical Analysis* 7.4 (1970), pp. 627–656. DOI: 10.1137/0707049. eprint: <https://doi.org/10.1137/0707049>. URL: <https://doi.org/10.1137/0707049>.
- [5] Philippe Chatelain et al. “Billion vortex particle direct numerical simulations of aircraft wakes”. In: *Computer Methods in Applied Mechanics and Engineering* 197.13-16 (2008), pp. 1296–1304.
- [6] Alexandre Chorin. “Numerical Study of Slightly Viscous Flow”. In: *Journal of Fluid Mechanics* 57 (Mar. 1973), pp. 785–796. DOI: 10.1017/S0022112073002016.
- [7] Alexandre Joel Chorin and Peter S Bernard. “Discretization of a vortex sheet, with an example of roll-up”. In: *Journal of Computational Physics* 13.3 (1973), pp. 423–429. ISSN: 0021-9991. DOI: [https://doi.org/10.1016/0021-9991\(73\)90045-4](https://doi.org/10.1016/0021-9991(73)90045-4). URL: <https://www.sciencedirect.com/science/article/pii/0021999173900454>.
- [8] G.-H Cottet and Petros Koumoutsakos. *Vortex methods - theory and practice*. Mar. 2000. ISBN: 978-0-521-62186-1. DOI: 10.1017/CB09780511526442.
- [9] P. Degond and S. Mas-Gallic. “The Weighted Particle Method for Convection-Diffusion Equations. Part 1: The Case of an Isotropic Viscosity”. In: *Mathematics of Computation* 53.188 (1989), pp. 485–507. ISSN: 00255718, 10886842. URL: <http://www.jstor.org/stable/2008716> (visited on 03/25/2025).
- [10] Jeff D. Eldredge, Anthony Leonard, and Tim Colonius. “A General Deterministic Treatment of Derivatives in Particle Methods”. In: *Journal of Computational Physics* 180.2 (2002), pp. 686–709. ISSN: 0021-9991. DOI: <https://doi.org/10.1006/jcph.2002.7112>. URL: <https://www.sciencedirect.com/science/article/pii/S0021999102971123>.
- [11] R. A. Gingold and J. J. Monaghan. “Smoothed particle hydrodynamics: theory and application to non-spherical stars”. In: *Monthly Notices of the Royal*

- Astronomical Society* 181.3 (Dec. 1977), pp. 375–389. ISSN: 0035-8711. DOI: 10.1093/mnras/181.3.375. eprint: <https://academic.oup.com/mnras/article-pdf/181/3/375/3104055/mnras181-0375.pdf>. URL: <https://doi.org/10.1093/mnras/181.3.375>.
- [12] L Greengard and V Rokhlin. “A fast algorithm for particle simulations”. In: *Journal of Computational Physics* 73.2 (1987), pp. 325–348. ISSN: 0021-9991. DOI: [https://doi.org/10.1016/0021-9991\(87\)90140-9](https://doi.org/10.1016/0021-9991(87)90140-9). URL: <https://www.sciencedirect.com/science/article/pii/0021999187901409>.
 - [13] Ole H. Hald. “Convergence of Vortex Methods for Euler’s Equations. II”. In: *SIAM Journal on Numerical Analysis* 16.5 (1979), pp. 726–755. DOI: 10.1137/0716055. eprint: <https://doi.org/10.1137/0716055>. URL: <https://doi.org/10.1137/0716055>.
 - [14] W. D. Henshaw, H. O. Kreiss, and L. G. Reyna. “Smallest scale estimates for the Navier-Stokes equations for incompressible fluids”. In: *Archive for Rational Mechanics and Analysis* 112.1 (Mar. 1990), pp. 21–44. ISSN: 1432-0673. DOI: 10.1007/BF00431721. URL: <https://doi.org/10.1007/BF00431721>.
 - [15] A HUBERSON S; JOLLES. “Correction de l’erreur de projection dans les méthodes particules/maillage”. French. In: *La Recherche aérospatiale* (1990). ISSN: 0034-1223.
 - [16] R.A James. “The solution of poisson’s equation for isolated source distributions”. In: *Journal of Computational Physics* 25.2 (1977), pp. 71–93. ISSN: 0021-9991. DOI: [https://doi.org/10.1016/0021-9991\(77\)90013-4](https://doi.org/10.1016/0021-9991(77)90013-4). URL: <https://www.sciencedirect.com/science/article/pii/0021999177900134>.
 - [17] P. Koumoutsakos and A. Leonard. “High-resolution simulations of the flow around an impulsively started cylinder using vortex methods”. In: *Journal of Fluid Mechanics* 296 (1995), pp. 1–38. DOI: 10.1017/S0022112095002059.
 - [18] Robert Krasny. “A study of singularity formation in a vortex sheet by the point-vortex approximation”. In: *Journal of Fluid Mechanics* 167 (June 1986), pp. 65–93. DOI: 10.1017/S0022112086002732.
 - [19] K. Lackner. “Computation of ideal MHD equilibria”. In: *Computer Physics Communications* 12.1 (1976), pp. 33–44. ISSN: 0010-4655. DOI: [https://doi.org/10.1016/0010-4655\(76\)90008-4](https://doi.org/10.1016/0010-4655(76)90008-4). URL: <https://www.sciencedirect.com/science/article/pii/0010465576900084>.
 - [20] Sanjiva K. Lele. “Compact finite difference schemes with spectral-like resolution”. In: *Journal of Computational Physics* 103.1 (1992), pp. 16–42. ISSN: 0021-9991. DOI: [https://doi.org/10.1016/0021-9991\(92\)90324-R](https://doi.org/10.1016/0021-9991(92)90324-R). URL: <https://www.sciencedirect.com/science/article/pii/002199919290324R>.
 - [21] Chloé Mimeau and Iraj Mortazavi. “A Review of Vortex Methods and Their Applications: From Creation to Recent Advances”. In: *Fluids* 6 (Feb. 2021), p. 68. DOI: 10.3390/fluids6020068.
 - [22] J.J Monaghan. “Extrapolating B splines for interpolation”. In: *Journal of Computational Physics* 60.2 (1985), pp. 253–262. ISSN: 0021-9991. DOI: [https://doi.org/10.1016/0021-9991\(85\)90006-3](https://doi.org/10.1016/0021-9991(85)90006-3). URL: <https://www.sciencedirect.com/science/article/pii/0021999185900063>.

- [23] George Papadakis. “Development of a hybrid compressible vortex particle method and application to external problems including helicopter flows”. PhD thesis. Dec. 2014. DOI: 10.13140/RG.2.2.26480.92168.
- [24] Paul N. Swarztrauber. “The Methods of Cyclic Reduction, Fourier Analysis and the FACR Algorithm for the Discrete Solution of Poisson’s Equation on a Rectangle”. In: *SIAM Review* 19.3 (1977), pp. 490–501. DOI: 10.1137/1019071. eprint: <https://doi.org/10.1137/1019071>. URL: <https://doi.org/10.1137/1019071>.
- [25] J. Thomas Beale and Andrew Majda. “Rates of convergence for viscous splitting of the navier stokes equations”. English (US). In: *Mathematics of Computation* 37.156 (Oct. 1981), pp. 951–958. ISSN: 0025-5718. DOI: 10.1090/S0025-5718-1981-0628693-0.
- [26] SG Voutsinas, MA Belessis, and KG Rados. “Investigation of the yawed operation of wind turbines by means of a vortex particle method”. In: *AGARD CONFERENCE PROCEEDINGS AGARD CP*. AGARD Neuilly sur Seine, France. 1995, pp. 11–11.
- [27] Spyros G Voutsinas and Dimitris G Triantos. “Aeroacoustics of full helicopter configurations using vortex particle flow approximation”. In: *2Q00MW (M* (1999), p. 175.

Developed Software

A crucial component of this thesis is the extensive development and refinement of a high-performance simulation code that underpins the vortex particle method (VPM) and its augmentation with new features. The codebase, originally obtained from the Fluid Section Laboratory, was designed for parallel computations using MPI and OpenMP, ensuring exceptional performance in large-scale simulations. Building on this solid foundation, significant modifications and enhancements were undertaken to modernize, extend, and optimize the software in accordance with contemporary high-performance computing (HPC) practices.

The original FORTRAN code was comprehensively refactored to incorporate Modern FORTRAN features, including the introduction of advanced Fortran types that immensely simplify the code and improve its overall structure. The code was aggressively modularized so that each module has clearly defined responsibilities and features. This modular design not only facilitates seamless interconnection among components but also ensures that the code now produces no warnings when compiling—a result of zero-tolerance compilation under GFortran, Intel, and LLVM toolchains—and adheres strictly to best practices. Rigorous testing under the most scrutinous compiler settings further validates its proper implementation, allowing new users and developers to delve into the codebase with enhanced clarity and confidence.

For the simulation features of the VPM, new routines were developed to enable vorticity correction and pressure recovery. These enhancements have significantly improved the method’s capability to accurately compute the pressure field and implement robust correction strategies for the vorticity distribution, thereby addressing key challenges in capturing the dynamics of incompressible flows.

Furthermore, the original code performed vector field calculations (such as the calculation of derivatives, divergence, curl, etc.) in an ad hoc and nonparallel manner, with similar operations dispersed throughout the code. In response, a dedicated library for generalized vector field operators was developed to commonize the implementation of these calculations. This new library supports various finite-difference stencils and provides standardized interfaces that not only enhance modularity but also lay the groundwork for future improvements involving distributed, parallel calculations. The modular approach ensures that subsequent enhancements can be seamlessly integrated, significantly boosting overall performance.

To address previous bottlenecks in file input/output, support for HDF5 files was implemented based on the h5fortran library. This upgrade has greatly improved file I/O performance, allowing for efficient saving and loading of simulation data and thereby eliminating a major bottleneck encountered in earlier versions of the code.

The build process has been completely automated using CMake, resulting in a cross-platform solution that supports a variety of systems. Robust C bindings were exposed to transform the simulation code into a dynamic library with standardized algorithmic interfaces, allowing the FORTRAN routines to be seamlessly integrated into diverse software ecosystems. This approach facilitates future extensions in FORTRAN or other languages, ensuring that the VPM code remains versatile and accessible.

In addition, the modular refactoring of the code has enabled the straightforward integration of new solver modules. Previously, the simulation supported only Intel's Fast Poisson Solver; however, the enhanced code now includes additional solver options such as NCAR's Mudpack and Fishpack. Mudpack offers a robust multigrid solver framework, while Fishpack provides a modernized implementation of NCAR's Fishpack90. Thanks to the modular design, these solvers can be interchanged with minimal effort, and users can easily enable or disable domain discretization for parallel computations across all solver modules, ensuring compatibility and optimal performance tailored to their simulation requirements.

Recognizing the need for rapid prototyping and enhanced usability, a comprehensive Python wrapper and API were developed. This API manages the data flow between the Python environment and the underlying FORTRAN library, enabling users to leverage Python's interpreter for interactive simulation development. The Python module exposes both a low-level and a high-level API, thereby offering users the choice of granular control or abstracted interfaces as needed. Real-time visualization tools and parallelized execution capabilities further enhance usability, eliminating the need to recompile the entire program for each alteration—a feature. Those developments have proved immensely helpful and convenient during the development of this thesis.

The culmination of these efforts is encapsulated in the repository https://github.com/trifwn/vpm_py, where the code is packaged under the name `vpm_py`. This repository serves as a platform for continued development and collaboration, reflecting cutting-edge practices in HPC and computational fluid dynamics software. Overall, the developed software not only extends the capabilities of traditional vortex particle methods but also exemplifies a modern integration of robust, parallelized FORTRAN routines with flexible, user-friendly Python interfaces, positioning the work at the forefront of simulation software development in computational fluid dynamics.

Appendix A

Derivation of the Fourier Symbol for the Fourth-Order Second Derivative

We start with the discrete symbol defined by

$$\lambda(k) = \frac{-e^{2ik\Delta x} + 16e^{ik\Delta x} - 30 + 16e^{-ik\Delta x} - e^{-2ik\Delta x}}{12\Delta x^2}. \quad (42)$$

Step 1. Expressing in Terms of Cosines

Recall Euler's identities:

$$e^{i\theta} + e^{-i\theta} = 2\cos\theta, \quad \text{and} \quad (43)$$

$$e^{i\theta} - e^{-i\theta} = 2i\sin\theta. \quad (44)$$

Group the exponential terms in the numerator of (42) :

$$\begin{aligned} & -e^{2ik\Delta x} + 16e^{ik\Delta x} - 30 + 16e^{-ik\Delta x} - e^{-2ik\Delta x} \\ &= -\left(e^{2ik\Delta x} + e^{-2ik\Delta x}\right) + 16\left(e^{ik\Delta x} + e^{-ik\Delta x}\right) - 30. \end{aligned}$$

Using the identity $e^{i\theta} + e^{-i\theta} = 2\cos\theta$ with $\theta = k\Delta x$ and $\theta = 2k\Delta x$, we obtain

$$-\left(e^{2ik\Delta x} + e^{-2ik\Delta x}\right) + 16\left(e^{ik\Delta x} + e^{-ik\Delta x}\right) - 30 = -2\cos(2k\Delta x) + 32\cos(k\Delta x) - 30. \quad (45)$$

Thus, the symbol becomes

$$\lambda(k) = \frac{-2\cos(2k\Delta x) + 32\cos(k\Delta x) - 30}{12\Delta x^2}.$$

Step 2. Rewriting in Terms of $\sin^2\left(\frac{k\Delta x}{2}\right)$

We now use the following trigonometric identities:

1. $\cos(k\Delta x) = 1 - 2\sin^2\left(\frac{k\Delta x}{2}\right)$. (This follows from the double-angle formula for cosine.)

2. $\cos(2k\Delta x) = 1 - 2\sin^2(k\Delta x)$. (Again, a standard double-angle formula.)

3. $\sin(k\Delta x) = 2\sin\left(\frac{k\Delta x}{2}\right)\cos\left(\frac{k\Delta x}{2}\right)$, so that

$$\sin^2(k\Delta x) = 4\sin^2\left(\frac{k\Delta x}{2}\right)\cos^2\left(\frac{k\Delta x}{2}\right) = 4\sin^2\left(\frac{k\Delta x}{2}\right)\left(1 - \sin^2\left(\frac{k\Delta x}{2}\right)\right).$$

First, substitute the identity for $\cos(k\Delta x)$ into (45):

$$32\cos(k\Delta x) = 32\left(1 - 2\sin^2\left(\frac{k\Delta x}{2}\right)\right) = 32 - 64\sin^2\left(\frac{k\Delta x}{2}\right).$$

Next, to express $\cos(2k\Delta x)$ in terms of $\sin^2\left(\frac{k\Delta x}{2}\right)$, first write:

$$\cos(2k\Delta x) = 1 - 2\sin^2(k\Delta x)$$

and then substitute

$$\sin^2(k\Delta x) = 4\sin^2\left(\frac{k\Delta x}{2}\right)\left(1 - \sin^2\left(\frac{k\Delta x}{2}\right)\right).$$

Thus,

$$\cos(2k\Delta x) = 1 - 8\sin^2\left(\frac{k\Delta x}{2}\right)\left(1 - \sin^2\left(\frac{k\Delta x}{2}\right)\right).$$

Then,

$$\begin{aligned} -2\cos(2k\Delta x) &= -2\left[1 - 8\sin^2\left(\frac{k\Delta x}{2}\right)\left(1 - \sin^2\left(\frac{k\Delta x}{2}\right)\right)\right] \\ &= -2 + 16\sin^2\left(\frac{k\Delta x}{2}\right)\left(1 - \sin^2\left(\frac{k\Delta x}{2}\right)\right). \end{aligned} \quad (46)$$

Now, substitute these expressions back into (45):

$$\begin{aligned} -2\cos(2k\Delta x) + 32\cos(k\Delta x) - 30 &= \left[-2 + 16\sin^2\left(\frac{k\Delta x}{2}\right)\left(1 - \sin^2\left(\frac{k\Delta x}{2}\right)\right)\right] \\ &\quad + \left[32 - 64\sin^2\left(\frac{k\Delta x}{2}\right)\right] - 30. \end{aligned}$$

Combine the constant terms:

$$-2 + 32 - 30 = 0,$$

so that

$$\begin{aligned} -2 \cos(2k\Delta x) + 32 \cos(k\Delta x) - 30 &= 16 \sin^2\left(\frac{k\Delta x}{2}\right) \left(1 - \sin^2\left(\frac{k\Delta x}{2}\right)\right) \\ &\quad - 64 \sin^2\left(\frac{k\Delta x}{2}\right). \end{aligned} \quad (47)$$

Factor out $16 \sin^2\left(\frac{k\Delta x}{2}\right)$:

$$= 16 \sin^2\left(\frac{k\Delta x}{2}\right) \left[\left(1 - \sin^2\left(\frac{k\Delta x}{2}\right)\right) - 4\right] = 16 \sin^2\left(\frac{k\Delta x}{2}\right) \left[-3 - \sin^2\left(\frac{k\Delta x}{2}\right)\right].$$

Thus,

$$-2 \cos(2k\Delta x) + 32 \cos(k\Delta x) - 30 = -16 \sin^2\left(\frac{k\Delta x}{2}\right) \left[3 + \sin^2\left(\frac{k\Delta x}{2}\right)\right].$$

Step 3. Final Rearrangement

Recall that the symbol was given by

$$\lambda(k) = \frac{-2 \cos(2k\Delta x) + 32 \cos(k\Delta x) - 30}{12 \Delta x^2}.$$

Substitute the expression just derived:

$$\lambda(k) = \frac{-16 \sin^2\left(\frac{k\Delta x}{2}\right) \left[3 + \sin^2\left(\frac{k\Delta x}{2}\right)\right]}{12 \Delta x^2}.$$

Simplify the constant factor:

$$\frac{16}{12} = \frac{4}{3},$$

so that

$$\lambda(k) = -\frac{4}{3\Delta x^2} \sin^2\left(\frac{k\Delta x}{2}\right) \left[3 + \sin^2\left(\frac{k\Delta x}{2}\right)\right].$$

Finally, factor 3 out of the bracket:

$$3 + \sin^2\left(\frac{k\Delta x}{2}\right) = 3 \left[1 + \frac{1}{3} \sin^2\left(\frac{k\Delta x}{2}\right)\right].$$

Canceling the factor of 3 we obtain the desired result:

$$\lambda(k) = -\frac{4}{\Delta x^2} \sin^2\left(\frac{k\Delta x}{2}\right) \left[1 + \frac{1}{3} \sin^2\left(\frac{k\Delta x}{2}\right)\right]. \quad (48)$$

This completes the derivation.

Appendix B

Derivation of the Ψ Time Evolution

The time-evolved scalar field, given by Eq. (6.19), may be written as

$$\psi(r, t) = \sigma^2 \left[\frac{3}{\tau} - \frac{r^2}{\tau^2} \right] \frac{1}{[2\pi\tau]^{3/2}} \exp\left(-\frac{r^2}{2\tau}\right).$$

where $\tau = \sigma^2 + 2\nu t$ is introduced for notational convenience.

We define the vector field Ψ as

$$\Psi(\mathbf{r}, t) = \nabla\psi(\mathbf{r}, t) = \frac{\partial\psi(r, t)}{\partial r} \hat{\mathbf{r}},$$

where $\hat{\mathbf{r}} = \mathbf{r}/r$.

Defining the polynomial factor

$$P(r, t) = \frac{3}{\tau} - \frac{r^2}{\tau^2}$$

and the diffused Gaussian

$$G(r, t) = \frac{1}{[2\pi\tau]^{3/2}} \exp\left(-\frac{r^2}{2\tau}\right),$$

we have $\psi(r, t) = \sigma^2 P(r, t)G(r, t)$.

Taking the derivative with respect to r , we obtain

$$\frac{\partial\psi(r, t)}{\partial r} = \sigma^2 [P'(r, t)G(r, t) + P(r, t)G'(r, t)].$$

A straightforward differentiation yields

$$P'(r, t) = -\frac{2r}{\tau^2},$$

and, from the Gaussian,

$$G'(r, t) = -\frac{r}{\tau} G(r, t).$$

Thus,

$$\begin{aligned}\frac{\partial \psi(r, t)}{\partial r} &= \sigma^2 \left[-\frac{2r}{\tau^2} G(r, t) - \frac{r}{\tau} P(r, t) G(r, t) \right] \\ &= -\sigma^2 G(r, t) \frac{r}{\tau^2} [2 + \tau P(r, t)].\end{aligned}$$

Substituting for $P(r, t)$,

$$\tau P(r, t) = \tau \left(\frac{3}{\tau} - \frac{r^2}{\tau^2} \right) = 3 - \frac{r^2}{\tau},$$

so that

$$\frac{\partial \psi(r, t)}{\partial r} = -\sigma^2 G(r, t) \frac{r}{\tau^2} \left[5 - \frac{r^2}{\tau} \right].$$

Hence, the gradient-based vector field becomes

$$\mathbf{\Psi}(\mathbf{r}, t) = \nabla \psi(\mathbf{r}, t) = -\sigma^2 \frac{r}{\tau^2} \left(5 - \frac{r^2}{\tau} \right) G(r, t) \hat{\mathbf{r}}, \quad (49)$$

with

$$G(r, t) = \frac{1}{[2\pi\tau]^{3/2}} \exp\left(-\frac{r^2}{2\tau}\right) \quad \text{and} \quad \tau = \sigma^2 + 2\nu t.$$

Equation (49) provides the complete analytical expression for the time evolution of the vector field obtained via the gradient of the Ricker wavelet, which can be utilized for benchmarking.

Derivation of Ψ Energy Decay

We start from the gradient-based vector field

$$\mathbf{\Psi}(\mathbf{r}, t) = \nabla \psi(\mathbf{r}, t) = -\sigma^2 \frac{r}{\tau^2} \left(5 - \frac{r^2}{\tau} \right) G(r, t) \hat{\mathbf{r}}, \quad (50)$$

with

$$G(r, t) = \frac{1}{[2\pi\tau]^{3/2}} \exp\left(-\frac{r^2}{2\tau}\right) \quad \text{and} \quad \tau = \sigma^2 + 2\nu t.$$

The energy of the vector field is defined as

$$E_{\Psi}(t) = \int_{\mathbb{R}^3} \left| \mathbf{\Psi}(\mathbf{r}, t) \right|^2 dV. \quad (51)$$

Since $\Psi(\mathbf{r}, t)$ is purely radial, we can write

$$\left| \Psi(\mathbf{r}, t) \right|^2 = \sigma^4 \frac{r^2}{\tau^4} \left(5 - \frac{r^2}{\tau} \right)^2 G(r, t)^2,$$

and the volume element in spherical coordinates is $dV = 4\pi r^2 dr$. Therefore,

$$E_\Psi(t) = 4\pi \sigma^4 \frac{1}{\tau^4} \int_0^\infty r^2 \left(5 - \frac{r^2}{\tau} \right)^2 G(r, t)^2 r^2 dr. \quad (52)$$

Substituting the expression for $G(r, t)$

$$G(r, t) = \frac{1}{(2\pi\tau)^{3/2}} \exp\left(-\frac{r^2}{2\tau}\right),$$

we have

$$G(r, t)^2 = \frac{1}{(2\pi\tau)^3} \exp\left(-\frac{r^2}{\tau}\right).$$

Thus, Eq. (52) becomes

$$E_\Psi(t) = 4\pi \sigma^4 \frac{1}{\tau^4} \frac{1}{(2\pi\tau)^3} \int_0^\infty r^4 \left(5 - \frac{r^2}{\tau} \right)^2 \exp\left(-\frac{r^2}{\tau}\right) dr. \quad (53)$$

Noting that $(2\pi\tau)^3 = 8\pi^3\tau^3$, the prefactor simplifies to

$$\frac{4\pi \sigma^4}{\tau^4 (8\pi^3\tau^3)} = \frac{\sigma^4}{2\pi^2\tau^7},$$

so that

$$E_\Psi(t) = \frac{\sigma^4}{2\pi^2\tau^7} \int_0^\infty r^4 \left(5 - \frac{r^2}{\tau} \right)^2 \exp\left(-\frac{r^2}{\tau}\right) dr. \quad (54)$$

To evaluate the radial integral, we perform the change of variable

$$u = \frac{r^2}{\tau} \implies r = \sqrt{\tau u}, \quad dr = \frac{\sqrt{\tau}}{2\sqrt{u}} du.$$

Then, noting that $r^4 = (\tau u)^2 = \tau^2 u^2$ and

$$\left(5 - \frac{r^2}{\tau} \right)^2 = (5 - u)^2,$$

the integral in Eq. (54) transforms as follows:

$$\begin{aligned} \int_0^\infty r^4 \left(5 - \frac{r^2}{\tau}\right)^2 \exp\left(-\frac{r^2}{\tau}\right) dr &= \int_0^\infty \tau^2 u^2 (5 - u)^2 \exp(-u) \left(\frac{\sqrt{\tau}}{2\sqrt{u}}\right) du \\ &= \frac{\tau^{5/2}}{2} \int_0^\infty u^{3/2} (5 - u)^2 \exp(-u) du. \end{aligned}$$

Substituting back into Eq. (54) yields

$$\begin{aligned} E_\Psi(t) &= \frac{\sigma^4}{2\pi^2\tau^7} \cdot \frac{\tau^{5/2}}{2} \int_0^\infty u^{3/2} (5 - u)^2 \exp(-u) du \\ &= \frac{\sigma^4}{4\pi^2\tau^{9/2}} \int_0^\infty u^{3/2} (5 - u)^2 \exp(-u) du. \end{aligned} \quad (55)$$

Next, we evaluate the remaining integral. Expanding the square,

$$(5 - u)^2 = 25 - 10u + u^2,$$

we have

$$\begin{aligned} \int_0^\infty u^{3/2} (5 - u)^2 \exp(-u) du &= 25 \int_0^\infty u^{3/2} \exp(-u) du \\ &\quad - 10 \int_0^\infty u^{5/2} \exp(-u) du \\ &\quad + \int_0^\infty u^{7/2} \exp(-u) du. \end{aligned} \quad (56)$$

Recognizing these as Gamma integrals, where

$$\int_0^\infty u^{p-1} \exp(-u) du = \Gamma(p),$$

we identify

$$\begin{aligned} \int_0^\infty u^{3/2} \exp(-u) du &= \Gamma\left(\frac{5}{2}\right), \\ \int_0^\infty u^{5/2} \exp(-u) du &= \Gamma\left(\frac{7}{2}\right), \\ \int_0^\infty u^{7/2} \exp(-u) du &= \Gamma\left(\frac{9}{2}\right). \end{aligned} \quad (57)$$

Using the known values

$$\Gamma\left(\frac{5}{2}\right) = \frac{3\sqrt{\pi}}{4}, \quad \Gamma\left(\frac{7}{2}\right) = \frac{15\sqrt{\pi}}{8}, \quad \Gamma\left(\frac{9}{2}\right) = \frac{105\sqrt{\pi}}{16},$$

the integral becomes

$$25\frac{3\sqrt{\pi}}{4} - 10\frac{15\sqrt{\pi}}{8} + \frac{105\sqrt{\pi}}{16} = \frac{75\sqrt{\pi}}{4} - \frac{75\sqrt{\pi}}{4} + \frac{105\sqrt{\pi}}{16} = \frac{105\sqrt{\pi}}{16}.$$

Thus, Eq. (55) simplifies to

$$E_{\Psi}(t) = \frac{\sigma^4}{4\pi^2\tau^{9/2}} \cdot \frac{105\sqrt{\pi}}{16} = \frac{105\sigma^4}{64\pi^2} \cdot \frac{\sqrt{\pi}}{\tau^{9/2}}. \quad (58)$$

Since

$$\frac{\sqrt{\pi}}{\pi^2} = \frac{1}{\pi^{3/2}},$$

we finally obtain

$$E_{\Psi}(t) = \frac{105\sigma^4}{64\pi^{3/2}(\sigma^2 + 2\nu t)^{9/2}}. \quad (59)$$

Equation (59) represents the energy of the gradient-based vector field as a function of time, and it serves as a crucial benchmark for validating the accuracy of numerical diffusion solvers.